

A FOCUSED, PROJECT-GROUNDED CURRICULUM

Swift & SwiftUI for Backend Engineers

From Java & Quarkus to shipping an iOS app — the essential path

Written for Ahmet Tokalak

June 2026

Grounded in the Alltag iOS project

Contents

Preface	6
How this book is different	6
How it is organised	6
What you need	7
The Landscape: from JVM to iOS	9
No JVM, no GC at the platform level	9
The players, and how they map	9
Xcode is the whole toolchain	10
You do not own <code>main()</code>	10
The big mental shift: one device, offline, no logs	11
Corrections your mental model needs	12
Getting started costs nothing	12
Takeaways	12
Swift Fundamentals: Values, Optionals, Errors	14
<code>let</code> vs <code>var</code> , and type inference	14
Fundamental types, and value semantics	14
Optionals: the <code>null</code> problem, solved by the type system	15
Error handling: <code>throws</code> , <code>try</code> , <code>do / catch</code>	17
Takeaways	19
Types: Structs, Enums, Protocols, Generics	20
<code>struct</code> : the default type	20
<code>class</code> : reference type, used sparingly	21
<code>enum</code> : sum types with pattern matching	22
<code>protocol</code> : interfaces, then more	24
<code>extension</code> : open the type	25
Generics: reified, not erased	26
Takeaways	27
Functions, Closures, and Functional Style	28
Function syntax and argument labels	28
Closures: first-class functions	29
Higher-order functions vs Java Streams	30
KeyPaths	31
<code>@escaping</code> vs non-escaping	31
<code>@resultBuilder</code> and <code>@ViewBuilder</code>	31
Takeaways	32
Memory and Concurrency: ARC, async/await, Actors	33
Part 1 — Memory: ARC instead of a garbage collector	33
Part 2 — Concurrency: the big shift	35
Takeaways	40

The SwiftUI Model: Declarative Views and State	42
A view is a value, not a thing you keep	42
The render/diff loop	43
State primitives: where the truth lives	44
The old pattern you will still see	47
View identity and lifetime	47
Previews	47
Takeaways	48
Layout: Stacks, Modifiers, and the Layout System	49
Stacks: the three containers	49
Modifiers: order matters	50
The layout algorithm: a three-step negotiation	51
GeometryReader: use sparingly	52
Scrolling and lazy content	52
Adaptive layout: Dynamic Type and never hard-coding sizes	53
Color and light/dark via semantic tokens	53
Takeaways	54
Building Screens: Lists, Navigation, Forms, Tabs	55
The tab shell: <code>TabView</code>	55
Drill-down: <code>NavigationStack</code>	57
Lists	58
Forms and controls	59
Modals, alerts, and toolbars	60
Localization: <code>LocalizedStringKey</code> and the catalog	61
Worked example: the Settings screen	62
Takeaways	64
Architecture: State, Dependencies, and the @Observable Environment	65
MV, not MVVM	65
Dependency injection without a framework	66
The factory: <code>live()</code> vs a CDI producer	67
Where business logic lives	69
Unidirectional data flow	70
Choosing a state property wrapper	70
Mutating shared state: <code>@Bindable</code>	71
Takeaways	71
Persistence: SwiftData, the Keychain, Files, and CryptoKit	72
SwiftData: JPA without the server	72
The sandbox: which directory, and why it matters	75
CryptoKit: authenticated encryption without the footguns	76
The Keychain: the device's KeyStore	77
UserDefaults: the properties file	79
Tying it together: the Vault save path	79
Takeaways	80

The Toolchain: Xcode, SPM, XcodeGen, and the Build	83
Xcode is four tools wearing one icon	83
The <code>.xcodeproj</code> : an opaque bundle and a merge-conflict magnet	84
Why Alltag uses XcodeGen	84
Reading the real <code>project.yml</code>	85
Swift Package Manager vs. Maven/Gradle	87
The Simulator: a simulator, not an emulator	88
<code>xcodebuild</code> , <code>DerivedData</code> , and <code>Info.plist</code>	88
How to build and run Alltag, end to end	89
Takeaways	90
Testing: Swift Testing, XCTest, Previews, and CI	91
Swift Testing: the modern framework	91
XCTest: the incumbent (what Alltag uses)	93
Worked conversion: <code>FileCryptorTests</code> , both ways	93
Injecting fakes for tests	96
UI tests with <code>XCUITest</code>	97
Previews: the fast inner loop	97
Code coverage	98
CI: the real <code>ios/scripts/ci.sh</code>	98
Takeaways	99
Code Signing and Provisioning, Demystified	100
Why the chain of trust exists	100
The pieces, mapped to things you know	101
Automatic vs manual signing	103
Why Alltag defers signing — and how to flip it on	103
Common signing errors and how to read them	104
Takeaways	105
Distribution: App Store Connect, TestFlight, and Review	106
App Store Connect: the console	106
Build and upload	108
TestFlight: the staging rollout	109
App Review: the human gate	109
Release: controlling the rollout	110
Versioning: marketing version vs build number	110
Takeaways	111
Production: Privacy, Crashes, Performance, and Updates	112
Privacy: declaring what you touch	112
Crashes and observability with no server	113
Performance: Instruments and main-thread hygiene	114
Updates: shipping new versions	115
Accessibility as production quality	116
Takeaways	116

Java/Quarkus to Swift/SwiftUI Cheat Sheet	118
Language constructs	119
Concurrency	120
Build & tooling	121
Runtime, DI & frameworks	122
Idioms and Gotchas for Java Developers	124
Takeaways	129
A Study Plan	130
Before week 1: setup (about an hour)	130
The eight weeks	130
A faster track	132
A capstone worth doing	132
Takeaways	133
Curated Resources	134
Read these first	134
When you are stuck on SwiftUI	134
When you want depth	134
Tools worth installing	135
Staying current	135
For your specific app	135
A closing note	136

Preface

You are a backend engineer. You know Java well, you are productive in Quarkus, and you can reason about types, concurrency, build pipelines, and tests without anyone holding your hand. This book does not treat you like a beginner. It treats you like an expert in a *neighbouring* field who needs a fast, honest map of a new one.

The goal is narrow and practical: **take you from “I know the JVM” to “I can build, test, and ship an iOS app to the App Store.”** Not someday — on a realistic timeline, studying in your own free time.

How this book is different

Most Swift tutorials assume you have never programmed. They spend three chapters on variables and loops. You do not need that, and reading it would waste the one resource you are short on: time. So this book does three things differently.

It teaches by delta. For every new concept, the first question is “what is the closest thing you already know in Java or Quarkus?” — and the second is “where does that analogy break?” A Swift `struct` is *almost* a Java `record`, until you learn it has value semantics and mutating methods. An `@Observable` class is *almost* a CDI bean, until you see the SwiftUI render loop. The analogies get you 80% of the way in one sentence; the rest of the section is the 20% that actually matters.

It is grounded in a real app. Throughout, the running example is **Alltag** — a privacy-first iOS companion for navigating everyday German bureaucracy: an encrypted on-device document vault, an on-device LLM that decodes official letters, deadline tracking, and a multilingual UI (including right-to-left Arabic). It is a paid app with no backend: everything happens on the device. That makes it an unusually good teacher, because it exercises the parts of iOS that backend engineers find most foreign — local persistence, encryption, the Keychain, app lifecycle, signing — rather than just calling a REST API. The code in this book is the *actual* code from that project, not toy snippets.

It respects “the essence.” Swift and iOS are vast. This book deliberately omits the long tail: UIKit (the old imperative framework), Objective-C interop, Combine, the dozens of niche APIs you can look up when you need them. What remains is the load-bearing 20% that you will use every day, taught properly.

How it is organised

- **Part I — The Swift Language.** The language itself, mapped onto your Java knowledge: value vs. reference types, optionals, errors, protocols, generics, closures, and the concurrency model (ARC, `async` / `await`, actors).
- **Part II — SwiftUI.** Declarative UI: how views are pure functions of state, how state and dependencies flow, layout, real screens, app architecture, and on-device persistence with SwiftData, the Keychain, and CryptoKit.

- **Part III — Ship It.** The ecosystem: Xcode and the build system, testing and CI, the famously confusing world of code signing, and the full path through App Store Connect, TestFlight, and review to a live release.
- **Appendices.** A Java-to-Swift cheat sheet, a catalogue of gotchas that specifically trip up Java developers, a week-by-week study plan, and a curated resource list.

You can read Part I linearly, then dip into Parts II and III as you build. The appendices are meant to be kept open in a second window.

What you need

A Mac and a free copy of **Xcode** from the Mac App Store. That is enough for everything in Parts I and II and most of Part III — you can build, run in the Simulator, and write tests without paying anyone. The **Apple Developer Program** (99 € / year) is required only for the final step: running on your own iPhone and submitting to the App Store. We will be explicit about where that line is.

A note on conventions. Code you will recognise as Swift is shown in `monospace`. Boxed asides like this one flag three recurring things: a **Quarkus analogy** that ports your intuition, a **Gotcha** that catches Java developers specifically, and a plain **Note** for everything else. When the text says “see *Chapter Title*,” it means another chapter in this book.

Let us begin where every backend engineer’s mental model needs its first correction: the shape of the iOS world itself.

PART I

The Swift Language

You already know how to model a domain, manage state, and reason about types. This part maps that knowledge onto Swift — and shows you where the language thinks differently from Java.

The Landscape: from JVM to iOS

You know how a Quarkus service reaches production: `mvn package` emits a JAR or native image, a runtime (JVM or an app server) hosts your code, and a load balancer fronts it. iOS inverts almost every layer of that picture. This chapter maps the new terrain onto what you already know, and flags where the map lies.

No JVM, no GC at the platform level

Swift compiles **ahead of time to native machine code** through **LLVM** — the same backend as `clang`. There is no bytecode, no interpreter, no JIT warm-up, and no class loader. The binary that ships in your app is `arm64` instructions the iPhone CPU runs directly. The closest JVM analogy is a GraalVM native image, but that is the exception on the JVM and the only option here.

There is also no tracing garbage collector. Swift manages object lifetimes with **ARC** (Automatic Reference Counting): the compiler inserts retain/release calls at compile time, and an object is freed the instant its last strong reference drops. No background GC thread, no stop-the-world pauses, no `-Xmx`.

Quarkus analogy. Picture every build as `quarkus build --native`, every time, with no JVM fallback. You get fast launch and a small resident footprint, but you also inherit native-build realities: no runtime reflection-by-default, no hot class redefinition, no live JVM to attach a profiler to in the same way.

ARC is not free of hazards — reference cycles leak, exactly like a non-collected graph would. The full treatment, including `weak` / `unowned` and how value types sidestep the problem, is in *Memory and Concurrency: ARC, async/await, Actors*. For now, hold one fact: memory is deterministic, not collected.

The players, and how they map

JVM world	iOS world
Java (language)	Swift (language)
JDK / <code>java.*</code>	SDK frameworks (Foundation ...)
A web/UI framework	SwiftUI
Maven / Gradle	SPM + Xcode project (XcodeGen)
JUnit	Swift Testing / XCTest
CDI / Spring DI	SwiftUI <code>Environment</code>
App server hosts JAR	The OS hosts your <code>.app</code>
<code>public static main</code>	<code>@main App</code>

Swift is the language: statically typed, with generics, protocols (think interfaces with more power), value types, and first-class closures. It will feel like a denser, more value-oriented Java. **SwiftUI** is the declarative UI framework — roughly the role a server-side template engine or a frontend framework plays, except it renders native widgets and owns the render loop.

The **SDK frameworks** are your standard library plus platform APIs, the equivalent of the JDK. **Foundation** gives you `Data`, `URL`, `Date`, `FileManager`, JSON coding — the `java.*/javax.*` workhorses. Specialized frameworks layer on top: **CryptoKit** for AES-GCM (you will see it below), **SwiftData** for persistence, **CryptoKit** + **Keychain** for key storage. Unlike the JDK, these are versioned with the OS, not shipped inside your artifact.

Xcode is the whole toolchain

In the JVM world your tools are separable: IntelliJ for editing, Maven for builds, a separate JDK, a separate app server, JUnit on the classpath. **Xcode** collapses all of that into one application: editor, the Swift compiler and build system, the **Simulator**, the debugger, Instruments (the profiler), and the signing machinery. It is IntelliJ + Maven + the runtime + the profiler in a single window.

That bundling is the first big adjustment. You do not pick your build tool; the project is an Xcode build. In this repo we tame that with **XcodeGen**: the `.xcodeproj` is generated from a declarative `project.yml` and git-ignored, so the YAML is the source of truth — closer to a `pom.xml` than to the opaque, merge-conflict-prone Xcode project file. You will see this in *The Toolchain: Xcode, SPM, XcodeGen, and the Build*.

```
cd ios
xcodegen generate      # project.yml -> Alltag.xcodeproj
open Alltag.xcodeproj
```

The **Simulator** runs the same `arm64` build on your Mac, fast and free, with no code signing. It is excellent for development but it is not a device: it has no real camera, no Secure Enclave, no cellular, and different performance characteristics. A physical iPhone requires **code signing** even for a debug build — covered in *Code Signing and Provisioning, Demystified*.

You do not own main()

A Quarkus app, however thin, ultimately has an entry point the container calls. On iOS the OS hosts your app the way an app server hosts your code, but the contract is stricter: **you do not write `main()`**. The system launches your process, and your job is to hand it a single type marked `@main` that conforms to the `App` protocol. That type's `body` describes the window and the first view. Here is Alltag's actual entry point:

```

@main
struct AlltagApp: App {
    @State private var env = AppEnvironment.live()

    var body: some Scene {
        WindowGroup {
            RootView()
                .environment(env)
        }
    }
}

```

Read it as: build the dependency container once (`AppEnvironment.live()`), inject it into the SwiftUI **environment** — the framework’s DI mechanism, analogous to a CDI `BeanManager` you populate at startup — and mount `RootView` in a window. There is no `psvm`, no servlet, no `@ApplicationScoped` producer. `@main` is the only entry point, and the framework drives everything after it.

`RootView` is the concrete shell: a five-tab `TabView` that is the app’s whole top-level navigation.

```

struct RootView: View {
    @Environment(AppEnvironment.self) private var env
    @State private var selection: AppTab = .home

    var body: some View {
        TabView(selection: $selection) {
            ForEach(AppTab.allCases) { tab in
                screen(for: tab)
                    .tabItem {
                        Label(tab.titleKey,
                            systemImage: tab.systemImage)
                    }
                    .tag(tab)
            }
        }
        .tint(AppColor.primary)
        .environment(\.locale, env.language.locale)
    }
}

```

The five tabs — **Home · Docs · Decode · Dates · Settings** — are modeled as a plain `enum AppTab` so the set is testable and the ordering explicit, with Decode deliberately in the center. You will dissect views and tabs in *Building Screens: Lists, Navigation, Forms, Tabs*.

The big mental shift: one device, offline, no logs

Here is the inversion that matters most for a backend engineer. Your code no longer runs on a fleet of servers you administer. It runs on **one device you do not control**, in someone’s pocket, frequently **offline**, with **no server logs** to grep when something breaks. There is no SSH, no `kubectl exec`, no central database, no dashboard of your users’ state. Distribution is funneled through a single channel — the **App Store** — and Apple reviews every release.

Alltag embodies this by design. It is a privacy-first companion for German bureaucracy with **no backend at all**: documents are stored encrypted on-device, and even the AI runs locally (an on-

device LLM — see the brief). The `AppEnvironment` container holds persistence, language, theme, and the LLM facade, and pointedly *no* network service and *no* entitlement service — it is a **paid app**, unlocked at install, with nothing to phone home about.

Note. Alltag is a *paid* app: a single upfront App Store price, no subscriptions, no in-app purchases, nothing gated at runtime. That is a product choice, not a platform requirement — but it reinforces the “no backend” posture this book leans on.

This changes engineering priorities you take for granted. Observability shifts from server logs to opt-in crash reports and on-device diagnostics. Resilience means surviving with no network, not retrying a downstream. Security moves from “protect the server” to “protect data at rest on a possibly-stolen phone” — hence AES-GCM and the Keychain, in *Persistence: SwiftData, the Keychain, Files, and CryptoKit*.

Corrections your mental model needs

- **Native, not bytecode.** No JVM ships with your app; the binary is machine code. No write-once-run-anywhere — you build per platform.
- **ARC, not GC.** Memory is reclaimed deterministically at the last release; cycles still leak.
- **Value types are everywhere.** `struct` and `enum` are copied, not shared by reference. Much of the standard library (`String` , `Array` , `Int`) is a value type. This is the single biggest day-to-day difference from Java; *Types* covers it.
- **One IDE, one build.** Xcode is editor + build + Simulator + profiler. You configure, not assemble, the toolchain.
- **You do not own `main()`.** `@main App` is your entry point; the OS hosts and drives your process.
- **No backend by default.** State lives on the device. No server logs, often offline, App Store is the only door out.
- **DI without a container framework.** SwiftUI’s `Environment` is the idiomatic injection mechanism; there is no Spring/CDI bean graph scanning your classpath.

Getting started costs nothing

To begin you need exactly one thing: **Xcode**, free from the Mac App Store. It bundles the Swift compiler, the SDKs, and the Simulator, so you can build and run Alltag on a simulated iPhone without an Apple Developer account or any signing setup — the repo even builds with `CODE_SIGNING_ALLOWED=NO` for exactly this reason. A paid account and signing enter the picture only when you target a physical device or ship to the store. The full setup is in *The Toolchain: Xcode, SPM, XcodeGen, and the Build*.

Takeaways

- Swift compiles AOT to native code via LLVM; there is no JVM and no platform GC — **ARC** reclaims memory deterministically.

- The players map cleanly: Swift vs. Java, SDK frameworks vs. JDK, SwiftUI as the UI framework, Xcode as IntelliJ + Maven + runtime + profiler in one.
- You do not write `main()` ; a single `@main App` is the entry point and the OS hosts your process, like an app server hosting a JAR but stricter.
- The defining shift: your code runs on one uncontrolled, often-offline device with no server logs — Alltag's backend-less, on-device, paid design makes that concrete.
- All you need to start is Xcode from the Mac App Store; signing is only for devices and the store.

Next: *Swift Fundamentals: Values, Optionals, Errors* digs into the language itself — `let` / `var`, value semantics, and how optionals and `throws` replace `null` and checked exceptions.

Swift Fundamentals: Values, Optionals, Errors

Three things in Swift will reshape your reflexes the most: bindings are immutable by default, types are values that get copied, and the two things you fight hardest in Java — `null` and checked exceptions — are replaced by language features that the compiler enforces. We ground every point in real Alltag code.

let vs var, and type inference

`let` is an immutable binding, `var` a mutable one. The distinction is `final` in Java, except it is the *default* you reach for and the compiler nudges you toward — declare with `let` unless you mutate, and Xcode warns when a `var` is never reassigned.

```
let storageKey = "alltag.language" // immutable
var selection = AppTab.home        // reassignable
```

Types are **inferred** from the initializer, like Java's `var`, but inference is pervasive and idiomatic, not a last resort. Annotate when you want the type to be explicit at a boundary or when there is no initializer:

```
let directory: URL // type stated, no value yet
private(set) var language: AppLanguage
```

`private(set)` exposes a property for reading but restricts writes to the declaring type — the equivalent of a public getter with a private setter, in one keyword. `LanguageStore` uses it so views can read `language` but only `select(_:)` can change it.

Fundamental types, and value semantics

`Int`, `Double`, `Bool`, `String` are the everyday types. The shock for a Java developer: these are **value types**, not reference types, and there are no primitive/boxed pairs. `Int` is the type — no `int` vs `Integer`, no autoboxing, no `==` vs `.equals()` trap. `String` is a value type too: assigning or passing it gives you an independent copy (copy-on-write under the hood, so it is cheap until mutated).

```
var a = "Behörde"
var b = a // b is a copy
b.append("n") // mutating b does not touch a
// a == "Behörde", b == "Behörden"
```

This is your **first contact with value vs reference**. In Swift, `struct` and `enum` are value types (copied on assignment); only `class` (and actors) are reference types (shared by pointer, ARC managed). Most of Alltag's small models are values:

```
struct FileCryptor { /* ... */ }    // copied
enum AppLanguage: String { case de, en, /* ... */ }
```

while the long-lived, shared services are classes:

```
@Observable
final class LanguageStore { /* ... */ }
```

Quarkus analogy. A Swift `struct` behaves like a JPA `@Embeddable` or a Java `record` used purely by value: copy it and the copies are independent. A `class` behaves like a normal entity/bean: a shared identity passed by reference. The default in Swift is *value*; in Java it is *reference*. Full treatment in *Types: Structs, Enums, Protocols, Generics*.

`final` on a class forbids subclassing (and lets the compiler devirtualize) — the inverse of Java, where classes are open until you write `final`. Idiomatic Swift marks classes `final` unless inheritance is intended.

Optionals: the `null` problem, solved by the type system

This is the single most important section. Swift has no implicit `null`. A reference or value type **cannot** be absent unless its type explicitly permits it, and the compiler refuses to let you use a possibly-absent value without handling the absence. It is Java’s `Optional<T>`, but woven into the type system rather than a wrapper class you remember to use.

Declaration: `T?`

`T?` is sugar for `Optional<T>`, an enum with cases `.some(T)` and `.none`. A non-optional `String` is *guaranteed* to hold a string; a `String?` may hold a string or `nil`.

```
let raw: String? = defaults.string(forKey: storageKey)
```

`UserDefaults.string(forKey:)` returns `String?` because the key may be missing — the API encodes “might not be there” in the type, where a Java API would return `null` and hope you check.

Optional binding: `if let` / `guard let`

You cannot use a `String?` where a `String` is required; you must first **unwrap** it. `if let` binds the value to a name inside a block only when it is non-`nil`. This is exactly how `LanguageStore` reads its persisted setting:

```
init(defaults: UserDefaults = .standard) {
    self.defaults = defaults
    if let raw = defaults.string(forKey: Self.storageKey),
        let stored = AppLanguage(rawValue: raw) {
        language = stored
    } else {
        language = .default
    }
}
```

Two optionals are unwrapped in one `if let`, comma-separated and short-circuiting: read the stored string, *and* parse it into an `AppLanguage` (the failable `init(rawValue:)` also returns an optional). Only if both succeed do we adopt the stored value; otherwise fall back to `.default`. There is no `null` check, no NPE-prone path — the `else` branch is mandatory because `language` must end up non-nil.

`guard let` is the early-exit twin: unwrap, or leave the scope now. The unwrapped binding stays in scope *after* the `guard`, which keeps the happy path unindented. `FileCryptor.encrypt` uses it:

```
func encrypt(_ plaintext: Data,
             using key: SymmetricKey) throws -> Data {
    let sealed = try AES.GCM.seal(plaintext, using: key)
    guard let combined = sealed.combined else {
        throw FileCryptorError.sealingFailed
    }
    return combined           // `combined` is non-optional here
}
```

`sealed.combined` is `Data?`; the `guard` converts “nil means failure” into a thrown error and gives the rest of the function a plain `Data`. Compare to Java’s `Optional.orElseThrow()`, but the compiler tracks the unwrapped binding afterward.

Quarkus analogy. `Optional<T>` in Java is opt-in and only conventionally non-null at the boundary; nothing stops a field from being `null`. In Swift, *every* type is non-optional unless marked `?`, and the compiler enforces unwrapping. You spend far less time defending against `null` because the language forbids it by default.

Nil-coalescing ??, chaining ?., force-unwrap !

`??` supplies a default when the optional is nil — Java’s `Optional.orElse`:

```
let lang = AppLanguage(rawValue: raw) ?? .default
```

Optional chaining `?.` calls a member only when the receiver is non-nil, yielding an optional result — Java’s `map` / `flatMap` chain, but as terse punctuation:

```
let id: String? = record?.id           // nil if record is nil
let count = record?.id.count ?? 0      // chain + default
```

Force-unwrap `!` asserts “this is not nil” and **traps (crashes)** if it is — the moral equivalent of `Optional.get()` on an empty optional, or dereferencing a `null`. Avoid it in normal flow. It is accept-

able only when absence is a genuine programmer error you want to surface loudly, or for resources you know exist (a bundled asset). Reach for `if let` / `guard let` / `??` first.

Implicitly-unwrapped optionals (`T!`) are a third form: a value declared optional but auto-unwrapped on every use, trapping if `nil`. They exist mainly for two-phase initialization and legacy Objective-C interop (Interface Builder outlets). In modern, `@Observable`, code you will rarely write one — prefer a real optional or a non-optional initialized in `init`. Treat `T!` as a smell unless you have a specific interop reason.

Error handling: throws, try, do/catch

Swift errors look like exceptions but follow different rules. A function that can fail is marked `throws`; a call to it must be prefixed with `try`; and you handle errors with `do` / `catch`.

```
do {
    let bytes = try store.load(id: docID)
    show(bytes)
} catch {
    log("load failed: \(error)") // `error` is implicit
}
```

`EncryptedFileStore.save` threads `try` through several failable steps — key retrieval, encryption, the disk write — and lets any of them propagate:

```
@discardableResult
func save(_ data: Data, id: String) throws -> URL {
    let key = try keyStore.symmetricKey()
    let ciphertext = try cryptor.encrypt(data, using: key)
    let url = fileURL(for: id)
    try ciphertext.write(
        to: url, options: [.atomic, .completeFileProtection])
    return url
}
```

`save` is itself `throws`, so it does not catch — it lets the caller decide. That is the everyday pattern: annotate `throws`, sprinkle `try`, handle at the boundary that can do something about it. `@discardableResult` silences the warning you would otherwise get for ignoring the returned `URL` — the inverse of Java's `@CheckReturnValue`.

The Error protocol, and untyped throws

Anything thrown conforms to the **Error** protocol — usually a plain `enum`, which models a closed set of failures with zero ceremony:

```
enum FileCryptorError: Error {
    case sealingFailed
}
```

Here is the key delta from Java. A Swift function signature says *that* it throws, not *what* it throws: pre-Swift 6 `throws` is **untyped**, with no `throws SomeException` clause and **no exception-type hierarchy in the signature**. The `catch` site receives an opaque `Error` you pattern-match or downcast.

There is no checked/unchecked split either — there is one error channel, and the compiler enforces only that you mark `throws` and write `try`. (Swift 6 adds opt-in *typed* throws, but the untyped default remains idiomatic; do not reach for typed throws reflexively.)

Gotcha. Coming from Java you will hunt for the exception type in the signature. It is not there. `func load(id:) throws -> Data` tells you it can fail, full stop. Document the failure modes in a doc comment; the type system will not. This is closer to Java *unchecked* exceptions in information content, but *checked* in that `try` and handling are mandatory.

`try?`, `try!`, and the `live()` fallback

Two variants trade a `do / catch` for an expression:

- `try?` turns a throw into `nil`, giving you an optional result — “I do not care why it failed, just whether it did.”
- `try!` asserts the call will not throw and **traps** if it does — the error-handling sibling of force-unwrap.

`AppEnvironment.live()` shows the disciplined use of both a `do / catch` and a `try!` last-resort fallback:

```
static func live() -> AppEnvironment {
    let llm = makeLLM()
    do {
        return AppEnvironment(
            persistence: try PersistenceController(), llm: llm)
    } catch {
        assertionFailure(
            "Persistent store unavailable: \(error)")
        return AppEnvironment(
            persistence: try! PersistenceController(
                inMemory: true), llm: llm)
    }
}
```

The on-disk store is attempted with `try`; if it throws, we log via `assertionFailure` (which traps in debug, no-ops in release) and fall back to an in-memory store with `try!` — because an in-memory store realistically cannot fail, and if it somehow did there is no recoverable app state, so trapping is acceptable. `try!` is a deliberate, documented assertion here, not laziness.

Note. `assertionFailure` traps in debug builds but is a no-op in release — so this code crashes loudly in development yet degrades to the in-memory store in production. That is the on-device, no-server-logs posture from *The Landscape*: fail visibly while building, survive in the user’s hands.

`defer`: guaranteed cleanup

`defer` schedules a block to run when the current scope exits, by any path — return, thrown error, or fall-through. It is Java’s `finally`, decoupled from a `try` block and placed next to the resource it cleans up:

```
func process(_ url: URL) throws -> Data {
    let handle = try FileHandle(forReadingFrom: url)
    defer { try? handle.close() } // runs on every exit
    return try handle.readToEnd() ?? Data()
}
```

Multiple `defer`s run in reverse order. Use it for symmetric cleanup (close, unlock, restore state) so the teardown cannot be skipped by an early `throw`.

Result<Success, Failure>

`Result` reifies “value or error” as a value you can store and pass around — the explicit counterpart to `throws`, useful for async callbacks, caching an outcome, or batching:

```
let outcome: Result<Data, Error>
outcome = Result { try store.load(id: docID) }

switch outcome {
case .success(let data): show(data)
case .failure(let error): log("\(error)")
}
```

`Result { ... }` captures a throwing call as a value; `get()` converts it back, re-throwing. In modern Alltag the throwing function plus `async` / `await` (next part) covers most cases, so reach for `Result` when you specifically need the outcome as data rather than control flow.

Takeaways

- `let` / `var` are immutable/mutable bindings; prefer `let`. Type inference is pervasive; `private(set)` gives read-public, write-private in one keyword.
- `Int` / `Double` / `Bool` / `String` are **value types** — no boxing, no primitive/object split. `struct` / `enum` copy; `class` / actor share by reference.
- Optionals replace `null: T?` must be unwrapped via `if let` / `guard let`, defaulted with `??`, or chained with `?.`. Avoid `!` and `T!` outside genuine programmer-error and interop cases.
- Errors use `throws` / `try` / `do - catch`; `throws` is untyped (no exception class in the signature, no checked/unchecked split), but `try` and handling are compiler-enforced.
- `try?` yields nil on failure, `try!` traps; `defer` is a scope-local `finally`; `Result` turns success-or-error into a storable value.

Next: *Types: Structs, Enums, Protocols, Generics* expands value vs reference into the full type system — how `struct`, `enum`, protocols, and generics replace classes-and-interfaces as your primary modeling tools.

Types: Structs, Enums, Protocols, Generics

Swift's type system is wider than Java's: `struct`, `enum`, and `class` are all first-class, and most code is built from value types. This chapter maps each construct to its closest Java analog, then marks where the analogy breaks — which is often.

struct: the default type

A `struct` is a value type. The closest Java starting point is `record`: a data aggregate with a compiler-synthesised initializer.

```
struct FileCryptor {
    func encrypt(_ plaintext: Data,
                using key: SymmetricKey) throws -> Data {
        let sealed = try AES.GCM.seal(plaintext, using: key)
        guard let combined = sealed.combined else {
            throw FileCryptorError.sealingFailed
        }
        return combined
    }
    // decrypt(...) elided
}
```

If a `struct` has stored properties and you write no `init`, Swift synthesises a **memberwise initializer** taking every property in declaration order:

```
struct KeychainKeyStore: KeyProviding {
    let service: String
    let account: String
    // synthesised: init(service:account:) – unless you
    // write your own, which KeychainKeyStore does (to
    // supply defaults).
}
```

That is where the `record` analogy ends. Three differences matter.

1. Value semantics, everywhere. Assigning or passing a `struct` copies it. There is no aliasing, no shared mutable reference, no `equals` / `hashCode` contract to hand-write for identity.

```
var a = DateRange(start: .now, end: .now)
var b = a           // full copy
b.end = .distantFuture
// a.end is unchanged
```

In Java, two variables pointing at the same object see each other's mutations. In Swift, `b` is independent. (Copies are copy-on-write for the standard library's `Array` / `String` / `Dictionary`, so this is cheap until you mutate.)

2. **let** makes the whole value immutable. A **let** binding to a **struct** freezes *all* its properties — not just the reference. This is stronger than Java **final**, which only stops reassignment of the field itself.

```
let store = KeychainKeyStore()
store.account = "x" // compile error: store is a let
```

With a **class**, **let** only freezes the reference; you could still call a method that mutates the object's fields. With a **struct**, **let** is deep.

3. **mutating methods**. Because a **struct** value is copied, a method that changes **self** must say so explicitly with **mutating**. The compiler then forbids calling it on a **let** value.

```
struct Counter {
    private(set) var value = 0
    mutating func bump() { value += 1 }
}
```

Java has no equivalent because objects are always references; “mutating a record” is not a concept — records are final.

Quarkus analogy. Think of a **struct** like a JPA-detached DTO you deep-copy on every hand-off: no entity-manager identity, no shared state, no surprise mutation from another thread. The compiler enforces what you'd normally enforce by discipline.

class: reference type, used sparingly

A **class** is a reference type with the Java semantics you expect: identity, shared mutable state, inheritance, and (in Swift) a **deinit** that runs when the last reference is released — Swift uses ARC (automatic reference counting), not a tracing GC.

```
@Observable
final class ThemeController {
    private(set) var theme: AppTheme
    func select(_ theme: AppTheme) {
        self.theme = theme
        // persist...
    }
}
```

ThemeController is a **class** deliberately: it holds shared, observable, mutable state that the whole view tree reads. That is the canonical reason to reach for a **class** in Swift — reference identity and shared mutation are the *point*.

final blocks subclassing (like Java **final class**) and lets the compiler devirtualise calls. The Swift convention inverts Java's defaults: **classes are final unless inheritance is designed in**, and you prefer **struct** unless you specifically need reference semantics, identity, inheritance, or Objective-C interop.

`deinit` is deterministic — it runs the instant the refcount hits zero, unlike Java's `finalize`, which the GC may never call. Use it for cleanup (closing a handle, invalidating a timer), not for resurrection.

Gotcha. ARC cannot collect reference cycles. Two classes holding strong references to each other leak. The fix is `weak` / `unowned` references, covered in *Memory and Concurrency*. Value types can't form cycles, which is another reason to prefer them.

enum: sum types with pattern matching

Swift's `enum` is not Java's `enum`. It is a true algebraic sum type: each case can carry associated values, and the compiler enforces exhaustive handling. The mental model is Java's `sealed interface` with records as permitted subtypes — but lighter and built in.

Raw-value enums

When every case maps to a constant (a `String`, `Int`, etc.), give the enum a **raw type**. `AppTab` is the tab bar's five destinations:

```
enum AppTab: String, CaseIterable, Identifiable {
    case home, docs, decode, dates, settings

    var id: String { rawValue }

    var titleKey: LocalizedStringKey {
        switch self {
        case .home: return "tab_home"
        case .docs: return "tab_docs"
        case .decode: return "tab_decode"
        case .dates: return "tab_dates"
        case .settings: return "tab_settings"
        }
    }

    var systemImage: String {
        switch self {
        case .home: return "house"
        case .docs: return "folder"
        case .decode: return "doc.text.viewfinder"
        case .dates: return "calendar"
        case .settings: return "gearshape"
        }
    }
}
```

`: String` synthesises `rawValue` (here `"home"`, `"docs"`, ...) and a failable `init?(rawValue:)`. That round-trip is how `ThemeController` persists a theme: store `theme.rawValue` in `UserDefaults`, then `AppTheme(rawValue: raw)` to read it back. The initializer is failable (returns `AppTheme?`) because an arbitrary string may not match a case.

Computed properties, not field tables

Note that `titleKey` and `systemImage` are **computed properties** with a `switch`, not data attached to each case as in a Java enum constructor. Swift enums can't store per-instance properties on cases (a case with associated values is a different thing). The idiom is a computed property that switches over `self`. The compiler's exhaustiveness check guarantees that adding a sixth tab forces you to update every such `switch` — a missing case is a compile error, not a runtime `default` fall-through.

`AppLanguage` follows the same shape with richer derived data:

```
enum AppLanguage: String, CaseIterable, Identifiable, Sendable {
    case de, en, tr, fr, es, it, ar, ru, zh

    var isRTL: Bool { self == .ar }

    var layoutDirection: LayoutDirection {
        isRTL ? .rightToLeft : .leftToRight
    }

    var locale: Locale {
        Locale(identifier: rawValue)
    }

    var endonym: String {
        switch self {
        case .de: return "Deutsch"
        case .ar: return "العربية"
        case .zh: return "中文"
        // ...
        }
    }
}
```

CaseIterable and Identifiable

`CaseIterable` synthesises `static var allCases: [Self]` — the ordered list of cases. `RootView` iterates it to build the tab bar:

```
ForEach(AppTab.allCases) { tab in
    screen(for: tab)
        .tabItem {
            Label(tab.titleKey,
                  systemImage: tab.systemImage)
        }
    .tag(tab)
}
```

`Identifiable` requires an `id`; SwiftUI uses it for stable diffing in lists and `ForEach`, the way a UI framework needs a stable key. Both are **protocols whose requirements the compiler synthesises** for enums — no boilerplate.

Associated values: the sum-type payoff

Cases can carry data, making the enum a tagged union:

```
enum LoadState {
    case idle
    case loading
    case loaded([DocumentRecord])
    case failed(Error)
}
```

A `switch` then binds the payload:

```
switch state {
case .idle, .loading:
    ProgressView()
case .loaded(let docs):
    list(docs)
case .failed(let error):
    errorView(error)
}
```

This is exactly a `sealed interface` with record subtypes plus a pattern-matching `switch` — but with no class hierarchy, no `instanceof`, and exhaustiveness for free. `Optional<T>` and the `throws` machinery are themselves enums built this way.

protocol: interfaces, then more

A `protocol` is an interface: a set of requirements a type promises to satisfy. `KeyProviding` abstracts where the encryption key comes from:

```
protocol KeyProviding: Sendable {
    func symmetricKey() throws -> SymmetricKey
}
```

Two conformances — production and test — exactly as you’d swap a CDI bean for a mock:

```
struct KeychainKeyStore: KeyProviding { /* Keychain */ }

final class InMemoryKeyStore: KeyProviding,
    @unchecked Sendable {
    private let key: SymmetricKey
    func symmetricKey() throws -> SymmetricKey { key }
}
```

Two things a Java reader should register. First, conformance is declared at the type, but a `struct`, `class`, and `enum` can all conform — protocols are not class-only. Second, `Sendable` is a **marker protocol** with no methods; it tells the concurrency checker the type is safe to cross actor boundaries (covered later).

Protocol extensions = default methods, but more

A protocol extension provides implementations that all conformers get for free — like Java `default` methods, but able to add *new* members, not just defaults for declared ones:


```
extension KeyProviding {
    func keyFingerprint() throws -> String {
        try symmetricKey()
            .withUnsafeBytes { SHA256.hash(data: Data($0)) }
            .map { String(format: "%02x", $0) }
            .joined()
    }
}
```

Every `KeyProviding` now has `keyFingerprint()` without each type implementing it. This is the core of **protocol-oriented programming**: behaviour composed from small protocols + extensions, instead of an inheritance tree. It is Swift’s answer to “I’d put this in an abstract base class” — but it works for value types too.

any VS some

When you reference a protocol as a *type*, you choose between two forms:

```
let store: any KeyProviding // existential (boxed)
func makeStore() -> some KeyProviding // opaque
```

`any KeyProviding` is an **existential**: a box that can hold any conformer, resolved dynamically — analogous to a Java variable typed as the interface. It is flexible but has indirection and blocks protocols with associated types.

`some KeyProviding` is an **opaque type**: one specific, fixed conforming type the compiler knows but the caller doesn’t name. There’s no boxing, full optimization, and it can carry associated types. SwiftUI leans on this — every `body` returns `some View`. Reach for `some` unless you genuinely need to store mixed concrete types together, then use `any`.

Associated types

A protocol can have a placeholder type, like a Java generic interface `interface Repo<T>` — but declared *inside* the protocol:

```
protocol Repository {
    associatedtype Entity
    func all() -> [Entity]
}
```

A conformer fixes `Entity` (often inferred). The catch: a protocol with an associated type can’t be used as a plain `any Repository` in every position, because the associated type isn’t pinned — which is precisely where `some` and generic constraints come in.

extension: open the type

extension adds methods, computed properties, and protocol conformance to an existing type — including types you don’t own. There is no Java equivalent; the Java workaround is a `static` util class (`StringUtils.foo(s)`). `ColorTokens` extends Apple’s `UIColor`:

```
private extension UIColor {
    convenience init(rgb: Int, alpha: Double = 1) {
        self.init(
            red: CGFloat((rgb >> 16) & 0xFF) / 255,
            green: CGFloat((rgb >> 8) & 0xFF) / 255,
            blue: CGFloat(rgb & 0xFF) / 255,
            alpha: CGFloat(alpha))
    }
}
```

Now `UIColor(rgb: 0xFAF8F4)` reads as a real initializer, not `ColorUtils.fromRgb(0xFAF8F4)`. `private` scopes the extension to this file. Extensions are also the idiomatic way to add protocol conformance after the fact and to organise a type's members into logical groups.

Note. Extensions can't add stored properties or override existing methods — they widen the API surface, they don't change a type's layout or replace behaviour. That keeps them safe to apply to types you don't control.

Generics: reified, not erased

Generics look familiar: a type or function parameterised over `T`, with `where` constraints bounding it.

```
func first<T>(in items: [T],
             matching p: (T) -> Bool) -> T? {
    for item in items where p(item) { return item }
    return nil
}

func sortedByName<T>(_ items: [T]) -> [T]
    where T: Identifiable, T.ID: Comparable {
    items.sorted { $0.id < $1.id }
}
```

The constraint syntax `where T: Identifiable` is Java's `<T extends Identifiable>`. The deep difference is the **runtime model**.

Java generics are **erased**: `List<String>` and `List<Integer>` are the same `List` at runtime, `T` becomes `Object`, and you cannot write `new T()`, `T.class`, or `instanceof List<String>`.

Swift generics are **reified** and **monomorphised**: the compiler specialises generic code per concrete type, and the type argument survives at runtime. Consequences a backend engineer should internalise:

- No type-erasure boxing. `[Int]` stores `Int` values inline, not boxed `Integer`s. Value types stay flat through generics — performance you can rely on.
- `T` is a real type at runtime: you can call `T.self`, switch on the metatype, and satisfy `T: Initializable`-style constraints to construct values. There is no `Object` floor.

- Specialisation can mean larger binaries and longer compiles — the dual of C++ templates. Swift mitigates this with witness tables when it can't or won't specialise, but the semantic model is “the real type is there.”

Quarkus analogy. Java erasure forces tricks — `Class<T>` tokens, `TypeReference`, reflection — to recover a type you “lost.” Swift never loses it, so those patterns largely disappear. The cost moves to the compiler and binary size instead of runtime reflection.

Where generics meet protocols

`@Model final class DocumentRecord` shows the payoff in real code:

```
@Model
final class DocumentRecord {
    @Attribute(.unique) var id: UUID
    var fileName: String
    var category: String
    var createdAt: Date
    var expiresAt: Date?
}
```

SwiftData's generic `FetchDescriptor<DocumentRecord>` and `Query` carry the concrete element type with no erasure, so fetched results come back as `[DocumentRecord]`, fully typed, no casts. This is the same machinery as `[T]` above — generics + protocol constraints + reified types working together, which is the spine of idiomatic Swift APIs.

Takeaways

- Default to `struct` (value semantics, `let` = deep immutability, `mutating` for in-place change); reach for `class` only when you need identity, shared mutable state, inheritance, or `deinit`. Mark classes `final`.
- Swift `enum` is a true sum type: associated values model a `sealed interface`, and exhaustive `switch` turns “forgot a case” into a compile error. Raw-value enums give cheap `String` / `Int` round-trips.
- `CaseIterable` and `Identifiable` are compiler-synthesised protocol conformances; protocol extensions are default methods that can also add new behaviour — the basis of protocol-oriented programming.
- `extension` reopens any type (even Apple's) to add API — no Java equivalent; prefer it over utility classes. It can't add stored state or override.
- Use `some` (opaque, no boxing) over `any` (existential, boxed) unless you must store mixed concrete types.
- Swift generics are reified/monomorphised, not erased: type arguments survive at runtime, value types stay unboxed, and the `Class<T>` / reflection workarounds from Java disappear.

Next: *Functions, Closures, and Functional Style* — argument labels, closures and capture, `map` / `filter` / `reduce` vs Java Streams, and the `@resultBuilder` behind SwiftUI's `@ViewBuilder`.

Functions, Closures, and Functional Style

Swift functions look like Java methods until you hit argument labels, trailing closures, and result builders. This chapter covers the call-site ergonomics, first-class closures, and the higher-order collection API — anchored to Java methods and Streams, then the deltas.

Function syntax and argument labels

```
func encrypt(_ plaintext: Data,
            using key: SymmetricKey) throws -> Data
```

The surprising part for a Java reader is that most parameters have *two* names: an **external argument label** (seen at the call site) and an **internal parameter name** (used in the body).

```
func encrypt(_ plaintext: Data, using key: SymmetricKey)
//          ▲ label ▲ name    ▲label ▲name
```

- `_ plaintext`: the `_` suppresses the label, so the first argument is passed positionally. Inside the body it's `plaintext`.
- `using key`: the label is `using`; inside the body it's `key`.

The call site reads as a sentence:

```
let blob = try cryptor.encrypt(data, using: key)
```

This is deliberate API design, not noise. The method *name* is `encrypt(_:using:)`, and labels are part of it — `encrypt(_:with:)` would be a different function. There is no Java analog; Java distinguishes overloads only by parameter *types*, never by call-site labels. When you read `AES.GCM.seal(plaintext, using: key)`, the `using:` is doing the same job.

Defaults, variadics, and `inout` round out the signature surface:

```
init(service: String = "de.everydaygermany.app.documents",
      account: String = "document-encryption-key")

func sum(_ values: Int...) -> Int    // variadic
func reset(_ x: inout Int) { x = 0 } // pass-by-ref
```

`KeychainKeyStore`'s `init` uses **default arguments** so tests can pass an isolated slot while production calls `KeychainKeyStore()`. A variadic `(Int...)` is `int...` from Java. `inout` is true pass-by-reference for a value type: the callee mutates the caller's variable, and you must pass it with `&x`. It is not a shared reference — the value is copied in and copied back out at the call boundaries.

Quarkus analogy. Default arguments replace the telescoping- constructor and `@DefaultValue` patterns: one signature, optional params filled at the call site. No overload explosion, no builder.

Closures: first-class functions

A closure is a lambda: an anonymous function value that captures its environment. Functions are first-class — you store them, pass them, and return them. The full form mirrors a function signature in braces:

```
let add = { (a: Int, b: Int) -> Int in a + b }
add(2, 3)    // 5
```

In a context where the type is known, you drop the annotations, and shorthand argument names `$0`, `$1` replace the parameter list:

```
let names = docs.map { $0.fileName }
```

Trailing-closure syntax

If a closure is the last argument, it moves outside the parentheses; if it's the *only* argument, the parentheses vanish entirely. This is why so much Swift and all of SwiftUI reads as blocks rather than calls. The `dynamic(light:dark:)` helper in `ColorTokens` passes a closure to `UIColor.init(_:)`:

```
Color(uiColor: UIColor { traits in
    traits.userInterfaceStyle == .dark
    ? UIColor(rgb: dark)
    : UIColor(rgb: light, alpha: lightAlpha)
})
```

`UIColor { traits in ... }` is `UIColor(_: { traits in ... })` with the trailing closure pulled out. The closure runs *later*, whenever the trait environment changes — that's what makes the color dynamic.

A second example from `KeychainKeyStore` — a closure handed to `withUnsafeBytes` to copy raw key bytes into `Data`:

```
let data = key.withUnsafeBytes { Data($0) }
```

`$0` is the buffer pointer; `Data($0)` copies it out. The closure scopes the unsafe pointer's lifetime to that one expression.

Capture lists

A closure captures referenced variables. For reference types this can form a retain cycle, so a **capture list** lets you capture weakly:

```
loader.onFinish = { [weak self] result in
    self?.apply(result)
}
```

`[weak self]` captures `self` as an optional weak reference, breaking the cycle. Treat this as the intro; ARC, `weak`, and `unowned` are the subject of *Memory and Concurrency*. Value types captured by a closure are copied, so they never need a capture list for cycle reasons.

Higher-order functions vs Java Streams

The collection transforms you know from `Stream` are methods directly on the collection — no `.stream()` to enter and no `.collect()` to exit.

```
let selectable: [AppLanguage] = [.de, .en]
let visible = AppLanguage.allCases
    .filter { $0.isSelectable }
    .map(\.endonym)          // key path as a function
    .sorted()
```

Mapping to the Java equivalents:

Swift	Java Stream
<code>map</code>	<code>map</code>
<code>filter</code>	<code>filter</code>
<code>reduce(_:_:)</code>	<code>reduce</code>
<code>compactMap</code>	<code>map</code> + filter null
<code>flatMap</code>	<code>flatMap</code>

Real call from `AppLanguage` :

```
var isSelectable: Bool {
    Self.selectable.contains(self)
}
```

`contains` is a higher-order member too (`contains(where:)` takes a predicate; here the value overload). Two deltas matter against Streams:

1. **Methods on the collection, eager by default.** `map` / `filter` return a fully materialised `Array` immediately. There is no lazy pipeline and no terminal operation; the result is the array. You don't "open" or "close" a stream.
2. **Opt into laziness with `.lazy`.** For a large source where you want Stream-style deferred, fused evaluation:

```
let firstBig = records.lazy
    .filter { $0.expiresAt != nil }
    .map(\.fileName)
    .first
```

`.lazy` makes the chain compute elements on demand, so `first` stops after the first match instead of transforming the whole array. That's the Stream laziness you're used to — but it's opt-in, because eager is the common, simpler case.

`compactMap` deserves a note: it maps and drops `nil` s in one pass, unwrapping `Optional` results — the idiomatic way to parse-and-filter (e.g. `rawValues.compactMap(AppLanguage.init(rawValue:))`).

Gotcha. Because the standard operators are eager, chaining many of them over a large array allocates an intermediate array per step. For hot paths or huge inputs, prefix with `.lazy` to fuse them, the way a `Stream` already does.

KeyPaths

`\.id` is a **key path**: a first-class, type-safe reference to a property, usable wherever a `(Root) -> Value` function is expected.

```
.map(\.endonym)           // == .map { $0.endonym }
.sorted(using: KeyPathComparator(\.createdAt))
```

The nearest Java idea is a method reference (`Doc::getEndonym`), but a key path is a reusable *value* you can store, compose, and pass around, not just a call-site shorthand. SwiftUI and SwiftData use them pervasively (`\.createdAt` in a sort, `\.self` as an identity id).

@escaping vs non-escaping

By default a closure parameter is **non-escaping**: it must run before the function returns and cannot be stored. That lets the compiler skip capture-cycle worries and optimise harder.

A closure that outlives the call — stored in a property, scheduled, passed to another thread — must be marked `@escaping`:

```
func load(then completion: @escaping ([DocumentRecord]) -> Void)
```

The compiler enforces the distinction: try to store a non-escaping closure and it won't compile. Inside an `@escaping` closure, references to `self` must be explicit, which is the nudge to add `[weak self]`. With `async/await`, most of this fades — you write straight-line code instead of escaping completion handlers — but you'll still see `@escaping` in callback-style and AppKit/UIKit APIs.

@resultBuilder and @ViewBuilder

A **result builder** is a compile-time transform that turns a block of statements into a single aggregated value. You rarely write one, but you use one constantly: SwiftUI's `@ViewBuilder` is a result builder that collects the views in a block into one composite view.

`RootView.screen(for:)` is a `@ViewBuilder` function whose body is a `switch` returning a *different* concrete view type per branch:

```
@ViewBuilder
private func screen(for tab: AppTab) -> some View {
    switch tab {
    case .home: HomeView()
    case .docs: VaultView()
    case .decode: DecoderView()
    case .dates: DatesView()
    case .settings: SettingsView()
    }
}
```

Without `@ViewBuilder` this wouldn't compile: the branches return `HomeView`, `VaultView`, etc. — distinct types — yet the signature promises one `some View`. The builder rewrites the `switch` into a `_ConditionalContent` wrapper that erases the branches into a single view type. The same mechanism lets you stack views in a `VStack { ... }` body without commas, `return`s, or an array literal.

Conceptually it's a typed builder DSL the compiler generates for you — closer to a Java fluent builder than to anything in `Stream`, but applied implicitly to ordinary control flow.

Note. You can author your own `@resultBuilder` (validation DSLs, query builders), but the high-value takeaway is recognising that `@ViewBuilder` blocks are *not* normal closures — `if`, `switch`, and bare expressions inside them are being transformed, which explains why SwiftUI bodies don't use `return` or commas.

Takeaways

- Argument labels are part of a function's name: `_` drops the label, a word before the parameter names it, so `encrypt(data, using: key)` reads as a sentence. No Java equivalent.
- Closures are first-class lambdas; trailing-closure syntax (and `$0` shorthand) is why SwiftUI reads as blocks. `[weak self]` capture lists break retain cycles (full treatment later).
- `map` / `filter` / `reduce` / `compactMap` / `flatMap` are methods on the collection — no `.stream()` / `.collect()` — and **eager** by default; add `.lazy` for Stream-style deferred fusion.
- Key paths (`\.endonym`) are reusable, type-safe property references usable as functions, beyond a Java method reference.
- `@escaping` marks closures that outlive the call (stored/async); non-escaping is the optimised default.
- `@ViewBuilder` is a `@resultBuilder`: it transforms a block (including `switch` / `if`) into one composite view, which is why `RootView.screen(for:)` can return different view types as `some View`.

Next: *Memory and Concurrency: ARC, async/await, Actors* — the reference-counting model behind `[weak self]`, structured concurrency, and actor isolation.

Memory and Concurrency: ARC, async/await, Actors

This chapter covers the two runtime models where Swift diverges most sharply from the JVM: how objects are reclaimed, and how concurrent code is made safe. Coming from Quarkus you have a garbage collector you rarely think about and a thread-safety story you enforce by discipline (and hope). Swift inverts both. Memory is managed deterministically at compile time, and data races are caught by the compiler before the program runs.

Part 1 — Memory: ARC instead of a garbage collector

The JVM tracks live objects at runtime. A background collector periodically traces the object graph from GC roots, reclaims the unreachable, and occasionally stops your threads to do it. You tune heap sizes and collectors (G1, ZGC) precisely because this is non-deterministic and can pause request handling.

Swift uses **Automatic Reference Counting (ARC)**. Each class instance carries a reference count. The compiler — not a runtime collector — inserts `retain` (increment) and `release` (decrement) calls at the points where references begin and end. When the count hits zero, `deinit` runs and the memory is freed, immediately and at a known place in the code.

Aspect	JVM GC	Swift ARC
When	Runtime, async	Compile-inserted
Determinism	Non-deterministic	Deterministic
Pauses	Possible (STW)	None
Cost	Background threads	Inline retain/release

Quarkus analogy. ARC is closest to `try-with-resources` or a reference-counted `Auto-Closeable`, except the compiler writes the `close()` calls for you and threads them through every assignment, return, and capture. There is no collector thread and no STW pause to tune.

The trade-off: ARC cannot break **reference cycles**. A GC traces reachability, so two mutually-referencing objects with no external root are collected. ARC only counts; two objects pointing at each other keep each other's count above zero forever. That is the one memory failure mode a backend engineer must learn to recognize.

Retain cycles, weak, and unowned

Only **classes** participate in reference counting. A cycle forms when two class instances hold strong references to each other, or — far more common — when a closure captures the object that owns it.

```
final class Vault {
    var onUnlock: (() -> Void)?
    func arm() {
        // STRONG capture of self -> cycle:
        // self -> closure -> self
        onUnlock = { self.refresh() }
    }
    func refresh() { /* ... */ }
}
```

`self` retains the closure (it is stored in `onUnlock`); the closure retains `self`. Neither count reaches zero; `deinit` never runs. The fix is a **capture list** that captures `self` weakly:

```
onUnlock = { [weak self] in self?.refresh() }
```

`weak` references are optional and become `nil` automatically when the referent is deallocated — so you unwrap with `?` or `guard let`. `unowned` is the non-optional variant: use it only when the captured object is guaranteed to outlive the closure (accessing a deallocated `unowned` reference traps, like a `!`-forced `nil`). Default to `[weak self]`; reach for `unowned` only when the lifetime invariant is genuinely airtight.

Gotcha. Capture lists are evaluated once, when the closure is created. `[weak self]` does not “watch” `self`; it grabs a weak reference at that instant. This matters for `Task` closures and SwiftUI callbacks that outlive the current scope.

Value types and copy-on-write

The cycle problem is a *class* problem. Swift’s `struct` and `enum` are **value types**: assigning or passing them copies the value, so there is no shared identity to form a cycle and usually nothing to reference-count. `AppLanguage` in Alltag is a value type:

```
enum AppLanguage: String, CaseIterable,
    Identifiable, Sendable {
    case de, en, tr, fr, es, it, ar, ru, zh
}
```

Passing an `AppLanguage` around never allocates, never retains, and never leaks. This is the opposite of Java, where every non-primitive is a heap reference. In Swift, prefer `struct`; reach for `class` only when you need identity, inheritance, or shared mutable state.

Naively, value semantics imply copying large collections on every assignment. Swift avoids that with **copy-on-write (COW)**. `Array`, `String`, and `Dictionary` are structs that wrap a reference to heap storage. Assigning shares the storage; the copy is deferred until the first mutation, and only then if the storage is shared (referenced more than once).

```
var a = [1, 2, 3]
var b = a           // shares storage, no copy
b.append(4)         // now b copies; a is untouched
```

So you get value semantics (safe to pass across boundaries) with reference-level performance for reads. No defensive copying, no `Collections.unmodifiableList`, no aliasing surprises.

When a backend engineer should actually worry

Rarely. Value types do not leak. The realistic risks are two:

1. A `class` whose stored closure captures `self` strongly (the `Vault` example). Audit any `var handler: (() -> Void)?` field.
2. Long-lived objects (a controller, an `@Observable` store) captured strongly by closures with a longer lifetime than expected — delegates, notification observers, `Task` s.

Xcode’s Memory Graph Debugger and Instruments’ Leaks/Allocations play the role of a heap dump plus a leak detector. But unlike JVM leaks, the common Swift leak has a single shape — a captured `self` — so `[weak self]` is the one habit that prevents almost all of them.

Part 2 — Concurrency: the big shift

This is where Swift will feel most foreign. In Quarkus you reach for `ExecutorService`, `CompletableFuture`, or Mutiny `Uni`, and you protect shared state with `synchronized`, `ReentrantLock`, or `Atomic*`. You are responsible for thread-safety, and the compiler does not check it. Swift replaces this with `async` / `await`, structured tasks, `actor` isolation, and — the headline — **compile-time data-race safety**.

async/await vs CompletableFuture and Uni

An `async` function can suspend and resume without blocking its thread. `await` marks a suspension point. The shape reads like synchronous code but is non-blocking, much like Mutiny lets you compose without blocking a worker thread — except there is no pipeline type to thread through.

```
// Java: the type is contagious and explicit.
CompletableFuture<Document> load(UUID id) { ... }
load(id).thenApply(this::decrypt)
    .thenAccept(this::render);
```

```
// Swift: async is a function effect, like throws.
func load(_ id: UUID) async throws -> Document { ... }

let doc = try await load(id)
let plain = try await decrypt(doc)
render(plain)
```

Key differences from `CompletableFuture` / `Uni`:

- `async` is an *effect* on the function (like `throws`), not a wrapper type. There is no `CompletableFuture<T>` to unwrap or `Uni<T>` to subscribe to — you write `try await` and get `T`.
- You cannot `await` outside an `async` context. To bridge from sync code you start a `Task` (below), the rough analog of handing work to an executor.
- Suspension is cooperative: an awaiting function yields its thread back to a shared pool, so thousands of in-flight `async` calls do not mean thousands of OS threads.

Task: entering the concurrent world

A `Task` is the unit that runs async work — loosely, “submit this to the cooperative pool.” Creating one from synchronous code (a SwiftUI button, `viewDidLoad`) is how you cross into async:

```
Task {
    do {
        let doc = try await load(id)
        await render(doc)
    } catch {
        log.error("load failed: \(error)")
    }
}
```

Unlike `executor.submit`, a `Task` inherits context from where it is created — priority, and crucially the **actor** it runs on (so a `Task` created in `@MainActor` code stays on the main actor unless told otherwise).

Structured concurrency: async let and TaskGroup

Structured concurrency ties child task lifetimes to a lexical scope. When the scope exits, its children are guaranteed finished or cancelled — no orphaned futures, no leaked threads. There is no clean JVM equivalent before Project Loom’s `StructuredTaskScope`.

`async let` runs siblings concurrently and joins at the `await`:

```
func loadDashboard() async throws -> Dashboard {
    async let docs = fetchDocuments()
    async let lang = fetchLanguagePack()
    // both run concurrently; join here:
    return Dashboard(
        documents: try await docs,
        language: try await lang)
}
```

For a dynamic number of children, use a `TaskGroup` — the analog of `invokeAll` over a list, but scoped and cancellation-aware:

```
func decryptAll(_ ids: [UUID]) async throws
-> [Document] {
    try await withThrowingTaskGroup(
        of: Document.self
    ) { group in
        for id in ids {
            group.addTask { try await load(id) }
        }
        var out: [Document] = []
        for try await doc in group {
            out.append(doc)
        }
        return out
    }
}
```

Cancellation is cooperative

Cancelling a `Task` (or its parent scope exiting) does not forcibly stop anything — exactly like `Thread.interrupt()` sets a flag rather than killing the thread. Your code must observe it: call `try Task.checkCancellation()` (throws `CancellationError`) or read `Task.isCancelled` in loops. Most async stdlib calls (`URLSession`, `Task.sleep`) check cancellation for you.

```
for id in ids {
    try Task.checkCancellation()
    results.append(try await load(id))
}
```

Quarkus analogy. Structured concurrency is what you wish `CompletableFuture` had: child tasks cannot outlive their parent scope, errors propagate to the awaiting parent, and cancelling the parent cancels the children. No manually tracking and cancelling a `List<Future<?>>`.

actor: isolation instead of locks

An `actor` is a reference type whose mutable state is protected by the compiler. Only one task touches an actor's state at a time; all access from outside is serialized and must be `await`ed. This is the Swift answer to `synchronized`, `ReentrantLock`, and `Atomic*` — but enforced rather than remembered.

```
actor ModelDownloadTracker {
    private var bytesByModel: [String: Int] = [:]

    func record(_ model: String, bytes: Int) {
        // No lock needed: actor serializes this.
        bytesByModel[model, default: 0] += bytes
    }

    func total(_ model: String) -> Int {
        bytesByModel[model, default: 0]
    }
}
```

From outside, every call hops onto the actor and suspends:

```
await tracker.record("gemma", bytes: n)
```

The compiler *rejects* synchronous access to `bytesByModel` from another context. You cannot forget the lock, because there is no lock to forget — the type system requires the `await`. This is the classic actor model (Akka, Erlang), but built into the language and checked at compile time.

Note. Actor isolation prevents data races, not logical races. Two `await`ed `record` calls are each atomic, but the interleaving between them is still nondeterministic — just as two `synchronized` methods don't compose into one atomic operation. For read-modify-write you still need a single actor method that does the whole thing.

@MainActor: the UI thread, enforced

UI frameworks require their work on one thread — Swing’s EDT, JavaFX’s Application Thread. The JVM enforces this only at runtime (`IllegalStateException` , or worse, silent corruption). Swift has a global actor, `@MainActor` , and the compiler enforces it.

Annotating a type with `@MainActor` isolates all its members to the main thread. Alltag’s environment and stores do exactly this:

```
@MainActor
@Observable
final class AppEnvironment {
    let theme: ThemeController
    let language: LanguageStore
    let persistence: PersistenceController
    let llm: LLMService
    // ...
}
```

```
@MainActor
@Observable
final class LanguageStore {
    private(set) var language: AppLanguage
    func select(_ language: AppLanguage) {
        guard language.isSelectable else { return }
        self.language = language
        // ...
    }
}
```

Because `LanguageStore` is `@MainActor` , calling `select(_)` from a background `Task` requires an `await` that hops to the main thread — the compiler guarantees the UI-bound `@Observable` state is only mutated on the main thread. No more “updated the model off the EDT” bugs.

`PersistenceController` is `@MainActor` for the same reason: its `mainContext` is main-actor bound. The class comment captures the escape route for background work:

```
/// `@MainActor` because the app's `mainContext` is
/// main-actor bound; background work uses a `ModelContext`
/// created off a `ModelContainer` (Sendable).
@MainActor
final class PersistenceController {
    let container: ModelContainer
    // ...
}
```

The `ModelContainer` is `Sendable` , so it can cross actor boundaries; a background `Task` makes its own `ModelContext` from it rather than touching the main-actor `mainContext` .

Sendable and compile-time data-race safety

This is the selling point with no Java equivalent. `Sendable` is a marker protocol meaning “safe to pass across concurrency boundaries.” With `SWIFT_STRICT_CONCURRENCY: complete` — set in Alltag’s

`project.yml` — the compiler **checks** that every value crossing an actor or task boundary is `Sendable`, and flags anything that could be mutated from two places at once.

```
settings:
  base:
    SWIFT_STRICT_CONCURRENCY: complete
```

Java has nothing like this. `@GuardedBy` is a comment; the JMM defines what *correct* synchronization means but the compiler never verifies you did it. You find data races in production, via heisenbugs. Swift refuses to compile the race.

What is `Sendable` for free? Value types of `Sendable` members (so `AppLanguage`, which is declared `Sendable`), actors (isolated by construction), and immutable classes. Mutable classes are *not* `Sendable` unless you prove it.

Protocols can require it. `KeyProviding` is `Sendable` so any key store can be shared across the encryption pipeline’s tasks:

```
protocol KeyProviding: Sendable {
  func symmetricKey() throws -> SymmetricKey
}
```

`KeychainKeyStore` is a `struct` with immutable `let` fields, so it satisfies `Sendable` automatically. The compiler verifies it.

@unchecked Sendable: the escape hatch

Sometimes you know a type is safe but the compiler cannot prove it. `@unchecked Sendable` opts out of checking — “trust me.” It is the analog of writing thread-safe Java by hand: correct, but unverified. Use it sparingly and document the invariant.

Alltag’s test key store is the real example: a `final class` (reference type, so not auto-`Sendable`) holding a `let` key. It is logically immutable, hence safe to share, but the compiler won’t infer that for a class:

```
final class InMemoryKeyStore: KeyProviding,
  @unchecked Sendable {
  private let key: SymmetricKey

  init(key: SymmetricKey = SymmetricKey(size: .bits256)) {
    self.key = key
  }

  func symmetricKey() throws -> SymmetricKey { key }
}
```

The `key` is a `let` set once in `init` and never mutated, so concurrent reads are safe — but because it is a class, you assert that with `@unchecked Sendable` rather than getting it for free. (A `struct` here would have been `Sendable` automatically; it’s a class because the protocol is shared and the test needs a stable identity.)

Gotcha. `@unchecked Sendable` silences the checker *completely* for that type. If you later add a mutable `var`, the compiler will not warn you that you have reintroduced a race. Treat every `@unchecked` as a small unverified contract, and keep the type tiny so the invariant is obvious.

nonisolated, and Swift 6 defaults

`nonisolated` marks a member of an actor-isolated type as *not* isolated — it can be called synchronously from anywhere because it touches no isolated state (e.g. a computed property over `let` fields, or a `Hashable` conformance). It is the targeted exception to a `@MainActor` or actor type.

```
@MainActor
final class Catalog {
    let id: String           // immutable
    nonisolated var key: String { "catalog-\(id)" }
}
```

Under **Swift 6**, strict concurrency becomes the *default* — so `SWIFT_STRICT_CONCURRENCY: complete` is the on-ramp Alltag is already standing on. Code written this way today compiles unchanged under the Swift 6 language mode; code that ignored concurrency will face a wall of errors at migration. Building with `complete` now is paying down that debt incrementally instead of all at once.

Quarkus analogy. Think of `Sendable` as `@ThreadSafe` that the compiler actually enforces, and `@MainActor` as a statically checked “must run on the EDT.” The mental model is familiar; what’s new is that the guarantees are proven at build time, not discovered in production.

Takeaways

- ARC reclaims memory deterministically at scope/lifetime end — no GC pauses, but you must break reference cycles yourself with `[weak self]` (default) or `unowned` (sharp edge).
- Prefer value types (`struct` / `enum`); they don’t leak and can’t form cycles. `Array` / `String` / `Dictionary` use copy-on-write for value semantics at reference-read cost.
- `async` / `await` is an effect, not a wrapper type; `Task` bridges from sync code; structured concurrency (`async let` , `TaskGroup`) ties child lifetimes to a scope and cancels cooperatively.
- `actor` replaces locks with compiler-enforced serialized access; `@MainActor` makes the UI-thread requirement statically checked — Alltag’s `AppEnvironment` , `LanguageStore` , and `PersistenceController` are all `@MainActor` .
- `Sendable` plus `SWIFT_STRICT_CONCURRENCY: complete` gives compile-time data-race safety Java has no equivalent for; `@unchecked Sendable` (e.g. `InMemoryKeyStore`) is the documented escape hatch, and `complete` is the Swift 6 default in waiting.

Next: *The SwiftUI Model: Declarative Views and State*

PART II

SwiftUI

A declarative UI framework where the screen is a pure function of state. If you have written a templating engine or a reactive stream, the mental model will click fast — then surprise you.

The SwiftUI Model: Declarative Views and State

You have built UIs before, just not this kind. On the server you emitted HTML — Qute templates in Quarkus, maybe Thymeleaf or JSF earlier — where a request comes in, you compute a model, render a string, and the connection closes. The output is dead the moment it leaves the socket. If the data changes, the client must ask again. On the desktop side you may have touched imperative toolkits (Swing, or iOS’s own UIKit) where a view is a long-lived mutable object you hold a reference to and poke: `label.setText(...)`, `view.hidden = true`. SwiftUI is neither of those, and the gap is where every backend engineer stumbles first.

This chapter is about the single idea that makes the rest of SwiftUI make sense: **a view is a value, not an object you mutate**. Get this and the API stops feeling magical. Miss it and you will fight the framework for weeks.

A view is a value, not a thing you keep

Here is the app’s root, verbatim:

```
@main
struct AlltagApp: App {
    @State private var env = AppEnvironment.live()

    var body: some Scene {
        WindowGroup {
            RootView()
                .environment(env)
        }
    }
}
```

`RootView` is a `struct`. Not a class. When SwiftUI wants to know what the UI looks like, it *constructs* a `RootView` value and reads its `body`. That value is a lightweight description — “a tab view containing these five screens, tinted teal, rounded font.” It is not the pixels, and it is not a retained object with identity that you will mutate later. SwiftUI may create and throw away `RootView` values dozens of times per second. They are cheap, ephemeral recipes.

Compare this to a Qute template. Your template is also a *description* of output, and you also don’t hold a reference to the rendered HTML — you produce it and forget it. So far so familiar. The difference is that Qute renders **once per request** and SwiftUI re-renders **whenever the state it read changes**, automatically, for the life of the screen. The template is reactive and persistent; the server template is one-shot. SwiftUI is closer to a server-rendered template that re-runs itself the instant any input changes — with the framework, not you, deciding when.

Now contrast the imperative side. In UIKit (or Swing) you would write a `UIViewController`, keep an outlet to a label, and on a button tap call `label.text = greeting`. You mutate the live object tree. The screen is the source of truth, and your job is to keep it in sync with your data by hand. This is

exactly the bug factory SwiftUI deletes: forget one `label.text =` and the screen lies about your data.

Quarkus analogy. Think of `body` as a render method that Quarkus calls for you on every model change, not a constructor for a stateful component. You never `new` a view and store it. You describe what *should* be on screen for the current state; the framework computes the diff against what *is* on screen and mutates the real UIKit objects underneath. You write declarative; SwiftUI runs imperative for you.

The View protocol, body, and some View

Every screen conforms to one protocol:

```
struct HomeView: View {
    var body: some View {
        PlaceholderScreen(
            titleKey: "tab_home", systemImage: "house")
    }
}
```

`View` is a protocol with one required member: a computed property `body`. That is the whole contract — “I can describe myself.” In Java terms it is an interface with a single method, and `HomeView` is your implementation.

The return type is `some View`. This is an **opaque return type**: the method returns *one specific concrete type that conforms to View*, but the caller is not told which one. The actual type here is a deeply nested generic — something like `PlaceholderScreen` wrapped in modifier types — and it would be absurd to write out by hand. `some View` says “trust me, it’s a single `View` type, you don’t need its name.”

Why not just `-> View` (the protocol as a type, like returning a Java interface reference)? Because SwiftUI’s diffing needs the *static* concrete type at compile time to compare two renders efficiently without boxing every view behind a pointer. `some View` gives the compiler the exact type while hiding it from you. The nearest Java analogy is a method returning `var`-inferred concrete type rather than the interface — except Swift lets you keep that concreteness in the signature while still hiding the name. The practical rule: write `some View` for `body` and move on.

The render/diff loop

The mechanism is a tight loop, and it is the same loop for every SwiftUI app:

1. SwiftUI reads a view’s `body`, producing a value tree.
2. It records *which pieces of state that view read* while computing `body` — this dependency tracking is automatic.
3. It diffs the new value tree against the previous one and applies the minimal changes to the real (UIKit) view objects.
4. When any tracked state changes, SwiftUI re-invokes `body` for *only the views that read that state*, and goes back to step 3.

This is a virtual-DOM-style reconciliation, if you’ve met React, but without a manual `setState` call site — the dependency is captured by the act of reading the property. You do not call “re-render.” You mutate state; the framework noticed you read it; it recomputes.

Two consequences fall out of this loop, and both are load-bearing:

Views must be cheap. `body` may run constantly. If your `body` does real work — a network call, a database query, sorting ten thousand rows — you pay that cost on every recompute. Keep `body` to pure assembly of subviews. Heavy work belongs in your model layer, computed once, not in the render path.

Views must be side-effect-free. `body` is a pure function of state. Do not start a timer, write to disk, or fire an analytics event inside `body`. SwiftUI may call `body` more or fewer times than you expect, in any order. Side effects go in explicit hooks (`.task { }`, `.onAppear`, `.onChange`) that the framework calls deliberately. Treat `body` exactly as you would treat a Quarkus template expression: it reads the model and produces output, nothing more.

State primitives: where the truth lives

If views are disposable values recreated constantly, where does data that *survives* re-renders live? In **property wrappers** that SwiftUI manages outside the struct’s lifetime. The struct is reborn on every render; the state behind these wrappers persists. There are four you need, and choosing the right one is most of what “thinking in SwiftUI” means.

@State — local source of truth

```
struct RootView: View {
    @State private var selection: AppTab = .home
    // ...
}
```

`@State` declares a piece of mutable state **owned by this view**. The `RootView` struct is recreated on every render, but the `AppTab` value behind `@State` lives in storage SwiftUI keeps alive across those re-creations. When you write `selection = .docs`, two things happen: the value updates, and SwiftUI marks `RootView`’s `body` for recompute.

Rules of thumb, anchored to your instincts:

- `@State` is **private** and small. It is the view’s own scratch memory — a selected tab, a toggle, the text in a field. Think request-scoped local state, not a shared bean.
- It is the **single source of truth** for that piece of data. The screen renders *from* it; nothing else owns it.
- Initialize it inline with a default. SwiftUI uses that only on first creation, then preserves the live value.

Notice `AlltagApp` also uses `@State` for `env`, the whole DI container. That is the second valid use of `@State`: holding a reference-type model you want to *create once and keep alive* for the lifetime of the view. `@State` here isn’t “small local data” — it’s “I own this object and it must outlive my renders.”

@Binding — a two-way reference to someone else’s state

A child view often needs to *read and write* state owned by a parent. It must not copy it (copies don’t write back) and it must not own it (the parent already does). `@Binding` is a typed read/write reference into another view’s source of truth.

Look at how `RootView` hands its `selection` to `TabView`:

```
TabView(selection: $selection) {  
    // ...  
}
```

The `$` is the key. Every `@State` (and several other wrappers) exposes a **projected value** reachable with the `$` prefix. For `@State var selection`, plain `selection` is the `AppTab` value, and `$selection` is a `Binding<AppTab>` — a two-way handle. `TabView` takes that binding so that when the user taps a tab, `TabView` writes the new value *back through the binding* into `RootView`’s `@State`. The arrow goes both ways: parent → child for display, child → parent for edits.

If you have used `&` to pass an out-parameter in C, or an `AtomicReference` you hand to a collaborator so it can mutate the referent, that is the shape of a `Binding`: a reference to a slot, not the value in it. The closest everyday Java analogy is passing a setter+getter pair as a single object.

Note. The split is simple: own state with `@State`, lend it with `$state` as a `Binding`, receive it with `@Binding`. A `Binding` never creates a new source of truth — it always points at one that lives somewhere else. Unidirectional data flow holds: there is exactly one owner per piece of state, and edits route back to that owner.

@Observable + @Environment — shared, injected state

`@State` and `@Binding` cover state that lives *in the view tree*. But cross-cutting state — the logged-in user, the persistence layer, the selected language and theme — should live in a **model object** shared across many screens, exactly like an injected application-scoped bean in Quarkus. SwiftUI’s modern answer is the `@Observable` macro:

```
@Observable  
final class AppEnvironment {  
    var language: AppLanguage  
    var theme: ThemeSetting  
    // ... persistence, etc.  
}
```

(`AppEnvironment` in Alltag is the DI container holding language, theme, and persistence.) Marking a class `@Observable` makes SwiftUI track reads of its properties automatically. A view that reads `env.language` is recomputed when — and only when — `language` changes. You get per-property reactivity for free, no annotations on each field.

How does a deeply nested view *get* the shared model without threading it through every initializer? Through the **environment**, SwiftUI’s built-in dependency injection. The app injects it once:

```
RootView()
    .environment(env)
```

and any descendant pulls it out by type:

```
struct RootView: View {
    @Environment(AppEnvironment.self) private var env
    // ...
}
```

`@Environment(AppEnvironment.self)` means “find the `AppEnvironment` that an ancestor put into the environment and bind it here.” This is CDI for the view tree: `.environment(env)` is the producer/binding, `@Environment(...)` is the injection point, and the type is the lookup key. No view in between needs to know the model exists. When `env`’s observed properties change, every view that read them re-renders.

Quarkus analogy. `.environment(env)` is `@Produces` for an application-scoped bean; `@Environment(AppEnvironment.self)` is the matching `@Inject`. The difference is scope: the environment is scoped to a *subtree* of the UI, so children can override what ancestors provided — a feature plain CDI doesn’t give you.

@Bindable — bindings into an @Observable

One gap remains. From `@State` you get bindings with `$`. But `@Environment(AppEnvironment.self) var env` is a plain reference, not a binding source — how do you hand a `TextField` a two-way binding to `env.language`? You re-wrap it with `@Bindable`:

```
@Bindable var env = env
TextField("name", text: $env.userName)
```

`@Bindable` takes an `@Observable` object and lets you project bindings (`$env.userName`) into its individual properties. Use it when a control needs to *write back* into a shared model. Reading alone needs only `@Environment`; two-way editing of a model property needs `@Bindable` to mint the binding.

The full picture, four wrappers, one decision tree:

Wrapper	Use when
<code>@State</code>	This view owns the truth
<code>@Binding</code>	You edit a parent’s truth
<code>@Environment</code>	You read injected shared state
<code>@Bindable</code>	You edit an <code>@Observable</code> ’s field

The old pattern you will still see

Before the `@Observable` macro (pre-iOS 17), shared models used the `ObservableObject` protocol, marked changing fields with `@Published`, were owned with `@StateObject`, and injected with `@EnvironmentObject` / `@ObservedObject`. You will meet this in tutorials and older code:

```
// Older code you'll see – not how Alltag writes it:
final class OldEnv: ObservableObject {
    @Published var language: AppLanguage = .de
}
```

It works, but it is coarser (any `@Published` change notifies *all* observers, not just those reading that field) and noisier (every field needs `@Published`). **Modern code uses `@Observable`**. When you start a new model, reach for `@Observable`; only touch `ObservableObject` when maintaining existing code.

View identity and lifetime

Since views are recreated constantly, how does SwiftUI know that “this `HomeView` now” is the same one as “that `HomeView` a frame ago” — so it can *preserve* its `@State` rather than reset it? By **identity**. By default, a view’s identity is its position in the view tree (structural identity). In a `ForEach`, identity comes from an explicit ID:

```
ForEach(AppTab.allCases) { tab in
    screen(for: tab)
        .tabItem {
            Label(tab.titleKey, systemImage: tab.systemImage)
        }
        .tag(tab)
}
```

`AppTab` is `Identifiable` (its `id` is the raw string), so `ForEach` tracks each generated screen by that stable id. Identity is what ties the disposable view *value* to the persistent *state* behind its wrappers. Change a view’s identity and SwiftUI tears down its state and builds fresh; keep it stable and the state rides along across every re-render. This is why `Identifiable` ids must be stable and unique — they are the primary keys of your UI.

Previews

One payoff of value-type views: you can construct any screen in isolation, no running app, no simulator boot, no navigating five taps deep. The `#Preview` macro renders a view live in Xcode’s canvas:

```
#Preview {
    RootView()
        .environment(AppEnvironment.live())
}
```

Because `RootView` is just a value and its dependency arrives through `.environment(...)`, the preview wires up a real `AppEnvironment` and shows the whole tab shell instantly. This is the unit-test ergonomics of pure functions applied to UI: deterministic input (injected state) in, rendered view out. Build models to be constructible in a preview and your UI becomes as testable as your service layer.

Takeaways

- A SwiftUI `View` is a value-type `struct` describing UI for the current state — not a retained, mutable object (UIKit) and not a one-shot server template. SwiftUI re-runs `body` on state change and diffs the result onto real views.
- `body: some View` returns one hidden concrete `View` type. Keep `body` cheap and side-effect-free; it may run constantly.
- Four state primitives: `@State` (own the truth), `@Binding` + `$projection` (edit a parent's truth), `@Observable` model via `@Environment` (read injected shared state), `@Bindable` (edit an observable's field). One owner per piece of state; edits flow back to it — unidirectional data flow.
- `.environment(env)` plus `@Environment(AppEnvironment.self)` is CDI for the view tree, scoped to a subtree.
- Modern code uses the `@Observable` macro; `ObservableObject` / `@Published` / `@StateObject` is the older pattern you'll still read.
- View identity (structural, or `Identifiable` id in `ForEach`) ties disposable view values to persistent wrapped state. `#Preview` renders any view in isolation by injecting its dependencies.

Next: *Layout: Stacks, Modifiers, and the Layout System*

Layout: Stacks, Modifiers, and the Layout System

You know how to declare a view and where its state lives. Now: how does it get *positioned and sized*? SwiftUI has no `<div>`, no absolute coordinates you set, no constraint solver you configure by hand (UIKit's Auto Layout did that, and it was miserable). Instead it uses a small, predictable size-negotiation algorithm plus composable **stacks** and **modifiers**. Once you internalize the negotiation, layout stops being guesswork.

Stacks: the three containers

Almost all layout is three container views that arrange their children along an axis:

- **HStack** — children left-to-right (a row).
- **VStack** — children top-to-bottom (a column).
- **ZStack** — children back-to-front (overlapping, shared center).

You saw all the pieces in the placeholder screen:

```
ZStack {
  AppColor.paper.ignoresSafeArea()
  VStack(spacing: 12) {
    Image(systemName: systemImage)
      .font(.system(size: 44))
      .foregroundColor(AppColor.primary)
    Text(titleKey)
      .font(.title2.weight(.semibold))
      .foregroundColor(AppColor.ink)
    Text("placeholder_coming_soon")
      .font(.subheadline)
      .foregroundColor(AppColor.inkSoft)
  }
  .padding()
}
```

The **ZStack** layers a full-bleed paper background behind a centered **VStack** of icon + two text lines. The **VStack** stacks its three children vertically with **12** points between each.

Alignment and spacing

Each stack takes an **alignment** and a **spacing**:

```
VStack(alignment: .leading, spacing: 12) { ... }
HStack(alignment: .top, spacing: 8) { ... }
```

The crucial subtlety: **alignment is on the cross axis**. A **VStack** arranges *vertically*, so its **alignment** controls *horizontal* placement (**.leading**, **.center**, **.trailing**). An **HStack**'s alignment is *vertical*

(`.top`, `.center`, `.bottom`). `spacing` is along the main axis — the gap *between* children. Omit `spacing` and SwiftUI picks a sensible default per platform; pass `0` to butt children together.

Note. `.leading` and `.trailing`, not left/right. In Alltag, `RootView` sets `.environment(\.layoutDirection, env.language.layoutDirection)` so Arabic flips the whole UI to RTL. Because you wrote `.leading` (not `.left`), every stack mirrors for free. Hard-coded `.left` would break right-to-left users. Always think in leading/trailing.

Spacer and frame

A `Spacer` is a greedy empty view that expands to fill available space along its stack’s axis, pushing siblings apart:

```
HStack {
    Text("Alltag")
    Spacer()           // shoves the gear to the trailing edge
    Image(systemName: "gearshape")
}
```

`frame` constrains or expands a view’s size:

```
Image(systemName: "house")
    .frame(width: 44, height: 44)           // fixed
Text("Hi").frame(maxWidth: .infinity)      // grow to fill width
```

`maxWidth: .infinity` is the SwiftUI idiom for “take all the width my parent offers” — used constantly to make cards and rows fill their container.

Modifiers: order matters

This is the part that trips up everyone from a CSS background. A **modifier** is a method on a view that returns a *new view wrapping the original*. `someView.padding()` does not mutate `someView`; it returns a `Padding<SomeView>` value. Chaining modifiers builds an onion of wrapper views from the inside out. Because each modifier operates on the result of the previous one, **order changes the result**.

The canonical example — padding vs. background:

```
// A: pad first, then background
Text("Hi")
    .padding()
    .background(AppColor.card)

// B: background first, then pad
Text("Hi")
    .background(AppColor.card)
    .padding()
```

In **A**, `.padding()` wraps the text in 16 points of space, *then* `.background` colors that padded region — you get a colored card with the text inset and breathing room inside. In **B**, `.background` colors the tight text bounds first, *then* `.padding` adds *transparent* space around the already-colored box —

a small colored chip with empty margin around it. Same modifiers, opposite visuals, purely from order.

CSS muddies this intuition. In CSS, `padding` and `background` are properties of one box and there is no ordering between them. SwiftUI has no “box with properties”; it has a *pipeline of nested views*, and each modifier sees only what is inside it. Read a modifier chain inside-out, like nested function calls: `background(padding(text))` vs. `padding(background(text))`. That reframing makes the difference obvious.

Gotcha. When a card looks wrong, suspect modifier order before anything else. Background bleeding to the wrong bounds, tap-targets the wrong size, a `cornerRadius` clipping the inset instead of the fill — almost always an ordering bug. Sketch the onion: which modifier wraps which?

The layout algorithm: a three-step negotiation

Here is the mental model that explains everything. SwiftUI lays out in a single top-down then bottom-up pass, and for every parent/child pair it is three steps:

1. **The parent proposes a size.** The root is proposed the safe-area rectangle. Each container proposes some size to each child (a `VStack` proposes its full width and a share of height; a frame proposes its fixed dimensions; and so on).
2. **The child chooses its own size.** The child is *not* forced to take what was proposed. A `Text` takes only as much width as its glyphs need (then wraps). An `Image` takes its intrinsic size. `Color` and `Spacer` greedily take everything offered. `frame` reports the size you asked for. The child returns *its* chosen size.
3. **The parent positions the child.** Knowing the child’s chosen size, the parent places it within its own bounds per the alignment.

The whole tree is this conversation, recursively. The parent *proposes*, never *dictates*; the child *decides*. This is the inverse of Auto Layout, where you wrote constraints and a solver reconciled them globally. SwiftUI’s local, two-way negotiation has no solver and no constraint conflicts — but you must reason about who proposes and who chooses.

This single model explains the FAQs:

- *Why doesn’t my `Text` fill the width?* Because in step 2 `Text` chooses its glyph width, ignoring the wider proposal. Force it with `.frame(maxWidth: .infinity)` so the frame (which *does* accept the proposal) reports full width.
- *Why does `Color` fill everything?* Because in step 2 `Color` accepts any proposal — it is greedy. That is exactly why `AppColor.paper.ignoresSafeArea()` works as a full-screen background in the placeholder.
- *Why does `.padding()` shrink the proposal?* Because padding subtracts its inset from the proposal before passing it to its child, then reports the child’s size plus the inset.

GeometryReader: use sparingly

Sometimes a child needs the actual proposed size — to draw something proportional, say half the parent’s width. `GeometryReader` exposes it:

```
GeometryReader { proxy in
    Rectangle()
        .frame(width: proxy.size.width * 0.5)
}
```

The catch, straight from the algorithm: `GeometryReader` is greedy — in step 2 it accepts the *entire* proposed size, like `Color`. Drop one into a `VStack` and it expands to fill, shoving your other content. So use it in leaf positions, scoped tightly, only when you genuinely need a measured dimension. Reaching for `GeometryReader` to solve a layout that stacks, frames, and alignment could handle is the most common SwiftUI anti-pattern. Try the negotiation first.

Scrolling and lazy content

`ScrollView` makes its content scrollable. By itself it does *not* arrange anything — pair it with a stack:

```
ScrollView {
    VStack(spacing: 16) { /* rows */ }
}
```

A plain `VStack` builds *all* children immediately. For a long or unbounded list that is wasteful, so use the lazy variants, which build children only as they scroll into view:

- `LazyVStack` / `LazyHStack` — lazy linear stacks inside a `ScrollView`.
- `LazyVGrid` / `LazyHGrid` — lazy grids defined by `GridItem` columns.
- `Grid` — a non-lazy, true 2-D grid with aligned rows and columns, for small fixed layouts (like a form-ish table).

Rule of thumb: bounded, small content → `VStack` / `Grid`; long or data-driven content → `Lazy*` (or `List`, covered next chapter, which is lazy and adds platform row styling).

Safe area

The **safe area** is the region clear of the notch, home indicator, status bar, and tab bar. SwiftUI insets your content to it by default — which is why your text never hides under the clock. A *background* should usually ignore it to reach the screen edges, exactly as the placeholder does:

```
AppColor.paper.ignoresSafeArea()
```

The paper color floods the entire screen; the foreground `VStack` stays safely inset. Ignore the safe area for decoration, respect it for content.

Adaptive layout: Dynamic Type and never hard-coding sizes

iOS users can scale text system-wide (accessibility, large-text modes). A layout that hard-codes point sizes ignores that setting and breaks for those users. Alltag’s typography is deliberately built to scale. The *entire* font styling is one root modifier:

```
extension View {
    /// Applies the app's rounded type design to this subtree.
    func appFontDesign() -> some View {
        fontDesign(.rounded)
    }
}
```

applied once in `RootView` as `.appFontDesign()`. Note what it does *not* do: it sets the *design* (rounded SF Pro) but never a point size. Throughout the app, text uses **semantic font roles** — `.title2`, `.subheadline`, `.body` — as in the placeholder:

```
Text(titleKey).font(.title2.weight(.semibold))
Text("placeholder_coming_soon").font(.subheadline)
```

Semantic roles scale automatically with the user’s Dynamic Type setting; `.font(.system(size: 17))` would not. (The icon’s `.font(.system(size: 44))` is the deliberate exception — a glyph, not body text.) Because the design is applied once at the root and cascades down the view tree, every screen inherits rounded, scalable type with zero per-view ceremony. This is the layout payoff of the modifier model: a single modifier on `RootView` reaches every descendant.

Quarkus analogy. `.appFontDesign()` at the root is like setting a base theme in your master Quete template that every page extends — define it once, every screen inherits it. The difference is that SwiftUI’s cascade flows through the *runtime* view tree via the environment, so a subtree can override it locally without touching the root.

Color and light/dark via semantic tokens

The same “define once, resolve at use” philosophy drives color. Alltag never writes a raw hex in a view; it uses **semantic tokens** from `AppColor`, each a dynamic light/dark pair:

```
enum AppColor {
    static let paper =
        dynamic(light: 0xFAF8F4, dark: 0x1B1916)
    static let ink =
        dynamic(light: 0x2B2722, dark: 0xF2EEE6)
    static let primary =
        dynamic(light: 0x2BA39A, dark: 0x35B6AC) // teal
}
```

Each token resolves to the right shade for the active `ColorScheme` *automatically* — the `dynamic` helper wraps a `UIColor` that picks its value from the current trait collection. A view just says `.foregroundColor(AppColor.ink)` and gets dark ink in light mode, light ink in dark mode, no conditionals. The names are *semantic* (`paper`, `ink`, `primary`), not *literal* (`beige`, `tealHex`), so the palette is a

single reviewable file and a redesign touches one place, not every view. This is exactly how you'd centralize a color scheme behind named constants on the server — except these constants are *context-resolving*, returning a different value per appearance mode at render time.

The app-wide accent uses the same token. `RootView` applies:

```
.tint(AppColor.primary)
.appFontDesign()
.preferredColorScheme(env.theme.theme.colorScheme)
```

`.tint(AppColor.primary)` cascades the teal accent to interactive elements (selected tab, buttons) across the whole tree. `.preferredColorScheme(...)` lets the user's theme setting *force* light or dark (or follow the system when nil). All three are root modifiers cascading to every screen — the modifier-cascade pattern doing triple duty: font, color accent, and appearance, each defined once.

Takeaways

- Layout is `HStack` / `VStack` / `ZStack` plus `Spacer`, `padding`, and `frame`. A stack's `alignment` is the *cross* axis; `spacing` is the *main* axis. Use `.leading` / `.trailing`, not left/right, so RTL (Arabic) mirrors for free.
- Modifiers wrap views into a nested onion — `someView.padding()` returns a new view. **Order matters:** pad-then-background vs. background-then-pad give different results. Read chains inside-out.
- The layout algorithm is a 3-step negotiation: parent *proposes* a size, child *chooses* its own, parent *positions* it. This single model explains why `Text` won't fill width, why `Color` / `Spacer` are greedy, and what `padding` / `frame` do.
- `GeometryReader` is greedy and should be used sparingly, only as a scoped leaf. Use `ScrollView` + `Lazy*` for long content; respect the safe area for content, ignore it for backgrounds.
- Never hard-code point sizes: `.appFontDesign()` sets design once at the root and views use semantic roles (`.title2`) so Dynamic Type scales. Color uses `AppColor` semantic light/dark tokens that resolve to the active `ColorScheme`; `.tint`, `.appFontDesign`, and `.preferredColorScheme` cascade from `RootView` to every screen.

Next: *Building Screens: Lists, Navigation, Forms, Tabs*

Building Screens: Lists, Navigation, Forms, Tabs

By now you can compose a view and lay it out. This chapter assembles those primitives into *real screens*: a tabbed shell, drill-down navigation, scrollable lists, data-entry forms, and the modal surfaces (sheets, alerts) that sit on top. Everything here maps to things you have built a hundred times on the server and the web — a router, a list endpoint rendered to a table, a form POST — except the framework is declarative and the “router” is a value you mutate.

We anchor every primitive to the Alltag app: a five-tab, on-device German-bureaucracy companion. Its shell, tabs, and Settings screen are real code we will read verbatim.

The tab shell: `TabView`

The top-level container for most iOS apps is `TabView`. Think of it as the app’s coarse router — the equivalent of the top-level path segments (`/home`, `/docs`, `/settings`) that a Quarkus REST app exposes via JAX-RS `@Path`. The difference: there is no dispatch on a request. The selected tab is a piece of state, and SwiftUI re-renders when it changes.

Alltag’s information architecture fixes five destinations, with Decode deliberately in the center. Rather than inline the tabs in the view, the app models them as an `enum`:

```
enum AppTab: String, CaseIterable, Identifiable {
    case home, docs, decode, dates, settings

    var id: String { rawValue }

    var titleKey: LocalizedStringKey {
        switch self {
        case .home: return "tab_home"
        case .docs: return "tab_docs"
        case .decode: return "tab_decode"
        case .dates: return "tab_dates"
        case .settings: return "tab_settings"
        }
    }

    var systemImage: String {
        switch self {
        case .home: return "house"
        case .docs: return "folder"
        case .decode: return "doc.text.viewfinder"
        case .dates: return "calendar"
        case .settings: return "gearshape"
        }
    }
}
```

`CaseIterable` gives a synthesized `allCases` — your enum becomes its own ordered registry. `Identifiable` (here `id` is the `rawValue`) lets `ForEach` iterate it. Modeling tabs as a type means the set

and its ordering are testable and the center placement is explicit, not a fragile consequence of view declaration order. This is the kind of small thing a backend engineer appreciates: configuration as data.

The shell that consumes it, `RootView`:

```
struct RootView: View {
    @Environment(AppEnvironment.self) private var env
    @State private var selection: AppTab = .home

    var body: some View {
        TabView(selection: $selection) {
            ForEach(AppTab.allCases) { tab in
                screen(for: tab)
                .tabItem {
                    Label(tab.titleKey,
                        systemImage: tab.systemImage)
                }
                .tag(tab)
            }
        }
        .tint(AppColor.primary)
        .appFontDesign()
        .environment(\.locale, env.language.locale)
        .environment(\.layoutDirection,
            env.language.layoutDirection)
        .preferredColorScheme(env.theme.theme.colorScheme)
    }

    @ViewBuilder
    private func screen(for tab: AppTab) -> some View {
        switch tab {
        case .home: HomeView()
        case .docs: VaultView()
        case .decode: DecoderView()
        case .dates: DatesView()
        case .settings: SettingsView()
        }
    }
}
```

Three load-bearing pieces:

- `TabView(selection: $selection)` binds the active tab to state. The `$` makes a two-way **Binding** — when the user taps a tab, SwiftUI writes back into `selection`. (Bindings are covered in the architecture chapter; for now: a writable reference to state.)
- `.tabItem { Label(...) }` declares the bar item. `Label` pairs a localized title with an SF Symbol (`systemImage`). SF Symbols are Apple’s bundled icon font — no asset import.
- `.tag(tab)` associates each tab’s content with its `AppTab` value so `selection` can identify it. The tag type must match the selection type.

The modifiers chained after `TabView` apply app-wide: tint color, font design, locale, layout direction, and color scheme. Note `@ViewBuilder` on `screen(for:)` — it lets a function whose branches return *different* view types compile as a single `some View`, the way the `body` property itself works.

A subtlety that catches server engineers: `TabView` keeps every tab's view *alive*. Switching from Decode to Settings and back does not reconstruct `DecoderView` — its `@State` survives. This is the opposite of a stateless HTTP handler, where each request rebuilds everything. The view tree is long-lived and SwiftUI mutates it in place; reach for that model whenever you wonder “will my scroll position / partially typed text survive a tab switch?” It will. Conversely, do not stash request-scoped junk in a tab's `@State` expecting it to reset — it won't.

Quarkus analogy. `TabView` is your top-level router and `AppTab` is the route table. But there is no servlet dispatch and no request lifecycle — the “current route” is just a `@State` value, and switching tabs keeps all five screens alive (their state persists), unlike a stateless HTTP handler that is reconstructed per request.

Drill-down: NavigationStack

Tabs are lateral. To push a detail screen on top of a list — the bureaucratic-letter list to a single letter's decoded view — you need a navigation stack.

Modern SwiftUI uses `NavigationStack`. You will see older tutorials use `NavigationView`; it is deprecated and behaves differently (it doubles as a split view on iPad and has murkier programmatic control). Treat `NavigationView` as legacy. Always reach for `NavigationStack`.

The simplest form wraps a root and reacts to `NavigationLink` s:

```
NavigationStack {
    List(letters) { letter in
        NavigationLink(letter.subject) {
            DecodedLetterView(letter: letter)
        }
    }
    .navigationTitle("tab_decode")
}
```

For anything beyond toy apps, prefer **path-based** navigation: the stack is driven by an explicit array you own. This is the SwiftUI feature that finally makes navigation feel like server-side routing — deep links, “pop to root,” and restoring a navigation path become array mutations.

```
@State private var path: [Letter] = []

var body: some View {
    NavigationStack(path: $path) {
        List(letters) { letter in
            NavigationLink(letter.subject, value: letter)
        }
        .navigationDestination(for: Letter.self) { letter in
            DecodedLetterView(letter: letter)
        }
    }
}
```

`NavigationLink(value:)` does not name a destination view; it appends a *value* to `path`. A `.navigationDestination(for:)` modifier declares, “when a `Letter` is on the stack, render it like this.” That decoupling is the point — you can also push programmatically with `path.append(letter)`, clear the whole stack with `path.removeAll()`, or restore a saved path on launch. The destination type must conform to `Hashable`.

Gotcha. `.navigationTitle`, `.toolbar`, and `.navigationDestination` attach to the *content inside* the stack, not to `NavigationStack` itself. Put them on the `List` (or root view), not on the wrapper. A title set on the `NavigationStack` line silently does nothing.

Lists

`List` is the workhorse for vertical, scrollable, row-based content — the screen equivalent of rendering a query result to an HTML table, except it virtualizes rows for you.

Static rows read like markup:

```
List {
    Text("Anmeldung")
    Text("Steuer-ID")
    Text("Aufenthaltstitel")
}
```

Dynamic rows iterate a collection. As with `ForEach`, SwiftUI needs a stable identity per row to diff updates, so the element type conforms to `Identifiable`:

```
List(documents) { doc in
    Label(doc.title, systemImage: doc.icon)
}
```

`Identifiable` is the SwiftUI analogue of a JPA `@Id`: a stable key the framework uses to track a row across re-renders, decide what animates, and recycle views. Without it, deletions and moves can animate the wrong row.

Group rows into **sections** and add headers:

```
List {
    Section("Pending") {
        ForEach(pending) { Text($0.title) }
    }
    Section("Filed") {
        ForEach(filed) { Text($0.title) }
    }
}
```

Rows can carry leading/trailing swipe actions — the iOS gesture you use to delete a mail:

```

ForEach(documents) { doc in
  Text(doc.title)
    .swipeActions(edge: .trailing) {
      Button(role: .destructive) {
        store.delete(doc)
      } label: {
        Label("Delete", systemImage: "trash")
      }
    }
}
}

```

`role: .destructive` colors the button red and lets the OS treat it as the default full-swipe action. The `.onDelete` modifier offers a terser path when you only need delete.

For a long document list, add a search field with one modifier:

```

List(filtered) { doc in Text(doc.title) }
  .searchable(text: $query, prompt: "Search documents")

```

`.searchable` installs the system search bar and binds the query to your state; you filter `documents` yourself in a computed property. There is no server round-trip — filtering is local, synchronous, over data already in memory. Treat it like running a `Stream.filter` over a loaded collection, not like issuing a new query endpoint.

Finally, the list's own lifecycle. Use `.task` to load data when a screen appears:

```

List(documents) { doc in Text(doc.title) }
  .task { await store.load() }

```

`.task` runs an `async` closure when the view appears and *cancels it automatically* when the view disappears — the correct hook for fetching or, in Alltag, kicking off on-device work. Prefer it over the older `.onAppear`, which is synchronous and has no cancellation.

Forms and controls

`Form` is a `List` styled for data entry — grouped, inset rows with platform-correct spacing. It is where Alltag's Settings live. The controls you will use constantly:

Control	Purpose	Server analogue
<code>TextField</code>	single-line text	text input
<code>Toggle</code>	boolean	checkbox
<code>Picker</code>	one-of-N	select / radio
<code>Stepper</code>	bounded integer	number input

A tour:

```
Form {
    Section("Reminder") {
        TextField("Title", text: $title)
        Toggle("Notify me", isOn: $notify)
        Stepper("Days before: \$(days)",
            value: $days, in: 0...30)
        Picker("Persona", selection: $persona) {
            ForEach(Persona.allCases) { p in
                Text(p.label).tag(p)
            }
        }
    }
}
```

Every control binds to state with `$`. There is no submit button and no form-encoded POST — the bindings *are* the two-way data flow. The model updates as the user types or taps; you read it whenever you need it. A backend engineer should reframe a SwiftUI `Form` not as an HTML `<form>` you serialize on submit, but as a live, bidirectional binding between widgets and an in-memory model.

`Picker`'s `selection` must match the tag type of its options. Each option carries `.tag(p)` so SwiftUI knows which value the row represents — the exact mechanism you saw on `TabView`.

Modals, alerts, and toolbars

Pushing onto a stack is one way to present; *covering* the current screen is another. SwiftUI presents modals with view modifiers driven by state, not by imperatively calling a “present” method.

- `.sheet(isPresented:)` — a card that slides up, dismissible, partially covering. The default for most modal flows.
- `.fullScreenCover(isPresented:)` — opaque, full-screen; use for immersive flows like the camera capture in Decode.
- `.alert(_:isPresented:)` — a blocking system dialog with title and buttons.
- `.confirmationDialog(_:isPresented:)` — an action sheet of choices, good for “are you sure / pick one.”

```

struct VaultView: View {
    @State private var showingScanner = false
    @State private var confirmDelete = false

    var body: some View {
        List { /* documents */ }
            .toolbar {
                ToolbarItem(placement: .primaryAction) {
                    Button("Scan") { showingScanner = true }
                }
            }
            .sheet(isPresented: $showingScanner) {
                ScannerView()
            }
            .alert("Delete document?",
                isPresented: $confirmDelete) {
                Button("Delete", role: .destructive) {}
                Button("Cancel", role: .cancel) {}
            }
    }
}

```

The pattern is uniform: a `Bool` (or optional value) drives presentation, and the OS owns the lifecycle and the dismiss gesture. Set the flag false — or let the user swipe down — and the modal goes away. `.toolbar` declares bar buttons; `placement` lets the OS position them correctly per platform (and mirror them under RTL).

Note. There is also `.sheet(item:)`, driven by an optional `Identifiable` value instead of a `Bool`. Prefer it when the sheet needs data: setting `item` to a non-nil document both presents the sheet and hands it the document, avoiding the classic “flag is true but the data is stale” bug.

Two presentation traps worth naming up front. First, modifier *order* and *attachment point*: presentation modifiers attach to the view they decorate, and the presented content is a sibling of that view in the tree, not a child of wherever you “called” present. In practice, attach `.sheet` / `.alert` to a stable container (the `List`, the root of the screen), not to a transient row, or the modal can dismiss itself when its host row is recycled. Second, a sheet does *not* inherit the navigation stack of the screen that presented it. If the sheet’s content needs its own push navigation or its own title bar, wrap that content in its own `NavigationStack`. This surprises engineers who expect a presented view to live “inside” the caller’s route the way a forwarded request shares a servlet context — it does not; a sheet is a fresh presentation context.

Localization: LocalizedStringKey and the catalog

You may have noticed Alltag passes bare strings like `"tab_home"` and `"settings_appearance"` where you expected display text. That is deliberate.

Most SwiftUI text-accepting APIs take a `LocalizedStringKey`, not a `String`. When you write `Text("tab_home")`, the literal is treated as a *key* and resolved at render time against the app’s **String Catalog** (`Localizable.xcstrings`). The catalog is a JSON file Xcode manages; Alltag’s holds:

```

"tab_home" : {
  "localizations" : {
    "de" : { "stringUnit" :
      { "state" : "translated", "value" : "Start" } },
    "en" : { "stringUnit" :
      { "state" : "translated", "value" : "Home" } }
  }
}

```

So `Text(tab.titleKey)` renders “Start” in German and “Home” in English. This is your resource bundle / `messages.properties`, but type-aware: the compiler distinguishes a `LocalizedStringKey` literal (looked up) from a `String` variable (shown as-is). A `String` you pass through a variable is *not* localized — only literals and explicit `LocalizedStringKey`s are.

That is why `AppTab.titleKey` is typed `LocalizedStringKey` rather than `String`: it travels as a key, gets resolved late, and lets the same enum serve all nine of Alltag’s planned languages.

Locale and RTL

Selecting a language in Alltag does two environment writes on the root, both visible back in `RootView`:

```

.environment(\.locale, env.language.locale)
.environment(\.layoutDirection, env.language.layoutDirection)

```

The first changes which catalog column resolves. The second mirrors the entire layout for right-to-left scripts. Arabic is RTL; everything else LTR:

```

var isRTL: Bool { self == .ar }

var layoutDirection: LayoutDirection {
  isRTL ? .rightToLeft : .leftToRight
}

```

Setting `\.layoutDirection` to `.rightToLeft` flips leading and trailing, mirrors the tab bar and toolbars, and reverses swipe edges — for free, because you used semantic `leading` / `trailing` edges instead of hard-coded left/right. Build for RTL from day one and it costs nothing later; retrofit it and you re-audit every screen.

Worked example: the Settings screen

Pulling the chapter together, here is Alltag’s real `SettingsView` — a `Form` inside a `NavigationStack`, reading controllers from the injected environment, exposing a theme picker and a language picker:

```

struct SettingsView: View {
    @Environment(AppEnvironment.self) private var env

    var body: some View {
        NavigationStack {
            Form {
                Section("settings_appearance") {
                    Picker("settings_appearance",
                        selection: themeBinding) {
                        ForEach(AppTheme.allCases) { theme in
                            Text(theme.labelKey).tag(theme)
                        }
                    }
                    .pickerStyle(.segmented)
                }

                Section("settings_language") {
                    Picker("settings_language",
                        selection: languageBinding) {
                        ForEach(AppLanguage.selectable) {
                            language in
                            Text(language.endonym)
                                .tag(language)
                        }
                    }
                }
            }
            .navigationTitle("tab_settings")
        }
    }

    private var themeBinding: Binding<AppTheme> {
        Binding(
            get: { env.theme.theme },
            set: { env.theme.select($0) })
    }

    private var languageBinding: Binding<AppLanguage> {
        Binding(
            get: { env.language.language },
            set: { env.language.select($0) })
    }
}

```

Walk the moving parts:

- `@Environment(AppEnvironment.self)` reads the DI container injected at the app root. No singleton, no service locator — the container is handed down the view tree (architecture chapter next).
- The theme `Picker` iterates `AppTheme.allCases` and labels each with its `labelKey` (`appearance_system` / `appearance_light` / `appearance_dark`), resolved by the catalog. `.pickerStyle(.segmented)` renders the inline three-segment control.
- The language `Picker` iterates `AppLanguage.selectable` — today just `[.de, .en]`, even though the enum defines nine — and labels with `endonym`, each language in its own script (“Deutsch”, “English”), so users recognize their own.

- `.tag(theme)` / `.tag(language)` give each row its value; the picker's `selection` type matches.

The bindings deserve a closer look. The controllers expose `theme` and `language` as `private(set)` — read-only from outside, mutated only through `select(_)`, which also persists to `UserDefaults`. A picker needs a *writable* `Binding`, so Settings bridges read and write with a custom `Binding(get:set:)`: read the current value, and on write, route through `select(_)` instead of assigning the property.

That indirection is not ceremony. `LanguageStore.select(_)` guards against picking a not-yet-localized language:

```
func select(_ language: AppLanguage) {
    guard language.isSelectable else { return }
    self.language = language
    defaults.set(language.rawValue, forKey: Self.storageKey)
}
```

A naive `selection: $env.language.language` would bypass that guard and the persistence side effect. The explicit binding keeps the controller the single point of truth for *how* a selection takes effect. (When you have a `@Bindable` model and no such guard, `$model.property` is the idiomatic shortcut — the architecture chapter covers `@Bindable`.)

Because `ThemeController` and `LanguageStore` are `@Observable` and `RootView` reads the same `env`, a tap here propagates instantly: `select` mutates the controller, observation marks the root dirty, and `RootView` re-applies `preferredColorScheme` and `\.locale` across the whole app. One mutation, app-wide effect, no notification plumbing.

Takeaways

- `TabView` is the coarse router; model tabs as a `CaseIterable` `enum` (`AppTab`) for an explicit, testable, ordered set.
- Use `NavigationStack` (not the deprecated `NavigationView`). Path-based navigation with `.navigationDestination(for:)` turns routing into array mutation.
- `List` / `Form` render rows from `Identifiable` data; controls (`Picker`, `Toggle`, `TextField`, `Stepper`) bind two-way with `$` — no submit step.
- Modals are state-driven modifiers: `.sheet`, `.fullScreenCover`, `.alert`, `.confirmationDialog`. A `Bool` or optional drives presentation; the OS owns dismissal.
- Bare string literals in text APIs are `LocalizedStringKey`s resolved against the String Catalog; `\.locale` and `\.layoutDirection` switch language and RTL app-wide.
- Bridge read-only controllers to pickers with `Binding(get:set:)` so writes route through guarded, persisting `select(_)` methods.

Next: Architecture: State, Dependencies, and the `@Observable` Environment

Architecture: State, Dependencies, and the @Observable Environment

You know how to build screens. The harder question is how to wire an app: where state lives, where business logic goes, and how a view gets the services it needs without reaching for a global. On the server you answer this with a DI container — in Quarkus, CDI scans, builds, and injects beans, and you rarely think about object graphs explicitly. SwiftUI has no such framework, and that is not a gap to fill with a third-party clone. The platform gives you a smaller, sharper set of tools: `@Observable` models and an environment you build *by hand*. This chapter shows how Alltag uses them, and where the CDI muscle memory will mislead you.

MV, not MVVM

The first thing to unlearn is the reflex to add a ViewModel to every screen.

The dominant pre-`@Observable` pattern was **MVVM**: each view got an `ObservableObject` ViewModel publishing `@Published` properties, and the view observed it. This existed largely to work around old SwiftUI: only `ObservableObject` could drive updates, and it did so coarsely (any `@Published` change re-rendered every observer). Teams coming from server MVC cargo-culted a ViewModel per view because it *felt* like the service layer they knew.

iOS 17's `@Observable` macro changes the economics. A plain class annotated `@Observable` drives view updates directly, and crucially the tracking is *fine-grained*: a view re-renders only when a property it actually read changes. There is no `ObservableObject`, no `@Published`, no per-property ceremony.

```
@Observable
final class ThemeController {
    private(set) var theme: AppTheme
    func select(_ theme: AppTheme) { /* ... */ }
}
```

This enables **MV** — Model plus View. The “model” is your domain types and the `@Observable` services that own them; the view reads them directly. You do *not* interpose a pass-through ViewModel whose only job is to forward properties. That object would add indirection, a second source of truth, and tests for glue code that does nothing.

Gotcha. “MV” is not “put everything in the view.” It is “don’t add a ViewModel layer that earns nothing.” Logic still belongs in models and services (below) — just not in a mandatory per-screen wrapper. Add a dedicated observable object when a screen has real presentation state to manage; skip it when the screen only reads shared models.

Alltag has no `SettingsViewModel`. `SettingsView` reads `ThemeController` and `LanguageStore` straight from the environment. The controllers *are* the model layer.

Dependency injection without a framework

So how do those controllers reach the view? Through SwiftUI's **environment** — a typed, hierarchical context propagated down the view tree. You inject a value once near the root, and any descendant reads it by type.

Alltag's container is `AppEnvironment`:

```
@MainActor
@Observable
final class AppEnvironment {
    let theme: ThemeController
    let language: LanguageStore
    let persistence: PersistenceController
    let llm: LLMService

    init(
        persistence: PersistenceController,
        llm: LLMService,
        theme: ThemeController = ThemeController(),
        language: LanguageStore = LanguageStore()
    ) {
        self.persistence = persistence
        self.llm = llm
        self.theme = theme
        self.language = language
    }
}
```

This single object owns the long-lived foundations: persistence, language, theme, and the on-device LLM facade. It is injected once at the app root and read anywhere:

```
@main
struct AlltagApp: App {
    @State private var env = AppEnvironment.live()

    var body: some Scene {
        WindowGroup {
            RootView()
                .environment(env)
        }
    }
}
```

```
struct RootView: View {
    @Environment(AppEnvironment.self) private var env
    // ...
}
```

`.environment(env)` puts the container into the tree at the root. `@Environment(AppEnvironment.self)` pulls it out, by type, in any descendant — `RootView`, `SettingsView`, screens nested arbitrarily deep. No prop-drilling through intermediate views, no singleton.

Compared to CDI — and where it diverges

If you squint, this is dependency injection: a container of collaborators, resolved by type, handed to consumers. But the deltas from Quarkus CDI matter, and getting them wrong is how backend engineers fight SwiftUI.

Aspect	Quarkus CDI	AppEnvironment
Graph built	by container, at boot	by hand, in <code>live()</code>
Resolved by	type + qualifiers	type (env key)
When	runtime, reflection	compile-time
Mechanism	proxies	plain references

Point by point:

- **No reflection, no proxies.** CDI scans the classpath, reads annotations, and synthesizes proxy beans. SwiftUI does none of this. `AppEnvironment` is a `final class` you `init` with plain stored properties. What you write is what runs — the object graph is literal code in a factory method, not a thing assembled by a container at startup.
- **Built by hand in a factory.** There is no `@Inject` doing the wiring. You construct the graph yourself in `live()` (next section). This is more typing than CDI but radically more legible: the entire dependency graph is one readable function, steppable in a debugger, with zero “where did this bean come from” mystery.
- **Compile-time.** A missing dependency is a compile error (the `init` won’t satisfy), not a runtime `UnsatisfiedResolutionException`. You cannot ship an app with an unfulfilled dependency.
- **Passed down the tree, not globally scoped.** A CDI `@ApplicationScoped` bean is reachable from anywhere in the process. The SwiftUI environment is *hierarchical*: a value is visible only to descendants of where it was injected. That scoping is structural, by view position, not by annotation.
- **Resolution is by type alone.** The environment keys on the concrete type. There is no qualifier mechanism like `@Named` / `@Qualifier`. To inject two of the same type, you give them distinct types, not labels.

Quarkus analogy. `@Environment(AppEnvironment.self)` is `@Inject AppEnvironment`. `.environment(env)` is the producer that put it in scope. But the scope is *the view subtree*, not the application, and the wiring is your handwritten `init`, not a proxy the container forged.

The factory: `live()` vs a CDI producer

CDI assembles graphs with `@Produces` methods. Alltag’s equivalent is the static `live()` factory — the one place the production object graph is constructed:

```
static func live() -> AppEnvironment {
    let llm = makeLLM()
    do {
        return AppEnvironment(
            persistence: try PersistenceController(), llm: llm)
    } catch {
        assertionFailure(
            "Persistent store unavailable, in-memory: \(error)")
        return AppEnvironment(
            persistence: try! PersistenceController(
                inMemory: true), llm: llm)
    }
}
```

This reads like a CDI producer, but everything is explicit and sequential. It constructs the real persistence stack, and — a detail a server engineer will respect — if the on-disk SwiftData store can't be opened, it logs and falls back to an in-memory store so the app still launches rather than crashing on first run. A producer method scattering this fallback across annotations would obscure it; here it is four lines you can read top to bottom. `theme` and `language` default in the `init`, so `live()` doesn't even name them.

Note also *who owns* the container. In `AlltagApp` it is held by `@State` :

```
@State private var env = AppEnvironment.live()
```

`@State` here does not mean “view-local UI flag” — it means the `App` struct *owns* this reference for the process lifetime and SwiftUI keeps it alive across re-evaluations of `body`. The container is constructed exactly once, at launch, and never rebuilt. That is the SwiftUI idiom for a root-owned singleton: not a `static` global, but a `@State`-owned value created at the top of the tree. Descendants borrow it via `@Environment`; the root owns its lifetime. Contrast CDI, where the container owns every bean's lifecycle and scope; here ownership is a property of *where in the tree the value is declared*.

The test container

The payoff of hand-built DI is testability without a test container framework. Alltag's `PersistenceController` takes an `inMemory: Bool`; an in-memory `AppEnvironment` is just a different construction:

```
let env = AppEnvironment(
    persistence: try PersistenceController(inMemory: true),
    llm: LLMService(/* stub engine */))
```

No `@QuarkusTest`, no CDI test bootstrap, no `@Mock` beans swapped by the container. You build the graph you want and pass it in. Because every collaborator is constructor-injected, a test substitutes a stub `LLMEngine` or an isolated `UserDefaults` suite by *handing it to the initializer*. The `LanguageStore` and `ThemeController` initializers already accept an injected `UserDefaults` precisely so tests use an isolated suite:

```
init(defaults: UserDefaults = .standard) {
    self.defaults = defaults
    // ...
}
```

Constructor injection with sensible defaults is the whole dependency-inversion story. The default keeps production call sites clean; the parameter keeps tests hermetic. No framework required.

Where business logic lives

If there is no ViewModel and the container only holds foundations, where does logic go? Into **use-cases and services**, constructed in the environment from those foundations.

The `AppEnvironment` doc comment states the intent directly: “As features land, their use-cases are constructed here from these foundations (persistence, language, theme).” A Decoder use-case is built from the LLM facade plus persistence; a deadline calculator from persistence plus the calendar. The view calls the use-case; the use-case owns the rules. This is exactly the service layer you would put behind a Quarkus resource — the resource (view) stays thin and delegates to a service that does the work.

The factory `makeLLM()` shows a service being assembled from parts: a `ModelStore`, a `ModelProvisioner`, an engine, and a runtime are composed into one `LLMService` facade. The view sees the facade; the wiring stays in the environment.

The foundations themselves are real services, not config bags. `PersistenceController` owns the SwiftData stack — its `ModelContainer`, its schema, its on-disk location:

```
@MainActor
final class PersistenceController {
    let container: ModelContainer
    static let schema = Schema([DocumentRecord.self])
    init(storeURL: URL? = nil,
        inMemory: Bool = false) throws { /* ... */ }
}
```

A use-case that needs to read documents takes the container (or a `ModelContext` from it) injected through the environment, the same way a Quarkus service takes an injected `EntityManager`. The architectural rule that matters: the view never constructs a `ModelContext` or talks to SwiftData directly. It goes through a Control-layer use-case, which the environment built from the `PersistenceController` foundation. Foundations at the bottom, use-cases in the middle, views at the edge — the dependency arrows point inward, exactly as in a hexagonal server app.

Boundary–Control–Entity

Alltag organizes code with BCE, per feature. You will see it in the folder layout — `Features/Settings/Boundary`, `Features/Decoder/...`, each feature a vertical slice:

- **Boundary** — the SwiftUI views, the user-facing edge. `SettingsView`, `VaultView`, `DecoderView` live here.
- **Control** — use-cases and services, the logic constructed from the foundations.

- **Entity** — domain models and persisted records (e.g. `DocumentRecord`).

A backend engineer can read this directly as a hexagonal / ports-and-adapters layering: Boundary is the inbound adapter (the UI port), Control is the application/service layer, Entity is the domain. The discipline is the same one you apply server-side — keep the framework-facing edge thin, keep rules in the middle, keep the domain pure. Organizing *per feature* rather than per layer means a feature’s view, logic, and models sit together, so a change to Decode rarely touches Settings.

Unidirectional data flow

Tie it together and you get a clean one-way loop, the same shape as a well-behaved request handler:

1. A view reads state from an `@Observable` model in the environment (`env.theme.theme`).
2. A user action calls a method on a model or use-case (`env.theme.select(.dark)`).
3. The model mutates its own state and persists it.
4. `@Observable` tracking marks every view that read the changed property dirty; SwiftUI re-renders just those.

State flows down (model → view); events flow up (view → method). Views never mutate model state directly — they call intent methods (`select(_)`), which is why those properties are `private(set)`. That single rule — read freely, mutate only through methods — is what keeps a SwiftUI app debuggable. When `SettingsView` calls `select`, the change ripples through the shared `env` to `RootView`, which re-applies the color scheme and locale app-wide. One mutation, one source of truth, a deterministic re-render. No event bus, no observers to register, no `@Inject Event<ThemeChanged>` to fire.

Note. `@Observable` requires `@MainActor` discipline for UI-driving state — note `AppEnvironment`, `ThemeController`, and `LanguageStore` are all `@MainActor`. State that a view reads must mutate on the main actor. Background work (LLM inference, disk I/O) runs off-main with `async/await`, then hops back to the main actor to publish results. This is stricter than CDI, where a bean’s threading is your problem to coordinate; here the compiler enforces the boundary.

Choosing a state property wrapper

A recurring beginner question — “which wrapper do I use?” — has a small, learnable answer once you separate *ownership* from *access*. Three wrappers cover nearly everything:

Wrapper	Meaning	Use for
<code>@State</code>	this view/App owns it	local UI, root container
<code>@Environment</code>	injected, read by type	shared services (<code>env</code>)
<code>@Bindable</code>	writable view onto a model	binding to <code>@Observable</code>

`@State` is ownership: the declaring view (or `App`) creates and keeps the value. Use it for a screen’s transient flags (`showingScanner`) and, at the root, for the long-lived container. `@Environment` is the

read side of DI: the value was created elsewhere and injected; you borrow it by type and never own its lifetime. `@Bindable` is purely about producing `Binding`s into an `@Observable` reference you already hold.

The mistake to avoid is using `@State` for something a parent should own, or duplicating an environment value into local `@State` (which forks the source of truth and goes stale). Map it to the server analogue: `@State` is a field you allocate and manage; `@Environment` is an injected collaborator you must not re-instantiate. When in doubt, ask “who is responsible for this value’s lifetime?” — that answers the wrapper.

Mutating shared state: @Bindable

One loose end from the screens chapter. When a control needs to *write* into an `@Observable` model, you need a `Binding`. For a model you hold directly, `@Bindable` synthesizes them:

```
struct ThemeRow: View {
    @Bindable var controller: ThemeController
    var body: some View {
        // $controller.theme is a Binding<AppTheme>
    }
}
```

`@Bindable` is the bridge between an `@Observable` reference type and the `$`-binding syntax that controls expect. Where the model exposes a settable property, `$model.prop` just works. Alltag’s `SettingsView` instead writes a custom `Binding(get:set:)` because `select(_:)` carries a guard and a persistence side effect that a raw property write would skip — a reminder that `@Bindable` is the convenience, and an explicit binding is the escape hatch when a write needs to do more than assign.

Takeaways

- Prefer **MV**: `@Observable` models drive views directly. Don’t cargo-cult a `ViewModel` per screen — add an observable object only when a screen has real presentation state.
- DI is the **environment**: inject an `@Observable` container with `.environment(...)`, read it with `@Environment(...)`. `AppEnvironment` is Alltag’s whole container.
- It resembles CDI but is hand-built in a factory, resolved by type, scoped to the view subtree, compile-time checked, with no reflection or proxies.
- `live()` is the production producer; an `inMemory` container and constructor-injected defaults give framework-free testability.
- Logic lives in use-cases/services constructed in the environment, organized by feature as Boundary–Control–Entity.
- Data flows one way: read state down, send events up via intent methods on `@MainActor` `@Observable` models. Use `@Bindable` for writable bindings, an explicit `Binding(get:set:)` when a write must route through guarded logic.

Next: Persistence: `SwiftData`, the `Keychain`, `Files`, and `CryptoKit`

Persistence: SwiftData, the Keychain, Files, and CryptoKit

On a backend, persistence is somebody else's process. You write `@Entity` classes, point `persistence.xml` at a connection pool, and Hibernate talks to a Postgres that runs on its own host with its own backups, its own disk encryption, its own auth. Your application is a stateless client of that durable thing.

On a phone there is no other process. Your app *is* the database host, the file server, the key-management service, and the backup policy — all inside one sandboxed directory that the OS can wipe, encrypt, or back up to iCloud depending on *which subfolder* you chose. Alltag pushes this to the limit: it is a paid app with **no backend at all**. Every byte — structured metadata, encrypted document blobs, the encryption key itself — lives on the device. This chapter maps the four pieces of that local stack onto the JVM tools you already know, then shows how Alltag wires them together.

Concern	Backend (Java)	Alltag (iOS)
Structured data	JPA + Postgres	SwiftData
Bulk bytes	S3 / filesystem	Files + CryptoKit
Secrets	Java <code>KeyStore</code>	the Keychain
Small prefs	properties file	<code>UserDefaults</code>

SwiftData: JPA without the server

`SwiftData` is Apple's modern ORM, layered over the old Core Data engine the way JPA is layered over JDBC. If you have used Panache, the ergonomics will feel familiar — declarative models, very little boilerplate — but the runtime is an embedded SQLite file, not a remote server.

A model is a class annotated with the `@Model` macro. This is the `@Entity` of the framework. Here is Alltag's `DocumentRecord` verbatim:


```

@Model
final class DocumentRecord {
    @Attribute(.unique) var id: UUID
    var fileName: String
    var category: String
    var createdAt: Date
    var expiresAt: Date?

    init(
        id: UUID = UUID(),
        fileName: String,
        category: String = "uncategorized",
        createdAt: Date = .now,
        expiresAt: Date? = nil
    ) { /* assign each property */ }
}

```

Three things differ sharply from JPA. First, `@Model` is a *macro*, not a runtime annotation: at compile time it rewrites the class, turning every stored property into an observed, persisted attribute and synthesizing the schema. There is no reflection-at-startup scan and no `persistence.xml` listing classes. Second, `@Attribute(.unique)` on `id` is the rough equivalent of a `@Column(unique = true)` plus upsert semantics — inserting a second record with the same `id` *updates* the existing row rather than throwing a constraint violation. Third, there is no separate `@Id` annotation forcing a primary key; SwiftData manages an internal row identity for you, and `id` here is just an application key that doubles as the address of the encrypted blob on disk.

Quarkus analogy. `@Model` is `@Entity`, but the class is `final` and value-typed in spirit — no lazy proxies, no `@ManyToOne` fetch joins firing N+1 queries behind your back. Relationships exist (see below) but the cost model is local SQLite, not a network round trip, so the failure modes you instrument for in Quarkus mostly evaporate.

The container, the configuration, the context

Alltag's `PersistenceController` owns the stack. Compare its members to the JPA bootstrap you know:

SwiftData	JPA
<code>Schema</code>	set of <code>@Entity</code> classes
<code>ModelConfiguration</code>	<code>persistence.xml</code> unit
<code>ModelContainer</code>	<code>EntityManagerFactory</code>
<code>ModelContext</code>	<code>EntityManager</code> / session

The `Schema` is built explicitly, listing the model types:

```

static let schema = Schema([
    DocumentRecord.self,
])

```

The `init` then assembles a `ModelConfiguration` (which store file, in memory or not) and from it a `ModelContainer`:

```
let configuration = ModelConfiguration(
    schema: Self.schema, url: url)
container = try ModelContainer(
    for: Self.schema, configurations: [configuration])
```

The `ModelContainer` is the heavyweight, thread-safe, app-lifetime object — exactly like an `EntityManagerFactory`. You build **one**. From it you draw `ModelContext` instances, which are the unit-of-work, short-lived, *not* thread-safe analogue of an `EntityManager`. The container hands you a `mainContext` bound to the main actor for UI work; background work creates its own `ModelContext(container)` off the `Sendable` container, just as you would open a fresh `EntityManager` per request rather than sharing one across threads.

Note the class is `@MainActor`:

```
@MainActor
final class PersistenceController {
    let container: ModelContainer
```

That annotation is the concurrency contract: the default context lives on the main thread, so accessing it from a background task is a compile error, not a runtime `IllegalStateException`. Strict concurrency turns the “never share an `EntityManager`” rule into a checked invariant.

CRUD and @Query

A context exposes `insert(_:)`, `delete(_:)`, and `save()`. The unit-of-work shape mirrors JPA’s persistence context: inserted objects are tracked, mutations are flushed on `save()`. SwiftData also auto-saves periodically, which has no clean JPA equivalent — treat explicit `save()` as you would `flush()` plus `commit()`.

```
let record = DocumentRecord(fileName: "Anmeldung")
context.insert(record)
try context.save()
```

Reading is where SwiftData diverges most pleasantly. Instead of writing JPQL or a Panache `list(...)`, views declare a `@Query` property that the framework keeps *live*: it re-runs and re-renders when the underlying data changes.

```
@Query(sort: \DocumentRecord.createdAt, order: .reverse)
private var records: [DocumentRecord]
```

There is no JPA analogue to a query that is also a reactive data source wired straight into the render loop. The closest mental model is a Panache `list()` whose result is automatically re-fetched and pushed to the client whenever a relevant row changes — except it is synchronous, in-process, and free. For imperative fetches outside a view you use a `FetchDescriptor` with a `#Predicate`, which is the type-safe, compile-checked cousin of a JPA `CriteriaQuery`.

Note. `#Predicate` closures are compiled into SQLite `WHERE` clauses, so only a subset of Swift is legal inside them — much like a JPA criteria expression cannot contain arbitrary Java. If you reach for something the translator rejects, you get a build-time or runtime error, not a silent full-table scan.

Migrations, briefly

Add a non-optional property without a default and the existing store no longer matches the schema. SwiftData performs **lightweight migrations** automatically for additive, inferable changes (new optional attribute, renamed-with-hint) — the equivalent of Hibernate’s `hbm2ddl.auto=update` but safer, because it refuses ambiguous changes rather than guessing. For anything structural you write a `SchemaMigrationPlan` with explicit `VersionedSchema` stages, the disciplined analogue of a Flyway script. Alltag is on its foundation slice (one entity), so it relies on lightweight inference for now; the model comments flag where richer fields land later.

The sandbox: which directory, and why it matters

A backend has no concept of this. Your process can write anywhere its user can; backup and cleanup are the ops team’s problem. An iOS app runs in a **sandbox** — a per-app container the OS owns — and the subfolder you choose silently decides three policies: *is it backed up to iCloud / iTunes, can the OS delete it under storage pressure, and does it survive app updates.*

Directory	Backed up	OS may purge	Use for
Documents	yes	no	user-visible files
App Support	yes	no	app-private data
Caches	no	yes	rebuildable data
tmp	no	yes	scratch

Getting this wrong is a real bug class with no server analogue: put your SQLite store in Caches and the OS may delete it mid-flight to reclaim space; put rebuildable thumbnails in Documents and you bloat the user’s iCloud backup. `FileManager` is the API — there is no `new File(path)` with an absolute path you control; you *ask* for a directory by its semantic role:

```
let appSupport = try FileManager.default.url(
    for: .applicationSupportDirectory,
    in: .userDomainMask,
    appropriateFor: nil, create: true)
```

Alltag places its **SwiftData store** in Application Support (`Application Support/Alltag/Alltag.store`), because it is app-private structured data that must outlive updates and is never shown to the user as files. `PersistenceController.defaultStoreURL()` builds exactly that path, creating intermediate directories on demand. The **document blobs**, by contrast, are user-owned content, so `EncryptedFileStore` puts them under a `Documents/Vault` subfolder:

```
let docs = try FileManager.default.url(
    for: .documentDirectory, in: .userDomainMask,
    appropriateFor: nil, create: true)
self.directory = docs.appendingPathComponent(
    "Vault", isDirectory: true)
```

Gotcha. “Backed up to iCloud” cuts against a privacy-first design: anything in Documents *can* leave the device in the user’s encrypted backup. Alltag’s answer is that the blobs are themselves encrypted with a key marked `ThisDeviceOnly` (next sections), so even a backup of the ciphertext is inert without the device’s Keychain.

File protection

Beyond the directory choice, iOS offers per-file at-rest encryption tied to the device passcode, with no JVM equivalent. `EncryptedFileStore` writes every blob with `FileProtectionType.complete` semantics via the write options:

```
try ciphertext.write(
    to: url, options: [.atomic, .completeFileProtection])
```

`.completeFileProtection` means the file’s bytes are inaccessible — even to the app — while the device is locked. `.atomic` writes to a temp file and renames, the same write-then-rename durability trick you would do by hand on a server. This is a *second* layer beneath Alltag’s own CryptoKit encryption: defence in depth, not redundancy.

CryptoKit: authenticated encryption without the footguns

If you have written `javax.crypto`, you remember the ceremony: pick a `Cipher` transformation string, manage an `IvParameterSpec`, choose a padding, wrap a `SecretKeySpec`, and pray you didn’t reuse a nonce or pick ECB. JCA is powerful and almost entirely misuse-prone — the API lets you do the wrong thing by default.

CryptoKit is the opposite: a small, modern, misuse-resistant API where the safe path is the only obvious path. Alltag’s `FileCryptor` is the whole encryption surface, and it fits in a screen:

```

struct FileCryptor {
    func encrypt(_ plaintext: Data,
                using key: SymmetricKey) throws -> Data {
        let sealed = try AES.GCM.seal(plaintext, using: key)
        guard let combined = sealed.combined else {
            throw FileCryptorError.sealingFailed
        }
        return combined
    }

    func decrypt(_ ciphertext: Data,
                using key: SymmetricKey) throws -> Data {
        let box = try AES.GCM.SealedBox(combined: ciphertext)
        return try AES.GCM.open(box, using: key)
    }
}

```

Several things are doing heavy lifting here. `AES.GCM.seal` uses **AES-256 in Galois/Counter Mode**, an *authenticated* cipher: the output carries an authentication tag, so decryption verifies integrity *and* authenticity, not just confidentiality. Tamper with one bit and `AES.GCM.open` throws rather than returning corrupted plaintext — there is no “decrypt to garbage” failure mode. In JCA you get this with `AES/GCM/NoPadding` plus a `GCMParameterSpec`, but you must wire it correctly; here it is the default and only mode.

You never see an IV. `seal` generates a fresh random nonce per call and packs it into `sealed.combined` as `nonce || ciphertext || tag`. That one `Data` blob is self-contained and exactly what Alltag writes to disk. The nonce-reuse catastrophe that haunts GCM in JCA — same key, same IV, total break — is structurally prevented because you cannot accidentally hold an IV constant.

The `FileCryptorTests` prove all three properties as executable spec:

```

func testTamperedCiphertextFailsAuthentication() throws {
    var ciphertext = try cryptor.encrypt(
        Data("secret".utf8), using: key)
    ciphertext[ciphertext.count - 1] ^= 0xFF
    XCTAssertThrowsError(
        try cryptor.decrypt(ciphertext, using: key))
}

```

Flip a tag bit, `open` throws — round-trip, wrong-key, and tamper cases are all covered. A `SymmetricKey(size: .bits256)` is just 32 bytes of secure-random material; the interesting question is where those bytes *live* between launches. That is the Keychain’s job.

The Keychain: the device’s KeyStore

A Java `KeyStore` is an encrypted file (JKS, PKCS12) holding keys and certs, unlocked by a passphrase your app supplies — and critically, *you* manage that file and that passphrase. The iOS **Keychain** is a system-wide secure credential store: a small encrypted database the OS owns, protected by a hardware-backed keybag whose master keys never leave the Secure Enclave. Your app does not hold the unlock secret; the OS gates access by the device passcode and per-item accessibility rules.

So the Keychain is less “a `KeyStore` file you open” and more “a managed secrets service running inside the OS, scoped to your app.” It is where secrets go *because* the device hardware protects it; the SwiftData store and `UserDefaults`, by contrast, are ordinary files with no such protection — putting a key there would be like committing your `keystore.jks` and its password to the repo.

Alltag’s `KeychainKeyStore` stores the AES key as a **generic-password** item. The API is the unreconstructed C-style `SecItem*` family — you build a `[String: Any]` query dictionary out of `kSec...` constants and hand it to a free function returning an `OSStatus`. It is JCA-grade verbose; Alltag wraps it behind the tiny `KeyProviding` protocol.

Reading uses `SecItemCopyMatching`:

```
var query = baseQuery()
query[kSecReturnData as String] = true
query[kSecMatchLimit as String] = kSecMatchLimitOne

var item: CTypeRef?
let status = SecItemCopyMatching(query as CFDictionary, &item)
switch status {
case errSecSuccess:    return item as? Data
case errSecItemNotFound: return nil
default:               throw KeyStoreError.keychain(status)
}
```

Writing uses `SecItemAdd`, and crucially sets the **accessibility class**:

```
attributes[kSecAttrAccessible as String] =
    kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly
```

`kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` packs three policy decisions. *AfterFirstUnlock*: the key is unavailable until the user unlocks once after boot, then stays available (so background work can decrypt) — the right tradeoff for at-rest data that the app touches while running. *ThisDeviceOnly*: the item is never migrated to a new device and never synced to iCloud Keychain, so the key cannot leave this hardware. This is the linchpin of the privacy story: even if the encrypted Documents blobs ride along in an iCloud backup, the key that opens them does not.

The whole get-or-create lifecycle is one method:

```
func symmetricKey() throws -> SymmetricKey {
    if let existing = try loadKeyData() {
        guard existing.count == 32 else {
            throw KeyStoreError.malformedKey
        }
        return SymmetricKey(data: existing)
    }
    let key = SymmetricKey(size: .bits256)
    let data = key.withUnsafeBytes { Data($0) }
    try store(data)
    return key
}
```

First call mints and persists a 256-bit key; every later call returns the same bytes. `reset()` wraps `SecItemDelete` and is the GDPR “delete all my data” hook — wiping the key cryptographically

shreds every blob at once, since the ciphertext is now unopenable. That is a property a backend rarely gets for free: revoking the key is the deletion.

Gotcha. Keychain items survive app *deletion* on some iOS versions and are awkward to exercise in unhosted simulator test runs. That is exactly why `KeyProviding` is a protocol: production uses `KeychainKeyStore`; tests inject `InMemoryKeyStore`, a deterministic key with no Keychain dependency at all.

UserDefaults: the properties file

For small, non-secret preferences — chosen language, chosen theme — the Keychain is overkill and SwiftData is too heavy. `UserDefaults` is the iOS equivalent of a properties file plus the Java `Preferences` API: a key-value plist the OS persists for you, ideal for scalars and short strings, wrong for anything large or sensitive (it is an unencrypted file in the sandbox).

Alltag's `LanguageStore` and `ThemeController` each persist exactly one string key. The pattern is identical:

```
func select(_ theme: AppTheme) {
    self.theme = theme
    defaults.set(theme.rawValue, forKey: Self.storageKey)
}
```

On init they read the string back and decode it into the enum, falling back to a default if absent or unrecognized:

```
if let raw = defaults.string(forKey: Self.storageKey),
    let stored = AppTheme(rawValue: raw) {
    theme = stored
} else {
    theme = .system
}
```

Note that `defaults` is injected (`init(defaults: UserDefaults = .standard)`) so tests pass an isolated suite — the same dependency- inversion move as the `KeyProviding` protocol, applied to the prefs store. Both classes are `@Observable`, so changing the value re-renders the views that read it; persistence and reactivity come together.

Tying it together: the Vault save path

Now the four pieces compose. Alltag's design splits each document into a *searchable metadata half* and an *encrypted bytes half*, deliberately kept apart:

- **Metadata** -> SwiftData `DocumentRecord` (id, name, category, dates). Small, queryable, drives the UI list.
- **Bytes** -> `EncryptedFileStore`, an encrypted blob on disk addressed by the record's `id` (`"\ (id).bin"`).
- **Key** -> `KeychainKeyStore`, the one 256-bit key all blobs share.

Keeping blobs out of SwiftData keeps the store small and lets the OS apply file-level protection to the heavy data — you would not store multi-megabyte scans as `byte[]` columns in your Postgres either; you'd put them in object storage and keep a row pointing at them. Same split, no server.

The save path runs top to bottom in `EncryptedFileStore.save` :

```
@discardableResult
func save(_ data: Data, id: String) throws -> URL {
    let key = try keyStore.symmetricKey()
    let ciphertext = try cryptor.encrypt(data, using: key)
    let url = fileURL(for: id)
    try ciphertext.write(
        to: url,
        options: [.atomic, .completeFileProtection])
    return url
}
```

Read it as a pipeline:

1. **Fetch the key** from the Keychain (`symmetricKey()`), minting it on first ever call.
2. **Encrypt** the plaintext with `FileCryptor` -> AES-GCM combined blob, integrity tag included.
3. **Write** atomically with `.completeFileProtection` , so plaintext never touches disk and the ciphertext is itself OS-encrypted at rest.
4. **Index** the metadata: the caller inserts a `DocumentRecord` whose `id` matches the blob's filename and `save()` s the context.

Load is the mirror image: read the `.bin` , fetch the same key, `open` the sealed box (which fails loudly if the file was tampered with). Delete a document and you remove both the SwiftData row and the blob; run `KeychainKeyStore.reset()` and you shred *everything* at once.

The whole thing is testable without a device because every boundary is inverted: `PersistenceController(inMemory: true)` gives an ephemeral SwiftData store, and `InMemoryKeyStore` gives a fixed key — so the encryption round-trip and the persistence logic run in milliseconds in CI, with no Keychain, no disk, and no server anywhere in sight.

Takeaways

- SwiftData is your JPA: `@Model` for `@Entity` , `ModelContainer` for `EntityManagerFactory` , `ModelContext` for the session, `@Query` for a *live* JPQL/Panache list — but the store is an embedded SQLite file you host yourself.
- The sandbox directory is a policy decision with no server analogue: Application Support for the app-private store, Documents for user content, never Caches/tmp for anything that must survive.
- CryptoKit's `AES.GCM.seal` / `open` is authenticated encryption with the footguns removed — random nonce, integrity tag, no IV to misuse — where JCA leaves all of that to you.
- The Keychain is the device's `KeyStore` , but OS-managed and hardware- backed; `ThisDeviceOnly` accessibility is what keeps the key from ever leaving the device, making encrypted backups inert.
- Alltag composes all four: metadata in SwiftData, ciphertext on disk, key in the Keychain — and every boundary is a protocol, so the full save path runs in-memory in tests.

Next: *The Toolchain: Xcode, SPM, XcodeGen, and the Build*

PART III

Ship It

The part nobody writes down: the toolchain, tests, signing, and the App Store machinery that turns a project into an app on a stranger's phone. This is where backend intuition needs the most rewiring.

The Toolchain: Xcode, SPM, XcodeGen, and the Build

You know how to stand up a JVM project blindfolded: a `pom.xml` or `build.gradle`, a `settings.xml` pointing at Nexus, JUnit on the test classpath, and a CI YAML that runs `mvn verify`. The tools are separable. IntelliJ is *an* editor; Maven is *the* build; the JDK is *a* runtime; you could swap any of them. Apple’s world is more fused. This chapter maps the iOS toolchain onto what you already know, walks the real configuration of the Alltag app, and shows where the analogy holds and where it breaks.

Xcode is four tools wearing one icon

In the JVM world your daily work is spread across several programs: IntelliJ (or VS Code), the `mvn / gradle` CLI, a profiler like JFR/async-profiler, and maybe a separate device/container to run on. Xcode collapses all of that into a single `.app`:

Xcode role	JVM-world equivalent
Editor + indexer	IntelliJ IDEA
Build system	Maven / Gradle (<code>xcodebuild</code>)
Simulator	a local device to run on
Instruments	JFR, async-profiler, VisualVM

The build engine has a headless CLI, `xcodebuild`, analogous to `mvn` — the IDE is a front end over it. So you *can* build without the GUI (CI does exactly that), but you cannot, in practice, build without **Xcode installed**: the compiler (`swiftc`), the SDKs, the linker, and the Simulator runtimes all ship inside the Xcode app bundle, selected by `xcode-select`. There is no standalone “Swift SDK for iOS” you download separately the way you grab a JDK. That coupling is the first thing that feels wrong coming from Java: the toolchain version *is* the IDE version.

Quarkus analogy. `xcodebuild` is `mvn / gradle`; the Xcode GUI is IntelliJ driving it. But unlike a JDK you can `apt install`, the iOS SDK and simulators live inside Xcode itself — one big versioned bundle, not separable parts.

Alltag is built and tested on **Xcode 26** with an **iOS 17.0** deployment target (see [ios/README.md](#)). Swift here is **5.x** with strict concurrency on, the iOS analogue of compiling with `-Werror` plus a thread-safety checker baked into the language.

The fourth tool, **Instruments**, is the profiler — Time Profiler, Allocations, Leaks, and dozens of other “instruments” you record a trace with, the way you would attach JFR or async-profiler to a running JVM. The difference is integration: you launch a profiling session straight from the same scheme’s **Profile** action (which, per the scheme, builds Release), so the optimized binary you measure is the one you ship. There is nothing to wire up separately.

The .xcodeproj: an opaque bundle and a merge-conflict magnet

A Maven project’s source of truth is a human-readable `pom.xml`. Xcode’s is the `.xcodeproj` — and it is the opposite of human-readable. It is a **bundle** (a directory macOS shows as one file) whose core is `project.pbxproj`: a giant, machine-managed property list full of synthetic hex identifiers like `67478AD1A8217FB2ACD51368`. Every file you add, every build setting you toggle in the GUI, rewrites this file.

Two consequences hurt teams:

- **Merge conflicts.** Two engineers each add a file on their branch and the `pbxproj` collides in ways git cannot sanely three-way-merge. There is no clean `<dependency>` block to reconcile — just re-shuffled hex IDs.
- **Opacity.** You cannot meaningfully code-review a diff of it. The intent (“added a test target”) is buried under generated noise.

The fix the community converged on is to stop hand-editing the `.xcodeproj` and instead **generate** it from a readable spec. That is what Alltag does.

Why Alltag uses XcodeGen

[XcodeGen](#) reads a YAML file, `project.yml`, and produces the `.xcodeproj` from scratch. The generated project is **git-ignored**; the YAML is the source of truth. The header of `ios/project.yml` says it outright:

```
# The .xcodeproj is GENERATED from this file and is .gitignored.
# Regenerate locally with:  cd ios && xcodegen generate
```

This is precisely the `pom.xml` model. The spec is small, diffable, reviewable, and conflict-free; the heavy generated artifact never enters version control. After cloning, or after adding files or flipping a setting, you run:

```
cd ios
xccodegen generate
open Alltag.xcodeproj
```

`xccodegen generate` is the moral equivalent of letting Maven synthesize an IDE project from the POM. Files are picked up by *directory*, not enumerated by hand: the spec points a target at a folder and every source under it is included. Add a `.swift` file, re-run `generate`, done — no `pbxproj` surgery, no conflict.

Gotcha. Because the `.xcodeproj` is generated and ignored, never edit project settings in Xcode’s GUI and expect them to stick — the next `xccodegen generate` overwrites them. The GUI is for browsing and running; `project.yml` is for changing. Treat it like a `target/` directory you happen to open in an IDE.

Reading the real project.yml

Open `ios/project.yml` and read it top to bottom; it is the whole build definition in ~70 lines.

Project-level options and settings

```
name: Alltag

options:
  bundleIdPrefix: de.everydaygermany
  developmentLanguage: de
  deploymentTarget:
    iOS: "17.0"
```

`deploymentTarget: iOS "17.0"` is the minimum OS the app supports — the analogue of `<maven.compiler.release>17</...>` but for the *platform*, not the language. Code may use APIs up to the SDK Xcode ships, but must still run on iOS 17. `developmentLanguage: de` makes **German** the base localization (a deliberate decision for this app; English and others layer on later).

```
settings:
  base:
    MARKETING_VERSION: "0.1.0"
    CURRENT_PROJECT_VERSION: "1"
    SWIFT_VERSION: "5.0"
    DEVELOPMENT_TEAM: ""
    CODE_SIGN_STYLE: Automatic
    CODE_SIGNING_ALLOWED: "NO"
    CODE_SIGNING_REQUIRED: "NO"
    SWIFT_STRICT_CONCURRENCY: complete
    ENABLE_USER_SCRIPT_SANDBOXING: "YES"
```

These are **build settings** — Xcode’s equivalent of compiler flags plus packaging metadata, applied to everything unless a target overrides them.

- `MARKETING_VERSION` / `CURRENT_PROJECT_VERSION` are the user-facing version (`0.1.0`) and the internal build number — the `<version>` and a monotonic build counter.
- `SWIFT_VERSION: "5.0"` selects the **language mode**, like `--release 17` . (Swift 6 introduces a stricter mode; Alltag stays in 5 mode while opting into the new concurrency checks.)
- `SWIFT_STRICT_CONCURRENCY: complete` is the headline: it makes the compiler enforce data-race safety at compile time, flagging any value shared across concurrency boundaries that is not provably `Sendable` . There is no JVM analogue — imagine `javac` rejecting code that lets a non-thread-safe object escape to another thread. Covered in depth in the concurrency chapter; here, note that it is *on at the project level* .
- The four `CODE_SIGN*` / `DEVELOPMENT_TEAM` settings **defer signing**: empty team, automatic style, signing not allowed and not required. This is what lets the app build and test on the Simulator and in CI with no Apple Developer account — the subject of Chapter 13.
- `ENABLE_USER_SCRIPT_SANDBOXING: "YES"` sandboxes any build-phase scripts, a hardening default.

Targets

A **target** is a build product plus the rules to produce it — closest to a Maven *module* or a Gradle *sub-project*, but finer: the app and its tests are separate targets in one project. Alltag declares two.

```
targets:
  Alltag:
    type: application
    platform: iOS
    sources:
      - path: Alltag
    settings:
      base:
        PRODUCT_BUNDLE_IDENTIFIER: de.everydaygermany.app
        GENERATE_INFOPLIST_FILE: "YES"
        TARGETED_DEVICE_FAMILY: "1"
        # ... INFOPLIST_KEY_* and asset names
```

The `Alltag` target is `type: application` — it produces `Alltag.app`. Its `PRODUCT_BUNDLE_IDENTIFIER` is `de.everydaygermany.app`, the unique reverse-DNS identity of the app on device and in the App Store (think Maven `groupId` + `artifactId`, but globally unique and Apple-registered). `TARGETED_DEVICE_FAMILY: "1"` means iPhone only.

```
AlltagTests:
  type: bundle.unit-test
  platform: iOS
  sources:
    - path: Tests/AlltagTests
  dependencies:
    - target: Alltag
  settings:
    base:
      PRODUCT_BUNDLE_IDENTIFIER: de.everydaygermany.app.tests
      GENERATE_INFOPLIST_FILE: "YES"
```

`AlltagTests` is a `bundle.unit-test` target that **depends on** `Alltag`. The dependency is what lets the tests reach into the app with `@testable import Alltag` — analogous to test sources on the same module having package-private access, except here the app and tests are distinct targets and `@testable` grants that access explicitly. Tests get their own bundle identifier, `de.everydaygermany.app.tests`.

Schemes

A **scheme** ties targets to *actions*: Build, Run, Test, Profile, Archive. There is no clean Maven analogue — it is closest to a set of named run configurations plus a binding of which targets each lifecycle phase touches.

```
schemes:
  Alltag:
    build:
      targets:
        Alltag: all
    test:
      targets:
        - AlltagTests
      gatherCoverageData: true
```

The `Alltag` scheme builds the app target and, on **Test**, runs `AlltagTests` with **coverage gathering on**. XcodeGen writes this into the generated scheme file (`Alltag.xcscheme`), where you can see the same intent expanded: `codeCoverageEnabled = "YES"`, the `Debug` configuration for tests, and `Release` for Profile and Archive. That last detail matters: **build configurations** (Debug vs. Release) are chosen *per action*. You test and debug in Debug (assertions on, optimizations off) and ship/profile in Release (optimized) — the same Debug/prod split you know, but selected by which scheme action you invoke rather than a Maven profile flag.

Swift Package Manager vs. Maven/Gradle

Swift Package Manager (SPM) is the dependency manager, declared in a `Package.swift` — and unlike `pom.xml`, the manifest is *Swift code*, executed to produce the package description (closer in spirit to a Gradle Kotlin DSL than to declarative XML). A minimal manifest declaring one dependency:

```
// swift-tools-version: 5.9
import PackageDescription

let package = Package(
    name: "Example",
    dependencies: [
        .package(
            url: "https://github.com/apple/swift-log",
            from: "1.5.0"),
    ],
    targets: [
        .target(name: "Example", dependencies: [
            .product(name: "Logging", package: "swift-log"),
        ]),
    ]
)
```

The biggest mental shift: **SPM is decentralized**. A dependency is a **git URL**, resolved directly against GitHub (or any git host), pinned by a `Package.resolved` lockfile (the analogue of a Gradle lockfile). There is no Maven Central, no Nexus, no `settings.xml` with mirror and credential configuration. Versions are git tags; resolution is semantic-version ranges over those tags. No central coordinate namespace means no `groupId:artifactId` — the URL is the coordinate. For an app, you usually add packages through Xcode (File ▸ Add Package Dependencies), and Xcode records them in the project; XcodeGen also supports a `packages:` section in `project.yml` so dependencies stay declared in the readable spec.

Alltag, being a paid, on-device app with deliberately few external dependencies, keeps this lean — but the model is the one above: point at a git URL, pin a version, build.

Note. Decentralized resolution means a dependency can vanish if its repo goes away, and there is no built-in private registry story as mature as Nexus. The tradeoff buys zero-infrastructure setup: no artifact server to run, no `settings.xml` to distribute.

The Simulator: a simulator, not an emulator

This distinction matters and is easy to get wrong. The Android **emulator** virtualizes ARM hardware; it is slow because it emulates a different CPU. Apple's **Simulator** does *not* emulate a phone's CPU. On a Mac it builds your app for the host architecture and runs it as a **native macOS process** against a simulated iOS runtime (UIKit, the frameworks, a fake device environment). The result is fast — launches in seconds, no virtualized hardware — but it is *not* a faithful device: no real Keychain entitlements sometimes, no Apple-silicon-only hardware features, performance characteristics that differ from a real iPhone. You develop against the Simulator and *verify* on a device.

You will see this asymmetry in the tests: `KeychainKeyStoreTests` deliberately `XCTSkip`s when the Keychain is unavailable in an unsigned Simulator run (`errSecMissingEntitlement`, -34018), because the Simulator does not grant the same entitlements a signed device build would.

xcodebuild, DerivedData, and Info.plist

The CLI

`xcodebuild` is the headless build, the `mvn` of this world. Its three verbs you will use most:

```
# compile the app for a simulator
xcodebuild build -project Alltag.xcodeproj -scheme Alltag \
  -destination 'platform=iOS Simulator,name=iPhone 17'

# build + run the test suite
xcodebuild test -project Alltag.xcodeproj -scheme Alltag \
  -destination 'platform=iOS Simulator,name=iPhone 17' \
  CODE_SIGNING_ALLOWED=NO

# produce a distributable archive (Release)
xcodebuild archive -project Alltag.xcodeproj -scheme Alltag \
  -archivePath build/Alltag.xcarchive
```

`-scheme` selects what to build and `-destination` selects where to run it (which simulator or device). Note you can override build settings on the command line — `CODE_SIGNING_ALLOWED=NO` — exactly as the README and CI do, the way you would pass `-D` to Maven.

DerivedData

Build outputs, the module cache, indexes, and intermediate artifacts land in **DerivedData** (by default under `~/Library/Developer/Xcode/DerivedData/`), not in your project tree. It is the equivalent of `target/` plus the Maven local repo cache plus the IDE index — and when Xcode behaves strangely,

deleting DerivedData is the canonical “clean build” reset (`mvn clean`’s heavier cousin). Because it lives outside the repo, it is implicitly “ignored” without a `.gitignore` entry. A frequent gotcha is that stale DerivedData masks problems — a renamed type still resolving, a deleted file still linking — so when a build is mysterious, the first move is `xcodebuild clean` or deleting the per-project folder under DerivedData, not a clean `git` checkout.

Info.plist and entitlements

Every iOS app ships an **Info.plist** — a property list of runtime metadata: display name, supported orientations, scene configuration, required device capabilities. It is the rough analogue of a **MANIFEST.MF** plus deployment descriptor. Classically you hand-maintained this XML file; Alltag instead has Xcode **synthesize** it. In `project.yml`:

```
GENERATE_INFOPLIST_FILE: "YES"
INFOPLIST_KEY_UILaunchScreen_Generation: "YES"
INFOPLIST_KEY_UIApplicationSceneManifest_Generation: "YES"
INFOPLIST_KEY_UISupportedInterfaceOrientations:
  "UIInterfaceOrientationPortrait"
ASSETCATALOG_COMPILER_APPICON_NAME: AppIcon
ASSETCATALOG_COMPILER_GLOBAL_ACCENT_COLOR_NAME: AccentColor
```

`GENERATE_INFOPLIST_FILE: "YES"` tells the build to *generate* the **Info.plist** rather than read a checked-in one, and each `INFOPLIST_KEY_*` setting injects one entry into it. So the launch screen, the scene manifest, and the portrait-only orientation are all declared as build settings in the readable spec — no XML to merge. The asset settings point the build at the app icon and accent color in the asset catalog.

Entitlements are a separate, signing-time concern: a list of privileged capabilities the OS grants the app (Keychain sharing, push, app groups). They live in an `.entitlements` file and are sealed into the app at signing. Alltag defers signing, so it carries no custom entitlements yet — Chapter 13 returns to this.

How to build and run Alltag, end to end

Two paths, both starting from the same regenerate step.

Interactive (the inner dev loop):

```
cd ios
xcodegen generate
open Alltag.xcodeproj
# pick an iPhone simulator in the toolbar, press Cmd-R
```

Headless (what CI does): the repo ships a local CI script, `ios/scripts/ci.sh`, that regenerates the project, picks the newest available iPhone simulator via `simctl`, and runs `xcodebuild test` with signing disabled. You can run the identical pipeline locally:

```
ios/scripts/ci.sh
```

That single command — regenerate, select simulator, build, test — is the whole story of building this app without ever touching the Xcode GUI. The next chapter takes that test step apart.

Takeaways

- **Xcode** fuses editor, build system (`xcodebuild`), Simulator, and profiler into one app; the SDK and simulators live *inside* it, so toolchain version is tied to IDE version.
- The `.xcodproj` is an opaque, conflict-prone generated-style bundle; Alltag treats it as a build artifact, **git-ignoring** it and generating it from `project.yml` via **XcodeGen** — its `pom.xml`.
- Read `project.yml` as the build definition: iOS 17 deployment target, `SWIFT_STRICT_CONCURRENCY: complete`, two targets (`Alltag` app + `AlltagTests`), one scheme with coverage, generated `Info.plist`, bundle id `de.everydaygermany.app`.
- **SPM** is decentralized: dependencies are git URLs pinned by a lockfile — no Maven Central, no `settings.xml`.
- The **Simulator** runs natively (fast, not an emulator) but is not a faithful device — verify on hardware.
- Build & run with `xcodegen generate` then Xcode, or run the whole pipeline headless with `ios/scripts/ci.sh`.

Next: *Testing: Swift Testing, XCTest, Previews, and CI*

Testing: Swift Testing, XCTest, Previews, and CI

You arrive with JUnit 5 muscle memory: `@Test`, `assertEquals`, `@ParameterizedTest`, `@BeforeEach`, Mockito for fakes, and a CI job that runs `mvn verify`. Swift has two test frameworks. The modern one, **Swift Testing**, will feel like JUnit 5 redesigned for Swift’s value types and async. The older one, **XCTest**, is the JUnit-4-era workhorse — and it is what **Alltag uses today**. This chapter leads with the new framework, maps both onto JUnit, converts a real Alltag test suite line by line, shows how the app injects fakes for tests, and ends by walking the project’s actual local CI script.

Swift Testing: the modern framework

Swift Testing ships with the toolchain. You import it, mark functions with a macro, and assert with `#expect`. The shape mirrors JUnit 5 closely.

```
import Testing

@Test func twoPlusTwo() {
    #expect(2 + 2 == 4)
}
```

`@Test` is `@Test`. `#expect(...)` is the universal assertion — but note the design: instead of a wall of `assertEquals`, `assertTrue`, `assertNull`, you write **one** macro around an ordinary boolean expression. Because `#expect` is a macro, it decomposes the expression at compile time and, on failure, prints the operands — `#expect(x == 4)` failing reports the actual value of `x`, no message needed. That subsumes most of JUnit’s assertion zoo.

JUnit 5	Swift Testing
<code>@Test</code>	<code>@Test</code>
<code>assertEquals(a, b)</code>	<code>#expect(a == b)</code>
<code>assertTrue(c)</code>	<code>#expect(c)</code>
<code>assertThrows(E, () -> ...)</code>	<code>#expect(throws: E.self)</code>
<code>assertNotNull(x)</code> then use	<code>try #require(x)</code>
<code>@ParameterizedTest</code>	<code>@Test(arguments:)</code>

#require: assert-and-unwrap

`#require` is `#expect`’s strict sibling: it throws and **stops the test** if its condition fails, and when given an optional it *unwraps* it. This is the idiom for preconditions — the test cannot sensibly continue if this is false:

```
@Test func decodesHeader() throws {
    let value = try #require(parse(input)) // unwrap or fail
    #expect(value.amount == 90)
}
```

`try #require(optional)` is JUnit’s `assertNotNull` followed by a non-null cast, collapsed into one step that also short-circuits.

Parameterized tests

JUnit’s `@ParameterizedTest` with `@ValueSource` / `@MethodSource` becomes `@Test(arguments:)`. The runtime expands one case per argument, each reported independently:

```
@Test(arguments: ["de", "en", "tr"])
func languageCodeIsTwoLetters(code: String) {
    #expect(code.count == 2)
}
```

You can zip two argument collections, or pass tuples, the way you would feed a `@MethodSource` a stream of argument sets.

Suites, traits, async, throwing

Tests are grouped into **suites**, which are plain Swift types — idiomatically a `struct`. This is a real departure from JUnit and from XCTest: because a `struct` is a value type, the runtime creates a **fresh instance per test**, so stored properties initialized in `init` are automatically isolated between tests. There is no shared mutable instance to leak state — `init` is your `@BeforeEach`, and `deinit` (on a `final class` suite) is `@AfterEach`.

```
struct CryptorTests {
    let cryptor = FileCryptor() // fresh per test
    let key = SymmetricKey(size: .bits256)

    @Test func roundTrips() throws {
        let blob = Data("hello".utf8)
        let out = try cryptor.encrypt(blob, using: key)
        #expect(try cryptor.decrypt(out, using: key) == blob)
    }
}
```

Traits are metadata/behaviour attached to a test or suite, the analogue of JUnit’s `@DisplayName`, `@Disabled`, `@Tag`, and `@Timeout` rolled into one mechanism:

```
@Test("wrong key is rejected", .disabled("flaky on CI"))
func wrongKeyFails() throws { /* ... */ }

@Suite(.tags(.crypto))
struct VaultCryptoTests { /* ... */ }
```

Tests are first-class **async/throws**. A test may be `async` and the runner awaits it; a test may `throw` and an uncaught error fails it — so the happy path of “this must not throw” needs no assertion at all,

just `throws` on the signature. That is strictly nicer than JUnit, where you either declare `throws` and let the exception fail the test or wrap in `assertDoesNotThrow`.

Quarkus analogy. Swift Testing is JUnit 5 reimaged for value types: `struct` suites give you per-test isolation for free (fresh instance each run), `#expect` collapses the assertion zoo into one macro that prints operands, and `async` / `throws` are native to a test the way `@Test` methods can `throws` in JUnit.

XCTest: the incumbent (what Alltag uses)

XCTest is the JUnit-4-shaped framework: subclass a base, name methods `test...`, assert with a family of free functions, override lifecycle hooks. Every Alltag test today is XCTest, so this is what you will read and write first in this codebase.

JUnit	XCTest
class <code>extends</code> (implicit)	<code>final class T: XCTestCase</code>
<code>@Test</code> (method <code>testFoo</code>)	<code>func testFoo()</code> (name prefix)
<code>assertEquals(a, b)</code>	<code>XCTAssertEqual(a, b)</code>
<code>assertTrue</code> / <code>assertFalse</code>	<code>XCTAssertTrue</code> / <code>...False</code>
<code>assertThrows</code>	<code>XCTAssertThrowsError</code>
<code>@BeforeEach</code> / <code>@AfterEach</code>	<code>setUp</code> / <code>tearDown</code>
<code>Assumptions.assumeTrue</code>	<code>XCTSkip</code>

Key mechanical differences from JUnit and from Swift Testing:

- **Tests are discovered by the `test` prefix**, not an annotation. A method named `testX` runs; re-name it and it silently stops running — a classic footgun.
- **The suite is a `class`** (must be `final class ... : XCTestCase`), and unlike Swift Testing's `struct` suites, the instance is reused with explicit `setUp` / `tearDown` resetting state — exactly JUnit 4's model.
- **Throwing tests** just add `throws`; an uncaught error fails the test. `XCTAssertThrowsError(try expr)` asserts the *opposite* — that the expression *does* throw.
- `setUpWithError` / `tearDownWithError` are the throwing variants of the lifecycle hooks.
- `XCTSkip` throws a skip (JUnit's assumption), used when an environment can't support a test — as we saw with the Keychain in the unsigned Simulator.

Worked conversion: FileCryptorTests, both ways

Alltag's `FileCryptor` encrypts a blob under a `SymmetricKey` with authenticated encryption. Its real test suite asserts four things: a round-trip recovers the plaintext, a wrong key fails, a tampered ciphertext fails authentication, and empty data round-trips. Here is the **real XCTest source**, verbatim from `ios/Tests/AlltagTests/Persistence/FileCryptorTests.swift`:

```

import CryptoKit
import XCTest
@testable import Alltag

/// A-09: blob encryption is a round-trip and authenticated.
final class FileCryptorTests: XCTestCase {
    private let cryptor = FileCryptor()
    private let key = SymmetricKey(size: .bits256)

    func testEncryptThenDecryptRecoversPlaintext() throws {
        let plaintext = Data(
            "Behörden-Brief: Bitte zahlen Sie 90 €.".utf8)
        let ciphertext = try cryptor.encrypt(
            plaintext, using: key)
        XCTAssertNotEqual(ciphertext, plaintext,
            "ciphertext must not equal plaintext")
        let recovered = try cryptor.decrypt(
            ciphertext, using: key)
        XCTAssertEqual(recovered, plaintext)
    }

    func testDecryptWithWrongKeyFails() throws {
        let ciphertext = try cryptor.encrypt(
            Data("secret".utf8), using: key)
        let otherKey = SymmetricKey(size: .bits256)
        XCTAssertThrowsError(
            try cryptor.decrypt(ciphertext, using: otherKey))
    }

    func testTamperedCiphertextFailsAuthentication()
        throws {
        var ciphertext = try cryptor.encrypt(
            Data("secret".utf8), using: key)
        ciphertext[ciphertext.count - 1] ^= 0xFF // flip tag
        XCTAssertThrowsError(
            try cryptor.decrypt(ciphertext, using: key))
    }

    func testEmptyDataRoundTrips() throws {
        let ciphertext = try cryptor.encrypt(
            Data(), using: key)
        XCTAssertEqual(
            try cryptor.decrypt(ciphertext, using: key),
            Data())
    }
}

```

Note the patterns: every test is `throws` (so a `try` that throws unexpectedly fails it), the happy-path assertions use `XCTAssertEqual` / `XCTAssertNotEqual`, and the two negative tests use `XCTAssertThrowsError` to assert that decryption *rejects* bad input. The `cryptor` and `key` are stored properties — but because XCTest reuses the class instance, they are shared; here that is safe because neither is mutated.

Now the **same suite rewritten in Swift Testing**:

```

import CryptoKit
import Testing
@testable import Alltag

/// A-09: blob encryption is a round-trip and authenticated.
struct FileCryptorTests {
    let cryptor = FileCryptor()
    let key = SymmetricKey(size: .bits256)

    @Test func encryptThenDecryptRecoversPlaintext()
        throws {
        let plaintext = Data(
            "Behörden-Brief: Bitte zahlen Sie 90 €.".utf8)
        let ciphertext = try cryptor.encrypt(
            plaintext, using: key)
        #expect(ciphertext != plaintext)
        #expect(
            try cryptor.decrypt(ciphertext, using: key)
            == plaintext)
    }

    @Test func decryptWithWrongKeyFails() throws {
        let ciphertext = try cryptor.encrypt(
            Data("secret".utf8), using: key)
        let otherKey = SymmetricKey(size: .bits256)
        #expect(throws: (any Error).self) {
            try cryptor.decrypt(ciphertext, using: otherKey)
        }
    }

    @Test func tamperedCiphertextFailsAuthentication()
        throws {
        var ciphertext = try cryptor.encrypt(
            Data("secret".utf8), using: key)
        ciphertext[ciphertext.count - 1] ^= 0xFF
        #expect(throws: (any Error).self) {
            try cryptor.decrypt(ciphertext, using: key)
        }
    }

    @Test func emptyDataRoundTrips() throws {
        let ciphertext = try cryptor.encrypt(
            Data(), using: key)
        #expect(
            try cryptor.decrypt(ciphertext, using: key)
            == Data())
    }
}

```

What changed, point by point:

- `import XCTest` → `import Testing`.
- `final class ... : XCTestCase` → `struct` (a value type; fresh per test).
- `func testX` → `@Test func x` — discovery by attribute, not by name prefix, so the `test` prefix is dropped.
- `XCTAssertEqual(a, b)` / `XCTAssertNotEqual` → `#expect(a == b)` / `#expect(a != b)`.

- `XCTAssertThrowsError(try expr) → #expect(throws: ...) { try expr }`. (You can name a concrete error type instead of `(any Error).self` to assert *which* error is thrown — stricter than the XCTest form.)
- The explanatory failure messages mostly vanish, because `#expect` prints the operands itself.

Both express the identical contract; the Swift Testing version is shorter and its `struct` isolation removes any worry about shared state across the four tests.

Injecting fakes for tests

Alltag avoids a mocking framework (no Mockito equivalent in play) by designing for **constructor injection** and providing hand-written in-memory fakes — protocol-conforming test doubles. Three patterns recur across the suite.

1. In-memory key store. `EncryptedFileStore` takes a `keyStore` in its initializer; tests pass `InMemoryKeyStore()` instead of the real Keychain-backed one, so persistence tests run without entitlements:

```
private let keyStore = InMemoryKeyStore()
// ...
store = try EncryptedFileStore(
    directory: directory, keyStore: keyStore)
```

The same suite uses a fresh `InMemoryKeyStore()` to *simulate a different device* — proving a foreign key cannot decrypt another store’s blob:

```
let foreign = try EncryptedFileStore(
    directory: directory, keyStore: InMemoryKeyStore())
XCTAssertThrowsError(try foreign.load(id: id))
```

2. In-memory persistence. The SwiftData stack accepts an `inMemory` flag — an ephemeral store with no disk, the analogue of an H2 in-memory database for a fast test. `AppEnvironmentTests` builds the whole DI container against it:

```
let env = AppEnvironment(
    persistence: try PersistenceController(inMemory: true),
    llm: try LLMTTestFactory.service())
XCTAssertEqual(env.language.language, .de)
```

3. Isolated `UserDefaults` suite. `LanguageStore` reads/writes `UserDefaults`; injecting them lets each test run against a **throwaway suite** keyed by a UUID, created in `setUp` and wiped in `tearDown` — so no test pollutes global app preferences:


```

override func setUp() {
    super.setUp()
    suiteName = "alltag.tests.\(UUID().uuidString)"
    defaults = UserDefaults(suiteName: suiteName)
}

override func tearDown() {
    defaults.removePersistentDomain(forName: suiteName)
    super.tearDown()
}

```

Note. The lesson for a Java engineer: Swift apps lean on protocols + constructor injection rather than runtime mocking. You design the seam (a `keyStore` parameter, an `inMemory` flag, an injected `UserDefaults`) and hand-write the fake. It is more code than `@Mock`, but the doubles are explicit and type-checked.

Note that `LanguageStoreTests` and `AppEnvironmentTests` are annotated `@MainActor`: they touch UI-bound, main-actor-isolated state, so the whole test class is pinned to the main actor — a concurrency constraint with no JUnit analogue.

UI tests with XCUITest

Beyond unit tests, `XCUITest` drives the app as a black box — launching it, finding elements by accessibility identifier, tapping and asserting on screen — the rough analogue of Selenium/Playwright for a web app. It runs in a separate UI-test target and is far slower than a unit test. Alltag's foundation phase carries unit tests and a smoke test rather than a UI suite; the model, when added, is `app.buttons["save"].tap()` then assert a label exists.

Previews: the fast inner loop

SwiftUI **previews** are the closest thing to hot reload. A `#Preview` block renders a view live in Xcode's canvas without building and launching the whole app on a simulator — you edit the view body and the canvas updates in place. Alltag's `RootView` declares one:

```

#Preview {
    RootView()
        .environment(AppEnvironment.live())
}

```

This is your tightest feedback loop for UI work — seconds, not a full rebuild-and-launch cycle. It complements, rather than replaces, tests: previews check *appearance* by eye; tests check *behaviour* deterministically. (A handy trick: you can pass a preview an `inMemory` environment with seeded fakes, exactly as the tests do, to render specific states.)

Code coverage

Coverage is configured **on the scheme**, not via a separate plugin like JaCoCo. Recall from Chapter 11 that `project.yml`'s scheme sets `gatherCoverageData: true`; XcodeGen writes that into the generated scheme as `codeCoverageEnabled = "YES"` under the `TestAction`. When `xcodebuild test` runs, the toolchain emits coverage profiles into the result bundle, viewable in Xcode's report navigator or extractable on the CLI with `xcrun xccov`. No extra dependency, no separate report goal — it is a property of the test action.

CI: the real `ios/scripts/ci.sh`

Alltag's CI is **local and on-demand** — there is no cloud runner. The script `ios/scripts/ci.sh` mirrors exactly what a CI runner would do, in four steps:

1. **Print the toolchain** (`xcodebuild -version`, `swift --version`) for a reproducible log.
2. **Regenerate the project** with `xcodegen generate` (failing loudly if `xcodegen` isn't installed) — because the `.xcodproj` is git-ignored, CI *must* generate it first.
3. **Pick a simulator dynamically**. Rather than hard-code a device name that varies by machine and Xcode version, it queries `simctl` for available iPhone simulators and takes the newest:

```
UDID=$(xcrun simctl list devices available --json \
| python3 -c "import json,sys; \
ds=json.load(sys.stdin)['devices']; \
cands=[d for rt in ds for d in ds[rt] \
if d['isAvailable'] and 'iPhone' in d['name']]; \
print(cands[-1]['udid'] if cands else '')")
```

4. **Build and test**, with signing disabled:

```
xcodebuild test \
-project "$PROJECT" \
-scheme "$SCHEME" \
-destination "platform=iOS Simulator,id=$UDID" \
CODE_SIGNING_ALLOWED=NO
```

(If `xcbeautify` is installed it pipes through it for readable output; otherwise it runs raw.) `set -euo pipefail` at the top makes any failed step abort the run with a non-zero exit — the pass/fail signal a CI gate needs.

Why local CI here

The driving reason is **signing**. As Chapter 11 covered, Alltag defers code signing entirely (`CODE_SIGNING_ALLOWED=NO`), which is only possible because it builds and tests on the **Simulator**, not a device. A Simulator build needs **no Apple Developer account, no certificate, no provisioning profile**. That removes the single biggest reason teams need cloud CI infrastructure early — there are no secrets to inject. So the full pipeline (regenerate → pick simulator → build → test) runs identically on any developer's Mac with a single command, with nothing to configure.

What cloud CI would add later

A hosted runner — **GitHub Actions** on a macOS image, or Apple’s **Xcode Cloud** — would layer on top of the same `xcodebuild test` core and add: running on every push/PR as a merge gate; a clean, reproducible build environment; archiving and **signed** builds for TestFlight/App Store distribution (which *do* need a Developer account, certificates, and provisioning — Chapter 13); and historical coverage/test-result dashboards. None of that is needed to validate the on-device, paid, no-backend app during early development — which is exactly why Alltag ships a script, not a pipeline.

Takeaways

- **Swift Testing** is JUnit-5-shaped and modern: `@Test`, `#expect` (one macro, prints operands), `try #require` (unwrap + assert), `@Test(arguments:)` for parameterization, `struct` suites with per-test isolation, native `async` / `throws`, and traits for `@Disabled` / `@Tag` / `@Timeout`.
- **XCTest** is the JUnit-4-shaped incumbent and **what Alltag uses today**: `final class ... : XCTestCase`, `test`-prefixed methods, `XCTAssert*`, `setUp` / `tearDown`, `XCTAssertThrowsError`, `XCTSkip`.
- The real `FileCryptorTests` converts cleanly: `XCTAssertEqual` → `#expect(==)`, `XCTAssertThrowsError` → `#expect(throws:)`, `class` → `struct`, name-prefix → `@Test`.
- Test seams are **protocols + constructor injection**, not mocking: `InMemoryKeyStore`, `PersistenceController(inMemory: true)`, and per-test `UserDefaults` suites.
- **Previews** (`#Preview`) are the hot-reload-style inner loop; **coverage** is a scheme setting (`gatherCoverageData`), not a plugin.
- CI is **local** (`ios/scripts/ci.sh`) because deferred signing + Simulator means no Apple account or secrets are required; cloud CI adds merge gates and signed distribution later.

Next: *Code Signing and Provisioning, Demystified*

Code Signing and Provisioning, Demystified

You ship a Quarkus service by building a jar (or a container image) and pushing it somewhere a runtime can fetch it: `scp` to a host, `docker push` to a registry, `helm upgrade` on a cluster. The artifact is trusted because *you* control the destination. Nobody asks the jar to prove who built it. The host runs whatever bytes you hand it.

iOS inverts that. An iPhone runs almost no code that Apple’s trust chain has not vouched for. There is no `scp`-the-binary path to a real device, no “just run the artifact.” Every app that launches on a physical iPhone — yours, in development, or a paying customer’s — carries a cryptographic chain proving (a) a known Apple Developer account built it, (b) for a specific app identity, (c) with a specific set of permissions, (d) onto an authorized device or the App Store. **Code signing** is how that chain is expressed.

This is, by wide agreement, the single most confusing part of iOS. The good news for you: the primitives are ones you already use every day — keypairs, X.509 certificates, a certificate authority, and a signed manifest. The confusion is almost entirely vocabulary and the fact that Apple’s tooling hides the wiring. Once you name the pieces, it collapses into something you already understand from TLS and code-signing certs.

Why the chain of trust exists

Apple’s threat model is consumer-facing: a phone full of banking apps, photos, and health data, owned by someone who is not a system administrator. The platform guarantees that arbitrary, unvetted code cannot run. The kernel refuses to execute a binary whose signature it cannot validate against a trusted anchor — Apple’s own root.

Compare your world. Your JVM happily loads any class on the classpath. Your container runtime runs any image you pull. The trust boundary lives at the *deploy* step, enforced by your registry creds and your cluster RBAC, and you, the operator, are trusted. On iOS the trust boundary lives at *every app launch*, enforced by the OS, and the developer is *not* implicitly trusted — they must present a chain that terminates at Apple.

Quarkus analogy. Think of a corporate environment where the JVM refuses to load any jar that is not signed by a cert chaining to the company CA, and where the CA only issues certs to enrolled developers, scoped to one application’s coordinates. iOS is that, with Apple as the mandatory CA and the App Store as the only public distribution channel.

There are exactly two destinations that require a full chain: **physical devices** and the **App Store / TestFlight**. The *Simulator* is the exception — it runs your code as a normal macOS process under your own user account, so it needs no signature at all. That single fact is why Alltag can build and test today with no Apple account, and we will return to it.

The pieces, mapped to things you know

There are six moving parts. Learn the mapping once and the Xcode dialogs stop being mysterious.

iOS concept	What you already call it
Developer account/Team	CA enrollment + org identity
Certificate	Code-signing cert (keypair)
App ID	Application coordinates
Capabilities	Requested grants / scopes
Provisioning profile	Signed manifest binding them
The signature	<code>jarsigner</code> / <code>codesign</code> output

Apple Developer account and Team

Enrolling in the Apple Developer Program (€99/year) registers you as a party Apple's CA will issue certificates to. Enrollment yields a **Team ID** — a short opaque string like `A1B2C3D4E5` — that identifies the legal entity (an individual or an organization) behind every artifact you sign. It is the equivalent of being onboarded to your company CA so it will mint certs in your name. In `project.yml` the Team is named exactly once, by ID:

```
settings:
  base:
    DEVELOPMENT_TEAM: "" # fill in when an account is ready
```

It is empty today because Alltag has no account attached yet.

Certificates: a keypair Apple signed

An iOS signing **certificate** is precisely what you think a certificate is: an X.509 cert wrapping a public key, issued (signed) by an Apple intermediate CA, with the matching **private key** held in your **login Keychain** on the Mac. This is the same shape as a code-signing cert or a TLS client cert. The private key never leaves your machine; losing it means you can no longer produce signatures that validate, exactly as losing a TLS private key would.

There are two flavors, differing only in what destinations they authorize:

- **Development** (`Apple Development`): signs builds for devices you control, for debugging.
- **Distribution** (`Apple Distribution`): signs builds destined for the App Store or TestFlight.

Inspect what you hold today the same way you would list identities in a keystore:

```
security find-identity -p codesigning -v
```

This prints the certificates in your Keychain whose private key is present — the candidates Xcode can actually sign with. An empty list on a CI box is normal and expected.

App IDs: the application's coordinates

An **App ID** registers a bundle identifier — Alltag's is `de.everydaygermany.app` — in Apple's portal, tying that string to your Team and to a set of allowed capabilities. It is the reverse-DNS equivalent of a Maven `groupId:artifactId`: a globally unique name for *this* application. Apple uses it as the key under which it tracks which capabilities the app may use. The value lives in the target:

```
PRODUCT_BUNDLE_IDENTIFIER: de.everydaygermany.app
```

Note. A *free* personal team (a plain Apple ID, no €99 program) can also sign for devices, but it cannot reuse a bundle ID another team has already claimed. If `de.everydaygermany.app` is taken on Apple's side, append a suffix such as `de.everydaygermany.app.dev` for local device builds. This is purely an Apple-namespaces collision, not a code problem.

Capabilities and entitlements: what the app may do

Capabilities are the privileged features an app requests — push notifications, iCloud, App Groups, Keychain sharing, Sign in with Apple, HealthKit, and so on. Each enabled capability is recorded as an **entitlement**: a signed key/value baked into the app and cross-checked at runtime by the OS. Think OAuth scopes, but cryptographically welded to the binary: an app cannot grant itself a capability it was not provisioned for, because the entitlement is covered by the signature and the OS enforces it.

Here is the most reassuring sentence in this chapter for Alltag: **it needs no capabilities beyond the defaults**. Alltag is a paid, fully on-device app — no backend, no push, no iCloud sync, no App Groups, no Keychain sharing across apps. Its document vault encrypts files locally with a key in the device Keychain, which is ordinary app-sandbox behavior requiring no special entitlement. So the entitlements set is empty, the App ID needs no capabilities toggled, and a whole category of signing pain — mismatched entitlements between profile and app — simply cannot occur here. Fewer scopes, fewer failure modes.

Provisioning profiles: the binding document

A **provisioning profile** is the piece that newcomers cannot place, because nothing in plain backend deployment corresponds to it. It is a signed property list that *binds together*:

- one or more **certificates** (who may sign),
- one **App ID** (which app),
- a list of **device UDIDs** (for development/ad-hoc; the App Store variant lists none),
- the **entitlements** (what the app may do).

Apple signs this document. At build time, Xcode embeds it into the `.app` bundle as `embedded.mobileprovision`. At install/launch time the OS reads it and verifies: is this app's signature made by a cert the profile authorizes? Does the bundle ID match? Is *this* device in the list (or is this a Store build)? Do the requested entitlements match what the profile grants? All four must hold or the app will not launch.

Gotcha. Almost every “it builds but won’t install on my phone” error is a profile mismatch: a cert not listed, a device UDID absent, or an entitlement the profile does not grant. The build compiles fine — signing is a *post-compile* packaging step — so the failure shows up late, at install or upload, which is why it feels mysterious. Read the error as “the binding document does not cover this combination,” then ask which of the four parts is off.

The mental model: the **certificate** proves *who*, the **App ID** names *what*, the **device list** scopes *where*, the **entitlements** declare *which powers*, and the **profile** is the notarized contract tying all four together. The signature on the binary is the wax seal.

Automatic vs manual signing

You can manage those pieces two ways.

Manual signing means you create certificates, register devices, generate profiles in Apple’s portal, download them, and point each build configuration at a specific profile by name. Total control, and the right choice for locked-down CI where you inject credentials deliberately. It is also the most error-prone by hand.

Automatic signing (“Automatically manage signing” in Xcode) means you give Xcode your Team and it talks to Apple’s portal for you: creates a development cert if you lack one, registers the attached device, generates and refreshes the matching profile, and selects it. For a solo developer it removes nearly all the manual portal work. It is what Alltag selects:

```
CODE_SIGN_STYLE: Automatic
```

On the command line the flag that lets `xcodebuild` perform those portal mutations (registering a device, minting a profile) is `-allowProvisioningUpdates`, which the README already uses for device builds. Without it, automatic signing can read existing assets but will not create new ones — useful when you want CI to fail loudly rather than silently touch your Apple account.

Why Alltag defers signing — and how to flip it on

Alltag ships today with signing *deferred*. The header comment in `project.yml` states the decision plainly:

```
# Signing is deferred (D-decision): the project builds for the
# Simulator and CI without a signing identity. To run on a physical
# device or upload to App Store Connect, fill in DEVELOPMENT_TEAM and
# flip CODE_SIGNING_ALLOWED to YES below.
```

The mechanism is two build settings:

```
CODE_SIGNING_ALLOWED: "NO"
CODE_SIGNING_REQUIRED: "NO"
```

These tell the build system to skip the signing step entirely. This is correct and deliberate, because the only targets Alltag currently runs are the **Simulator** (local development) and **CI** (the `ios/scripts/ci.sh` pipeline). Neither needs a signature: the Simulator runs the app as a normal process under your macOS user, and CI only builds and tests against the Simulator. Demanding a signing identity there would force every contributor and every CI run to carry an Apple account and certificates to do work that does not require them. Deferral keeps the barrier to entry at zero until there is an actual device or Store to target. This is the same instinct as not provisioning production TLS certs for a service you are only running against `localhost` in tests.

To target a real device or App Store Connect, flip it on. The steps are mechanical:

1. **Set the Team.** Put your Team ID in `DEVELOPMENT_TEAM`. Find it via Xcode ▸ Settings ▸ Accounts ▸ your Apple ID ▸ *Manage Certificates*, or from `security find-identity -p codesigning -v`.

```
DEVELOPMENT_TEAM: "A1B2C3D4E5"
```

2. **Enable signing.** Change both flags to `YES`:

```
CODE_SIGNING_ALLOWED: "YES"
CODE_SIGNING_REQUIRED: "YES"
```

3. **Regenerate the project**, since the `.xcodeproj` is generated and git-ignored:

```
cd ios
xcodegen generate
```

4. **Register the device.** Connect the iPhone by USB, unlock it, tap **Trust**, and on iOS 16+ enable **Settings ▸ Privacy & Security ▸ Developer Mode**. With automatic signing, building from Xcode (⌘R) or `xcodebuild ... -allowProvisioningUpdates build` registers the device's UDID and mints the profile for you.

For someone with no paid account, **free provisioning** is the shortcut: sign in to Xcode with a personal Apple ID and automatic signing issues a **7-day** development profile. The app runs on your own device for a week, then must be re-signed — fine for trying the real thing, insufficient for distribution. A paid account is required the moment you want TestFlight, the App Store, or profiles that do not expire weekly.

Common signing errors and how to read them

The error text is intimidating; the cause is almost always one of the four profile parts being off. A field guide:

Error	Cause and fix
No profiles match	No profile binds this App ID + cert. Enable automatic + updates or generate one in the portal.
Code signing identity not found	Cert/private key absent from the Keychain. Re-create or re-import.
Certificate has expired	Issue a new one; old builds keep running, new signatures fail.
Device not registered	UDID not in the profile. Build with <code>-allowProvisioningUpdates</code> .
Provisioning profile doesn't include entitlement X	The App ID's capabilities and the app's entitlements diverge.

Two extra notes that resolve a surprising share of cases. First, **wrong team**: if `DEVELOPMENT_TEAM` names a team you are not a member of, or an old one, every profile lookup fails confusingly — check that the ID matches an account in Xcode ▸ Settings ▸ Accounts. Second, the calm default: **let automatic signing fix it**. Toggle “Automatically manage signing” on, ensure the right Team is selected, build once with `-allowProvisioningUpdates`, and Xcode will regenerate whatever is missing. Manual surgery on certs and profiles is a tool for when you specifically need determinism — in CI, or when debugging — not the first move.

Takeaways

- iOS refuses to run unsigned code on real devices and the Store; the Simulator is the exception, which is why Alltag builds today with no Apple account.
- The pieces map onto PKI you know: a **certificate** is an Apple- signed keypair (private key in your login Keychain); an **App ID** is the bundle ID `de.everydaygermany.app`; a **provisioning profile** is a signed manifest binding cert + App ID + devices + entitlements into the app.
- Alltag needs **no capabilities** — it is paid and fully on-device — so an entire class of entitlement mismatch errors cannot occur.
- Deferred signing (`CODE_SIGNING_ALLOWED: "NO"`, empty `DEVELOPMENT_TEAM`, `CODE_SIGN_STYLE: Automatic`) is deliberate; flip it by setting the Team, switching both signing flags to `YES`, and `xcodegen generate`.
- Free provisioning (personal Apple ID, 7-day profile) runs the app on your own device with no paid account; a paid account is required for TestFlight and the Store.
- Almost every signing error is one of four profile parts being off (cert, App ID, device, entitlement); automatic signing with `-allowProvisioningUpdates` regenerates most of them.

Next: *Distribution: App Store Connect, TestFlight, and Review*

Distribution: App Store Connect, TestFlight, and Review

With signing demystified, you can produce a trusted artifact. Now you need a place to put it and a process to release it. In your world that is a registry plus a deploy pipeline: `docker push`, then a rollout that a dashboard watches, with stages you can pause and roll back. Apple's equivalent has the same shape — a console, an artifact upload, a staged rollout, and a release gate — but with one component you have no analogue for: a **human reviewer** stands between your upload and your customers. This chapter walks the path from archive to live for Alltag, a paid, on-device app, and names every gate along the way.

App Store Connect: the console

App Store Connect (ASC) is the web console at `appstoreconnect.apple.com`. Treat it as your deploy dashboard plus product catalog plus billing back office, all behind your Apple account. It is where the app's *record* lives — its metadata, pricing, privacy declarations, uploaded builds, TestFlight groups, and release state. Crucially, ASC holds everything *except* the signed binary, which you upload to it from Xcode or the CLI.

Create the app record

Before any build can be associated with the app, the record must exist. You create it once, in ASC ▶ **My Apps** ▶ + ▶ **New App**, supplying:

- **Platform:** iOS.
- **Name:** the public App Store name (Alltag).
- **Primary language:** **German** — matching the project's `developmentLanguage: de` and the app's audience.
- **Bundle ID:** pick `de.everydaygermany.app` from the dropdown. It appears there only after the App ID is registered in the Developer portal (chapter 13), which is why that step precedes this one.
- **SKU:** an internal identifier of your choosing; ASC never shows it to users. It is a private key for your own bookkeeping.

This record is the durable thing. Builds come and go against it; metadata and pricing attach to it.

Metadata

Each App Store *version* carries marketing metadata, edited in ASC and submitted alongside the build. For Alltag's German listing you supply:

- **Description:** the long-form pitch.
- **Keywords:** a comma-separated list feeding search.

- **Screenshots:** required, per device size class. ASC will not let you submit without them. iPhone-only is fine here — `TARGETED_DEVICE_FAMILY: "1"` means Alltag is an iPhone app, so you provide iPhone screenshots only.
- **Support URL:** a reachable page where users can get help. Required.
- **Promotional text, what's new:** optional and per-release.

Metadata is part of what review checks, so it must be truthful and match the app's actual behavior.

Pricing: Alltag is a paid app

In ASC › **Pricing and Availability** you choose a **price tier**. Apple sells through a fixed ladder of price points per region; you pick a tier (e.g. the €4.99 point) and Apple maps it to a local price in every territory you enable. This is the entire monetization model for Alltag: **the app is paid once, up front, and everything unlocks on purchase.**

That has a clean engineering consequence worth stating plainly: **there is no in-app purchase code.** No StoreKit transactions, no receipt validation, no entitlement-unlock logic, no subscription renewal handling. The purchase happens entirely in Apple's infrastructure at download time; your binary ships fully functional to anyone who has paid. A paid-up-front app is the simplest possible commerce integration — you write zero commerce code — which is exactly why it was chosen for an on-device app with no backend.

Quarkus analogy. A paid-up-front app is like selling a perpetual license that gates the *download* rather than runtime features. There is no license-check service to run, no entitlement call on startup — possession of the artifact is the license, enforced by the store, not by your code.

App Privacy and age rating

ASC requires two declarations before submission.

App Privacy — Apple's "nutrition labels" — is a questionnaire about what data your app collects and how it is used, surfaced on the product page. This is Alltag's easiest path by a wide margin: it collects **no** data. Everything is on-device, there is no backend, no analytics SDK, no tracking, no account. You answer "**Data Not Collected**" and you are done. No data types to enumerate, no linkage or tracking disclosures.

Gotcha. The privacy answers are a *binding claim*, not marketing. If you later add even a crash-reporting SDK that phones home, the declaration must change and a mismatch is a guideline violation and a rejection reason. Keep "Data Not Collected" honest: any third-party SDK that transmits data off-device invalidates it.

Age rating is a short questionnaire about content (violence, mature themes, gambling, and so on). Alltag is informational, so it lands at the lowest rating. Answer it once; it can be revised later.

Build and upload

The artifact you ship is an **archive** — a signed, Release- configured build of the app, the iOS analogue of a release jar built from your **main** profile rather than a dev one.

From Xcode

The GUI path, suitable for a first release:

1. Select a **generic iOS device** (not the Simulator) as the run destination, so Xcode builds for the device architecture.
2. **Product** ▶ **Archive**. This builds the Release configuration and signs it with your **distribution** certificate and an App Store provisioning profile.
3. The **Organizer** window opens with the archive. Choose **Distribute App** ▶ **App Store Connect** ▶ **Upload**.
4. Xcode validates and uploads the build to ASC.

From the command line

For CI or reproducibility, the same in two steps — archive, then export-and-upload:

```
cd ios
xcodegen generate
xcodebuild -project Alltag.xcodeproj -scheme Alltag \
  -configuration Release \
  -destination 'generic/platform=iOS' \
  -archivePath build/Alltag.xcarchive \
  -allowProvisioningUpdates archive
```

```
xcodebuild -exportArchive \
  -archivePath build/Alltag.xcarchive \
  -exportOptionsPlist ExportOptions.plist \
  -exportPath build/export \
  -allowProvisioningUpdates
```

The export produces an **.ipa**, which you upload with **Transporter** (Apple’s upload app) or the **xcrun altool** / **notarytool** family. Note that App Store and TestFlight builds are **not** separately notarized the way a Developer-ID Mac app is — Apple’s backend processes them after upload. **notarytool** is the right tool when you distribute a Mac app *outside* the Store; iOS Store uploads are handled by the ASC ingestion described next.

Processing on App Store Connect

After upload, the build is **Processing** in ASC for a few minutes to an hour: Apple unpacks it, runs automated checks, computes thinned variants per device, and then the build appears under the app’s **TestFlight** and **App Store** tabs, ready to attach to a version or hand to testers. This is the asynchronous “image accepted by the registry, now indexing” stage of your mental pipeline.

TestFlight: the staging rollout

TestFlight is Apple’s beta distribution channel — your staging environment before production. A processed build can go to testers without going to the public Store. Two audiences:

- **Internal testers:** up to 100 members of your ASC team. They get builds **immediately** after processing, with **no review**. This is your fast inner loop — the equivalent of deploying to a staging cluster you fully control.
- **External testers:** up to 10,000 people by email or a public link. Their *first* build per version requires a lightweight **Beta App Review**, after which subsequent builds flow with less friction.

Note. Beta App Review for external testers is lighter and faster than full App Store review, but it is still a human gate. Plan for it if you intend a public beta; internal testing has no such wait, so use internal testers to validate the upload pipeline itself before exposing anything externally.

The staging analogy is exact: prove the build on a controlled audience (internal), widen to a representative beta population (external), then promote the *same artifact* to production. You do not rebuild for release — the build that testers approve is the build you ship.

App Review: the human gate

This is the component with no backend equivalent. Before a build reaches the public Store, an Apple reviewer exercises the app against the **App Store Review Guidelines**. There is no automated-only fast path for a first release.

What reviewers check, and what is relevant to Alltag:

- **The app works.** No crashes, no broken features, no placeholder content, no dead links. A reviewer will open the app and use it; an obvious crash on launch is an instant rejection.
- **Privacy claims are true.** They cross-check your “Data Not Collected” declaration against observed behavior. For Alltag this is a strength — there is nothing to contradict — provided you keep it data-free.
- **A paid app delivers obvious value.** Apps that charge money must visibly justify the price; a thin or non-functional paid app is a classic rejection. Alltag must present clear, working functionality on first launch.
- **Metadata accuracy.** Screenshots and description must match the actual app; misleading marketing is a rejection reason.
- **Completeness.** A support URL that 404s, or a required field left to a placeholder, gets bounced.

Common rejection reasons map directly to those: crashes/bugs, inaccurate privacy labels, insufficient value for a paid app, metadata that misrepresents the app, and broken links. Review times vary but commonly land within a day or two; you can request expedited review for genuine emergencies. The practical posture: ship a build that genuinely works, keep the privacy declaration honest, and make the paid value obvious on first launch — for an honest on-device app like Alltag, review is a formality, not a hurdle.

Release: controlling the rollout

Approval does not automatically mean *live*. When you submit a version, you choose how it releases:

- **Manual release:** after approval the build sits in **Pending Developer Release** until you click to publish. This is the “approved, awaiting my go” gate — useful for coordinating a launch.
- **Automatic release:** it goes live as soon as it is approved.
- **Phased release:** once live, the update rolls out to existing users in increasing percentages over **seven days**, rather than to everyone at once. This is a canary/progressive rollout, and you can **pause** it if something looks wrong. New downloaders always get the latest version immediately; phasing only throttles the auto-update wave to existing users.

The pattern matches a progressive deployment with a manual promotion gate: hold at “approved,” release on your signal, then ramp exposure gradually with the ability to halt.

Versioning: marketing version vs build number

Two numbers travel with every build, and ASC treats them very differently. Both are real settings in `project.yml`:

```
MARKETING_VERSION: "0.1.0"
CURRENT_PROJECT_VERSION: "1"
```

- **MARKETING_VERSION** is `CFBundleShortVersionString` — the public, human-facing version, here **0.1.0**. This is the number users see on the App Store page. Each *App Store version record* is keyed to one of these; you cannot submit two different builds to the Store under the same marketing version as separate releases.
- **CURRENT_PROJECT_VERSION** is the build number (`CFBundleVersion`), here **1**. This identifies a specific build *within* a marketing version. It is the value that must be **unique and increasing** for every upload: re-upload **0.1.0 (1)** and ASC rejects it as a duplicate; bump to **0.1.0 (2)** and it accepts.

The rule of thumb: bump **CURRENT_PROJECT_VERSION** for **every upload** (including each TestFlight build of the same release), and bump **MARKETING_VERSION** when you ship a new *public* version. The pair **0.1.0 (1)** is read as “marketing version 0.1.0, build 1” — semver for humans, monotonic counter for the upload system.

Quarkus analogy. **MARKETING_VERSION** is your released semver (**0.1.0**) — what a consumer pins. **CURRENT_PROJECT_VERSION** is like a CI build number or a unique image tag: it must increment on every push to the registry so two artifacts never collide, even within the same released version.

Takeaways

- App Store Connect is the console: it holds the app *record* (bundle id `de.everydaygermany.app`, German primary language), metadata, pricing, privacy, and uploaded builds — everything but the signed binary, which you upload to it.
- Alltag is **paid up front**, so you pick a price tier and write **no in-app purchase code** — the store gates the download, your binary ships fully unlocked.
- App Privacy is the easy path: Alltag collects **no data**, so you answer “Data Not Collected” — but keep it honest, since any data-transmitting SDK invalidates the claim.
- Distribute by **archiving** the Release build (Xcode Organizer or `xcodebuild archive` + Transporter/ `altool`), then let ASC process it.
- **TestFlight** is staging: internal testers get builds instantly with no review; external testers need a lightweight Beta App Review.
- **App Review** is the human gate with no backend analogue: it checks that the app works, that privacy claims are true, and that a paid app delivers obvious value.
- Release manually, automatically, or in a 7-day **phased** rollout you can pause — a progressive deploy with a promotion gate.
- `MARKETING_VERSION` (`0.1.0`) is the public semver; bump `CURRENT_PROJECT_VERSION` (`1`) on **every upload** — it must be unique and increasing.

Next: *Production: Privacy, Crashes, Performance, and Updates*

Production: Privacy, Crashes, Performance, and Updates

On a Quarkus service, “production” means a fleet you own: you tail logs, scrape Prometheus, watch Sentry, and roll a new container when something breaks. An iOS app inverts every one of those assumptions. The **device is production**, you do not own it, you cannot SSH into it, and — for *Alltag* — you have deliberately chosen to collect nothing about it. This chapter walks the four things you still must get right with no server underneath you: privacy declarations, crash and metric observability, performance (including an on-device LLM), and shipping updates.

Privacy: declaring what you touch

A backend declares privacy in a GDPR record and a DPA. Apple wants it *in the binary*. Two mechanisms matter.

The Privacy Manifest

`PrivacyInfo.xcprivacy` is a property-list file you add to the app target (and that every third-party SDK must now ship too). It declares four things: data you collect, tracking, tracking domains, and **required-reason APIs**. For *Alltag* the data and tracking sections are almost empty — the app collects nothing and tracks nothing — but the file is still mandatory.

```
<key>NSPrivacyTracking</key>
<false/>
<key>NSPrivacyCollectedDataTypes</key>
<array/>
<key>NSPrivacyTrackingDomains</key>
<array/>
```

The interesting part is **required-reason APIs**. Apple flags a set of common APIs (file timestamps, `UserDefaults`, disk space, system boot time) that have historically been abused for fingerprinting. If you call one, you must declare an approved reason code, or App Review rejects the upload. *Alltag* uses `UserDefaults` for theme and language preferences, so it declares reason `CA92.1` (app’s own data, not shared). It checks free disk space before downloading the 3.11 GB model, so it declares the disk-space reason too.

Quarkus analogy. Think of `PrivacyInfo.xcprivacy` as a `compile`-time SBOM for *behavior*, not dependencies. The linker-level manifest is checked at upload the way a strict license scanner gates a `mvn deploy` — except here a missing entry is a hard reject, not a warning.

App Tracking Transparency

ATT is the `Ask App Not To Track` prompt. It is required only if you track the user across other apps/sites or share data with data brokers. *Alltag* does neither, so this is trivial: **no `AppTrackingTransparency` import, no prompt, nothing to do**. That is the whole point of the local-only design — entire

categories of compliance work simply evaporate. The privacy “nutrition label” on the App Store product page reads *Data Not Collected*, which is also a marketing asset for an app whose pitch is “your documents never leave this phone.”

Crashes and observability with no server

Here is the uncomfortable truth: you have **no server logs**. Nothing flows to a central place. When a `fatalError` fires on a stranger’s iPhone in Munich, you find out only if Apple’s own pipeline tells you.

Xcode Organizer and MetricKit

Apple gives you two privacy-preserving channels, both opt-in by the user via “Share With App Developers”:

- **Organizer crash reports.** Symbolicated crash logs aggregated in Xcode’s Organizer window, grouped by signature, with device and OS breakdowns. This is your Sentry — except it is sampled, delayed by hours to days, and only covers users who opted in.
- **MetricKit (`MXMetricManager`).** A daily on-device digest of launch time, hang rate, memory peaks, disk writes, battery, and crash diagnostics, delivered to your app the next day. You receive it locally and *could* forward it — but for *Alltag* that would mean adding a network call to a no-network app.

```
final class Metrics: NSObject, MXMetricManagerSubscriber {
    func didReceive(_ payloads: [MXMetricPayload]) {
        // Persist locally; never transmit.
        for payload in payloads {
            store.append(payload.jsonRepresentation())
        }
    }
}
```

The telemetry tradeoff

On Quarkus you would not hesitate to wire Sentry — more signal is strictly better. For *Alltag* it is a genuine conflict. Any crash SDK (Sentry, Crashlytics, Bugsnag) opens a network connection, ships device identifiers, and forces you to add data-collection entries to the privacy manifest — directly contradicting the *Data Not Collected* promise. So the decision is deliberate: **rely on Apple’s opt-in Organizer/MetricKit only, and forward nothing**. You trade observability for an airtight privacy claim. That is the right call here, but make it consciously — debugging a crash you cannot reproduce, with no breadcrumbs, is the price.

Gotcha. “I’ll just log to a file and read it later” does not help: you have no way to retrieve that file from a user’s device. The on-device crash log is only useful if Apple surfaces it in Organizer. Design for low crash *rates* (no force-unwraps, no `try!`), because your ability to *react* is structurally limited.

Performance: Instruments and main-thread hygiene

Profile on a **real device**, never the Simulator — the Simulator uses your Mac’s CPU/GPU/RAM and lies about everything that matters, especially Metal and memory pressure. Instruments is your profiler:

Instrument	Finds	Backend analog
Time Profiler	CPU hot paths	async-profiler
Allocations	leaks, growth	heap dump / MAT
Hangs	main-thread stalls	event-loop block
SwiftUI	redundant redraws	—

The cardinal rule: **the main thread renders the UI at up to 120 Hz, so it must never block**. SwiftUI views and observable UI state should be `@MainActor`; anything expensive (file crypto, OCR, model inference) must run off it. This is stricter than a Quarkus worker thread — there, a slow request hurts one caller; here, a 200 ms main-thread stall is a visible, juddering hang for the user holding the phone.

```
@MainActor
final class DecodeViewModel: ObservableObject {
    @Published var result: DecodeResult?

    func decode(_ image: Data) async {
        // Inference runs off the main actor (see below);
        // only the assignment hops back on.
        result = await engine.decode(image)
    }
}
```

Watch **launch time** too. iOS will *kill* an app that takes too long to launch (the watchdog), so never block `init` on heavy work — lazy-load the model, the SwiftData store, and large content files.

The on-device LLM is the performance story

Alltag’s Decoder runs **Gemma 4 E2B**, GGUF **Q4_K_M** (3.11 GB), on **llama.cpp** with the **Metal backend** — entirely on the phone. No backend service exists. That makes the model the single biggest performance concern in the app, and it touches every axis at once:

- **Model load time.** Reading 3.11 GB of weights into memory takes seconds. Load lazily, off the main actor, with a warm-up pass and a visible progress state — never on launch.
- **RAM footprint.** Weights plus KV cache can dominate the process’s memory budget. iOS jetsams (force-kills) memory-hungry apps without warning. *Alltag* **caps context far below the model’s 128K** (letters are short — 4–8K is plenty) to bound the KV cache, runs a single inference at a time, and unloads under memory pressure.
- **Why Q4 and Metal matter.** **Quantization** (Q4 = 4-bit weights) is what makes 2.3B effective parameters fit and run at all; full-precision weights would be several times larger and far slower. **Metal** moves the matrix math onto the GPU/Neural Engine instead of the CPU, which is the differ-

ence between a usable few-seconds decode and an unusable one. A low-RAM device may fall back to a smaller `Q3_K_M` quant; a too-small device is gated out with a clear message rather than crashed.

- **Thermals and battery.** Sustained inference heats the device and drains the battery; iOS responds by *throttling* the CPU/GPU, which slows the next decode. The Decoder is a short, on-demand burst (decode one letter, stop) — not a streaming chatbot — which keeps this bounded. Always run inference off the main actor so the UI stays at 60/120 Hz while the GPU is busy.

Note. On a backend you would scale a slow model horizontally — add pods. On-device there is exactly one “pod,” and it is also rendering the UI, playing audio, and trying not to get hot. Every millisecond and every megabyte is a fixed, non-elastic budget.

App size and the shipped model

A multi-hundred-megabyte — here, multi-gigabyte — model is a real shipping constraint. You **cannot bundle 3.11 GB into the app binary**: the App Store cellular-download limit and review friction make it impractical. *Alltag*’s answer is to **download the model on first run** — resumable, Wi-Fi-recommended, checksum-verified — and store it in Application Support, **excluded from iCloud/iTunes backup** (it is large and re-downloadable). The app binary stays small; the heavy asset arrives just-in-time. (On-Demand Resources and `Background Assets` are Apple’s alternatives for large bundled content, but a gated HF/CDN download fits a 3 GB model better.)

Updates: shipping new versions

Without a server you cannot hotfix; **every change ships through App Review** (see the previous chapter). Two production concerns dominate.

SwiftData store migration

Your users’ data lives in a SwiftData store on their device, and an update must open the *old* store without losing anything. For additive changes (new optional property, new model) SwiftData does a **lightweight migration** automatically. For anything structural (renames, type changes, splits) you must define a `SchemaMigrationPlan` with explicit stages:

```
enum AlltagMigration: SchemaMigrationPlan {
    static var schemas: [any VersionedSchema.Type] {
        [SchemaV1.self, SchemaV2.self]
    }
    static var stages: [MigrationStage] {
        [.lightweight(fromVersion: SchemaV1.self,
                      toVersion: SchemaV2.self)]
    }
}
```

This is your Flyway/Liquibase — except there is **no DBA to run it and no staging database to test against**. It executes on the user’s phone, once, irreversibly. Test migrations against a real old store (seed v1 data, ship v2, assert it survives — *Alltag* already has a “data survives store reopen” test). A botched migration is data loss for a paid user with no recovery path.

Backward compatibility and OS support

Alltag targets **iOS 17.0 minimum**. You cannot force anyone to update the OS, so deciding to raise the floor to iOS 18 strands every user still on 17 until *they* upgrade. Gate newer APIs with `if #available(iOS 18, *)` and keep a working path for the minimum. Treat the minimum-iOS bump as a product decision, not a refactor.

Accessibility as production quality

On a backend, accessibility is invisible; on a consumer app it is a correctness requirement — and for an app helping newcomers navigate German bureaucracy, it is core to the mission. Three pillars:

- **VoiceOver.** Every interactive element needs a meaningful label; decorative ones are hidden. Severity pills must announce *meaning* (“urgent, reply by June 30”), not color.
- **Dynamic Type.** *Alltag* uses **SF Pro Rounded** via the system text styles with **no hard-coded font sizes**, so text scales from small up to the largest accessibility sizes. Layouts must not clip at XXL — test there.
- **RTL correctness.** Arabic ships in v1, so layout uses **leading/trailing**, never left/right, and is verified in a pseudo-RTL locale from day one.

Localization rollout

Alltag is built in **German (default) + English first**, with the remaining seven of nine languages (TR, FR, ES, IT, AR, RU, ZH) localized near the end via String Catalogs. The i18n and RTL *plumbing* exists from day one even though the *content* lands late — so adding a language is a translation pass, not a re-architecture. Decoder *output* follows the user’s chosen app language, which the on-device model handles directly. Ship DE+EN polished rather than nine languages half-done.

Takeaways

- The device is production: no logs, no SSH, no hotfix. Design for low crash rates because your ability to react is limited.
- `PrivacyInfo.xcprivacy` + required-reason API codes are mandatory at upload; ATT is a no-op for a no-tracking app like *Alltag*.
- Observability is Apple’s opt-in Organizer + MetricKit; adding a crash SDK would break the *Data Not Collected* promise — a conscious tradeoff, not an oversight.
- Profile on device with Instruments; keep the main actor unblocked. The on-device LLM (Q4 GGUF on Metal, off the main actor, capped context, downloaded not bundled) is the central performance and app-size concern.
- Updates ship only through review; SwiftData migrations run once, irreversibly, on the user’s phone — test them like Flyway you can never roll back.
- Accessibility (VoiceOver, Dynamic Type, RTL) is production correctness; ship DE+EN well, localize the rest last.

Next: Appendix A *Java/Quarkus to Swift/SwiftUI Cheat Sheet*.

APPENDICES

Reference

A translation table, a list of the traps that catch Java developers, a concrete study plan, and the resources worth your time.

Java/Quarkus to Swift/SwiftUI Cheat Sheet

A keep-it-open translation table. Left is what you know; right is the Swift/SwiftUI equivalent. Notes flag the cases where the mapping leaks. Examples reference the Alltag app (`AppEnvironment` , `DocumentRecord` , `KeychainKeyStore`).

Language constructs

Java/Quarkus	Swift	Note
<code>record</code>	<code>struct</code>	Value type; memberwise init free.
sealed interface	<code>enum</code>	Cases carry associated values.
<code>interface</code>	<code>protocol</code>	Can have default impls.
abstract class	protocol + extension	No state in the extension.
<code>Optional<T></code>	<code>T?</code>	Language-level, no boxing.
<code>null</code>	<code>nil</code>	Only for optionals.
<code><T></code> generics	<code><T></code>	Reified, not erased.
lambda	closure	<code>{ x in ... }</code> .
<code>Stream</code>	<code>map / filter / reduce</code>	Eager, on <code>Array</code> .
<code>.collect(toList())</code>	returns <code>Array</code>	No terminal op needed.
<code>final</code> field	<code>let</code>	Immutable binding.
<code>var</code> (mutable)	<code>var</code>	Mutable binding.
<code>instanceof</code>	<code>is</code>	Returns <code>Bool</code> .
cast <code>(T)x</code>	<code>as? / as!</code>	<code>as?</code> is safe, <code>as!</code> traps.
<code>static</code> member	<code>static</code>	On struct/enum/class.
<code>static</code> (overridable)	<code>class</code>	On class only.
checked exception	<code>throws</code>	Untyped; no signature list.
<code>equals / hashCode</code>	<code>Equatable / Hashable</code>	Often synthesized.
<code>toString()</code>	<code>CustomStringConvertible</code>	<code>var description</code> .
<code>Comparable</code>	<code>Comparable</code>	Implement <code><</code> .
<code>cond ? a : b</code>	<code>cond ? a : b</code>	Same ternary.
<code>x != null ? x : y</code>	<code>x ?? y</code>	Nil-coalescing.
enhanced <code>switch</code>	<code>switch</code>	Exhaustive, no fallthrough.
<code>enum</code> (plain)	<code>enum</code>	Plus raw/associated values.
varargs <code>T...</code>	<code>T...</code>	Variadic param.
<code>this</code>	<code>self</code>	Explicit in closures.

```
// sealed interface -> enum with associated values
enum LoadState {
    case idle
    case loading
    case loaded([DocumentRecord])
    case failed(Error)
}
```

```
// Java equivalent
sealed interface LoadState
    permits Idle, Loading, Loaded, Failed {}
```

Concurrency

Java/Quarkus	Swift	Note
<code>new Thread(r)</code>	<code>Task { }</code>	Cooperative, not 1:1.
<code>Runnable</code>	closure / <code>@Sendable</code>	
<code>CompletableFuture<T></code>	<code>async</code> func	Returns <code>T</code> .
<code>Uni<T></code> (Mutiny)	<code>async</code> func	<code>await</code> to resolve.
<code>.thenApply</code>	<code>await</code> then call	Linear code.
<code>ExecutorService</code>	<code>TaskGroup</code>	Structured.
submit + join	<code>async let</code>	Child task, awaited.
<code>synchronized</code>	<code>actor</code>	Serializes access.
<code>Lock</code>	<code>actor</code>	No manual lock/unlock.
<code>volatile</code>	<code>Sendable</code>	Compile-checked.
thread-safety	<code>Sendable</code>	Enforced by compiler.
EDT / UI thread	<code>@MainActor</code>	Annotate type or func.
<code>SwingUtilities.invokeLater</code>	<code>await MainActor.run</code>	
<code>ThreadLocal<T></code>	<code>@TaskLocal</code>	Inherited by children.
<code>future.cancel(true)</code>	<code>task.cancel()</code>	Cooperative.
check interrupted	<code>Task.isCancelled</code>	Or <code>try Task.checkCancellation()</code> .

```
// async let – parallel children, awaited together
async let docs = store.fetchDocuments()
async let user = api.currentUser()
let view = ViewModel(docs: try await docs,
                     user: try await user)
```


Build & tooling

Maven/Gradle	Swift	Note
Maven / Gradle	SPM + Xcode	SPM for libs/deps.
<code>pom.xml</code>	<code>Package.swift</code>	Manifest is Swift code.
<code>build.gradle</code>	<code>project.yml</code>	XcodeGen (Alltag).
generated <code>.idea</code>	generated <code>.xcodproj</code>	Don't commit it.
<code>mvn test</code>	<code>xcodebuild test</code>	
JUnit 5	Swift Testing	New; XCTest also OK.
<code>@Test</code>	<code>@Test</code>	Swift Testing macro.
<code>assertEquals(a,b)</code>	<code>#expect(a == b)</code>	Swift Testing.
<code>assertEquals(a,b)</code>	<code>XCTAssertEqual(a,b)</code>	XCTest.
<code>assertThrows</code>	<code>#expect(throws:)</code>	
Mockito mock	protocol + hand fake	No mock framework.
<code>@InjectMock</code>	inject a fake impl	Via initializer.
JAR artifact	<code>.app</code> / <code>.ipa</code>	Bundle, not archive.
<code>mvn package</code>	Archive	Product > Archive.
IntelliJ IDEA	Xcode	
<code>~/ .m2</code>	<code>~/Library/Caches</code> SPM	Resolved per project.

```
// Mockito -> protocol + hand-written fake
protocol KeyStore {
    func key(for id: String) throws -> SymmetricKey
}
struct FakeKeyStore: KeyStore {
    var stub: SymmetricKey
    func key(for id: String) throws -> SymmetricKey {
        stub
    }
}
```

Runtime, DI & frameworks

Java/Quarkus	Swift / SwiftUI	Note
<code>@Inject</code>	<code>@Environment</code>	Resolve from env.
<code>@ApplicationScoped</code> bean	<code>@Observable</code> class	Held in environment.
CDI producer <code>@Produces</code>	<code>static func live()</code>	Factory on the type.
<code>@PostConstruct</code>	initializer	Plain <code>init</code> .
bean scope (request)	view lifetime / <code>@State</code>	
<code>application.properties</code>	<code>Info.plist</code>	Static config.
runtime config	<code>UserDefaults</code>	Mutable prefs.
<code>@ConfigProperty</code>	<code>@AppStorage</code>	Bound to <code>UserDefaults</code> .
JPA <code>@Entity</code>	<code>@Model</code>	SwiftData macro.
<code>EntityManager</code>	<code>ModelContext</code>	Unit of work.
<code>em.persist(x)</code>	<code>context.insert(x)</code>	
Panache <code>list()</code>	<code>FetchDescriptor</code>	Typed query.
named query / <code>@Query</code>	<code>#Predicate</code>	In <code>FetchDescriptor</code> .
<code>EntityManagerFactory</code>	<code>ModelContainer</code>	App-wide.
<code>@Transactional</code>	<code>context.save()</code>	Explicit save.
Jackson <code>@JsonProperty</code>	<code>Codable</code>	Synthesized.
<code>@JsonProperty("x")</code>	<code>CodingKeys</code>	Rename mapping.
<code>ObjectMapper</code>	<code>JSONEncoder / Decoder</code>	
SLF4J <code>Logger</code>	<code>os.Logger</code>	Unified logging.
<code>log.info(...)</code>	<code>logger.info("...")</code>	
JAX-RS <code>RestClient</code>	<code>URLSession</code>	<code>data(for:)</code> async.
<code>@Path</code> / <code>@GET</code>	<code>build URLRequest</code>	No annotations.
Bean Validation <code>@NotNull</code>	<code>guard</code> / <code>throws</code>	Validate in code.
<code>KeyStore</code> / JCA	Keychain + CryptoKit	<code>KeychainKeyStore</code> .
<code>Cipher</code> AES-GCM	<code>AES.GCM.seal</code>	CryptoKit.

```
// CDI @Inject -> @Environment
struct DocumentList: View {
    @Environment(AppEnvironment.self) private var env
    var body: some View {
        List(env.documents) { Text($0.title) }
    }
}
```

```
// CDI producer -> static live() factory
extension AppEnvironment {
    static func live() -> AppEnvironment {
        AppEnvironment(keyStore: KeychainKeyStore(),
                        container: .live())
    }
}
```

```
// Panache list() -> FetchDescriptor + #Predicate
let recent = FetchDescriptor<DocumentRecord>(
    predicate: #Predicate { $0.isArchived == false },
    sortBy: [SortDescriptor(\.createdAt, order: .reverse)]
)
let docs = try context.fetch(recent)
```

Next: the gotchas that this table can't warn you about — see *Idioms and Gotchas for Java Developers*.

Idioms and Gotchas for Java Developers

A scannable catalogue of traps that specifically catch engineers coming from Java/Quarkus. Each item: the trap, why it bites, and the fix. Skim it once now; return to it when something compiles but behaves wrong.

Structs are values, not references

The biggest mental shift. A `struct` is *copied* on assignment and on every parameter pass. Mutating the copy does not touch the original — there is no shared object, no aliasing.

```
var a = Point(x: 1)
var b = a          // copy
b.x = 9            // a.x is still 1
```

Coming from Java, where everything but primitives is a reference, this surprises you constantly. `class` gives you Java-like reference semantics; `struct` (the default for models) does not.

Mutating a struct inside a collection/loop

Because structs are values, `for item in array { item.x = 1 }` does nothing useful — `item` is a *copy*. To mutate in place, index back into the storage:

```
for i in array.indices { array[i].x = 1 }
```

A `for-each` loop variable over a value-type array is read-only by design; this catches everyone once.

Optionals are not null

`String?` is a distinct *type*, not a nullable reference. You cannot call methods on it directly, and there is no silent `NullPointerException` — the compiler forces you to unwrap. Do **not** force-unwrap with `!`; that is a deliberate crash.

```
guard let name = maybeName else { return }
// name is a non-optional String here
```

Prefer `guard let`, `if let`, or `??`. Reserve `!` for genuine “impossible to be nil” invariants — it is the Swift equivalent of `throw new AssertionError()`.

== VS ===

In Java `==` is identity and `.equals()` is value. In Swift it is reversed: `==` is **value** equality (`Equatable`), and `===` is **reference identity** (class instances only).

```
a == b    // values equal? (Equatable)
a === b   // same object? (classes only)
```

`===` does not even compile for structs — they have no identity.

throws has no exception type list

Swift errors are untyped at the signature: a function is just `throws`, with no `throws IOException, SQLException` list. There are no checked-vs-unchecked categories. You `catch` and switch on the concrete error type at the call site.

```
do { try decode() }
catch let e as LLError { handle(e) }
catch { fallback() }
```

You lose compiler-enforced exception contracts; you gain less ceremony. (Swift 6 adds opt-in *typed* throws, rarely needed.)

String is not Int-indexable

`s[0]` does not compile. A Swift `String` is a sequence of *grapheme clusters* (user-perceived characters), not fixed-width chars, so you use `String.Index`, not integers.

```
let first = s[s.startIndex]
let third = s.index(s.startIndex, offsetBy: 2)
```

This protects you from breaking emoji and combined characters — but it means Java-style `charAt(i)` loops must be rewritten.

Trailing-closure syntax

When the last argument is a closure, it moves outside the parentheses. This is why SwiftUI looks the way it does, and why a method call can look like a language construct.

```
items.map { $0 * 2 }           // map(transform:)
Button("Save") { save() }     // trailing closure
```

`$0` is the first implicit argument. Multiple trailing closures use labels: `Button { ... } label: { ... }`.

self capture and retain cycles

Closures capture `self` *strongly* by default. A long-lived closure that captures `self`, stored on an object that owns the closure, creates a retain cycle — ARC never frees it (a leak). Break it with a capture list.

```
task = Task { [weak self] in
    await self?.load()
}
```

There is no tracing GC to clean this up; ARC is deterministic reference counting, so cycles leak permanently. Reach for `[weak self]` in stored, escaping closures.

@MainActor and the UI thread

All UI work must be on the main actor. Touching `@Published` state or a view from a background context is undefined behavior. Mark UI types `@MainActor`; hop with `await MainActor.run { ... }`.

```
@MainActor final class VM: ObservableObject { /* ... */ }
```

Like Swing's EDT, but compiler-enforced. The flip side: do **not** do heavy work (crypto, the on-device LLM) on `@MainActor` — push it off, then hop back to assign results.

Struct mutation needs `var` + mutating

A method that changes a struct's own stored properties must be marked `mutating`, and you can only call it on a `var`. A `let` struct is *deeply* immutable.

```
struct Counter { var n = 0
    mutating func bump() { n += 1 } }
let c = Counter(); // c.bump() x won't compile
```

`final`-by-value: think Java `record` that can declare in-place mutators.

Protocols with associated types can't be plain existentials

A protocol with an `associatedtype` (or `Self` requirements) — the Swift analog of `interface Repo<T>` — cannot be used as a bare variable type the way a Java generic interface can.

```
func first<C: Collection>(_ c: C) -> C.Element? // ✓
let xs: any Collection = [] // limited
let xs: some Collection = [1] // opaque, one type
```

Use a generic constraint, `some` (opaque, one concrete type), or `any` (boxed existential, with limits). This is the error `...can only be used as a generic constraint`.

Array/Dictionary are value types (copy-on-write)

Collections are structs: assigning or passing one is logically a copy. Swift optimizes this with **copy-on-write** — the buffer is shared until you mutate, then it clones. Cheap to pass, but no shared mutation across references.

```
var a = [1, 2]; var b = a
b.append(3) // a is still [1, 2]
```

No `Collections.unmodifiableList` needed — immutability is the default via `let`.

Compiler-synthesized memberwise init

A `struct` gets a free `init` covering all stored properties — no Lombok, no constructor boilerplate.

```
struct Letter { let title: String; let date: Date }
let l = Letter(title: "Bescheid", date: .now)
```

Caveat: define *any* custom `init` and you lose the synthesized one (put yours in an `extension` to keep both). Classes get no memberwise `init`.

switch must be exhaustive

A `switch` must cover every case or include `default`; a non-exhaustive switch does not compile. Over an `enum` with no `default`, adding a case turns *every* switch into a compile error — which is a feature: the compiler lists what to update.

```
switch severity {
case .info: ...
case .urgent: ...
} // x if .legal exists and isn't handled
```

No implicit numeric conversion

`Int` and `Double` do not auto-promote. `let x: Double = anInt` fails; you must convert explicitly. There is no `1 + 1.0` for free.

```
let d = Double(count) / 2.0
```

Tedious at first, but it kills a whole class of silent integer/float bugs. Same for `Int8` / `Int` / `UInt` — all explicit.

enum is far more than a Java enum

A Swift `enum` can carry **associated values** per case — it is a sum type / sealed hierarchy, not just a named constant set.

```
enum Decode {
    case pending
    case done(summary: String, deadline: Date?)
    case failed(LLMError)
}
```

This is your `sealed interface` + records, but lighter. Combined with exhaustive `switch`, it models state machines cleanly.

Access control defaults to internal

No modifier means `internal` — visible across the whole **module**, not package-private. A module is a *build target* (the app, a Swift package), not a folder. Levels: `private` < `fileprivate` < `internal` < `public` < `open`. There is no `protected`.

```
struct API { func go() {} } // internal by default
public struct Lib { /* ... */ }
```

`public` exposes across modules but does not allow subclassing/ overriding from outside — that needs `open`.

guard for early return

`guard` is “assert-or-leave”: its `else` branch *must* exit scope (`return` / `throw` / `break`). It keeps the happy path unindented and binds unwrapped values into the *enclosing* scope.

```
guard let url = makeURL() else { return }
use(url)    // url in scope, non-optional
```

Prefer it over a pyramid of `if let`.

lazy and computed properties

`lazy var` defers initialization until first access (handy for the expensive model handle). A computed property (`var x: T { ... }`) runs code on every read — no stored backing, like a getter.

```
lazy var engine = LlamaEngine()    // built once, on use
var isReady: Bool { engine.loaded } // recomputed each read
```

`lazy` is not thread-safe; `lazy` requires `var`.

Codable vs Jackson

JSON mapping is compiler-synthesized from the type — **no annotations**, no `ObjectMapper` bean. Conform to `Codable` and it just works; customize key names with a `CodingKeys` enum.

```
struct Letter: Codable {
    let title: String
    enum CodingKeys: String, CodingKey {
        case title = "betreff"
    }
}
```

No reflection at runtime; encoding/decoding is generated at compile time.

Naming conventions

Methods and properties are `lowerCamelCase`; types and protocols are `UpperCamelCase`. There are **no `get` / `set` prefixes** — a property *is* the accessor. Booleans read as questions (`isReady` , `hasDeadline`). Argument labels are part of the method name: `move(from:to:)`.

```
let isReady = doc.hasDeadline    // not getHasDeadline()
```

Enum cases are `lowerCamelCase` too (`.info` , not `.INFO`).

import pulls a whole module

`import SwiftUI` brings in the *entire* module — there is no per-type import like `import java.util.List`. You do not (and cannot) import individual symbols normally; the module namespace is flat once imported.

```
import SwiftData    // all of it, not one class
```

Fewer import lines, but name collisions across modules are resolved by qualifying (`SwiftUI.Path` vs `CoreGraphics` -era types).

Takeaways

- Value vs reference is the root of most surprises: structs and collections copy; only `class` aliases.
- Optionals replace `null` safely — `guard let`, never `!`; `==` is value, `===` is identity.
- Errors are untyped, numerics never auto-convert, `switch` is exhaustive, and `enum` carries data — lean on the compiler.
- Watch `[weak self]` (ARC, no GC) and `@MainActor` (UI thread, enforced); push heavy work off it.
- Idioms — synthesized inits/ `Codable`, trailing closures, `lazy`, module-wide `import`, camelCase with no `get` / `set` — are less boilerplate once the reflexes flip.

Next: Appendix C *A Study Plan*.

A Study Plan

A book is inert without a schedule. This appendix turns the chapters into an 8-week, part-time curriculum — roughly 5-7 hours per week, the kind of load you can sustain in evenings and on weekends without burning out. It assumes you are employed full-time as a backend engineer and learning on the side.

The structure is deliberate: **read a little, build a lot**. Each week pairs reading with a concrete, small deliverable in a throwaway practice app, plus optional reinforcement against the real *Alltag* codebase, which you already have on disk. You learn Swift by writing Swift, not by reading about it.

Before week 1: setup (about an hour)

- Install **Xcode** from the Mac App Store (it is large — start the download the night before).
- Open Xcode once, let it install the additional components, and accept the licence (`sudo xcodebuild -license accept` if it nags from the CLI).
- `brew install xcodegen xcbeautify` — the two tools *Alltag* uses.
- Clone/open the Alltag project: `cd ios && xcodegen generate && open Alltag.xcodeproj`, then press the **Run** button (the simulator boots).
- Create a scratch project: **File > New > Project > iOS App**, interface *SwiftUI*. This is your sandbox for the exercises below.

Note. Do not try to be productive in week 1. The goal is a working toolchain and the muscle memory of build-run-iterate. Everything else builds on that loop.

The eight weeks

Week 1 — Orientation and the language floor

- **Read:** *The Landscape: from JVM to iOS, Swift Fundamentals: Values, Optionals, Errors*.
- **Build:** in your scratch app, write a plain Swift file with a handful of functions that take optionals and `throws`, and call them from a test. Force yourself to use `guard let`, `if let`, `??`, and a `do / catch`.
- **Reinforce:** open Alltag's `FileCryptor.swift` and `AppEnvironment.swift` and trace every `try`, `guard`, and optional. You have read these in the book; now read them cold.
- **Milestone:** you can explain, out loud, why `let env = AppEnvironment.live()` cannot throw but the initializer it calls can.

Week 2 — The type system

- **Read:** *Types: Structs, Enums, Protocols, Generics*.

- **Build:** model a tiny domain you know (say, an invoice) three ways — as a `struct`, with an `enum` for its status (with associated values for, e.g., `.refunded(amount:)`), and behind a `protocol` with two conforming types.
- **Reinforce:** re-implement Alltag's `AppLanguage` enum from memory, then diff against the real file. Note every computed property you forgot.
- **Milestone:** you reach for `struct` by default and can articulate when you would deliberately choose a `class`.

Week 3 — Functions, closures, and concurrency

- **Read:** *Functions, Closures, and Functional Style, Memory and Concurrency: ARC, async/await, Actors.*
- **Build:** write an `async` function that fetches from a public JSON API with `URLSession`, decode it with `Codable`, and call it from a `Task` in a button action. Add a `TaskGroup` that fetches three things in parallel.
- **Reinforce:** find every `@MainActor` and `Sendable` in Alltag and explain why each is there. Turn on **strict concurrency** in your scratch app (`SWIFT_STRICT_CONCURRENCY = complete`) and fix the warnings.
- **Milestone:** you no longer reach for completion handlers, and a data-race warning reads as a helpful compiler message rather than noise.

Week 4 — SwiftUI: the mental model

- **Read:** *The SwiftUI Model: Declarative Views and State, Layout: Stacks, Modifiers, and the Layout System.*
- **Build:** a single screen with `@State`, a `Toggle`, a `TextField`, and a computed result that updates live. Deliberately break it by mutating state from the wrong place, and watch what the compiler says.
- **Reinforce:** add a `#Preview` to one of Alltag's placeholder views and iterate on padding/colour using the canvas, without ever launching the simulator.
- **Milestone:** you can predict what re-renders when a given `@State` changes.

Week 5 — SwiftUI: real screens

- **Read:** *Building Screens: Lists, Navigation, Forms, Tabs.*
- **Build:** a two-tab app with a `NavigationStack`, a `List` of `Identifiable` items you can tap into a detail screen, and a `Form`-based settings tab with a `Picker`. Add `.swipeActions` to delete a row.
- **Reinforce:** read Alltag's `RootView` and `AppTab`; then rebuild the Settings screen exercise from that chapter against the real `ThemeController` and `LanguageStore`.
- **Milestone:** you can assemble a multi-screen navigable app from memory.

Week 6 — Architecture and persistence

- **Read:** *Architecture: State, Dependencies, and the @Observable Environment, Persistence: SwiftData, the Keychain, Files, and CryptoKit.*

- **Build:** give your practice app an `@Observable` environment container injected at the root (mirroring `AppEnvironment`), and a `@Model` entity persisted with `SwiftData`, listed via `@Query`. Add, edit, and delete records.
- **Reinforce:** this is the week to read Alltag's whole `Core/Persistence` folder end to end — `PersistenceController`, `DocumentRecord`, `EncryptedFileStore`, `FileCryptor`, `KeyStore` — and trace one document from bytes to encrypted file to metadata row.
- **Milestone:** you could add a new `@Model` entity to Alltag and wire it through the environment yourself.

Week 7 — The toolchain and tests

- **Read:** *The Toolchain: Xcode, SPM, XcodeGen, and the Build*, *Testing: Swift Testing, XCTest, Previews, and CI*.
- **Build:** add a Swift Testing target to your practice app and port a couple of XCTest cases to `@Test` / `#expect`. Inject a fake via a protocol.
- **Reinforce:** run Alltag's `ios/scripts/ci.sh` locally and read what it does line by line. Add one new test to the suite and watch it run.
- **Milestone:** you can add a dependency with SPM, regenerate the project with XcodeGen, and run the full test suite from the command line.

Week 8 — Shipping

- **Read:** *Code Signing and Provisioning*, *Demystified*, *Distribution: App Store Connect, TestFlight, and Review*, *Production: Privacy, Crashes, Performance, and Updates*.
- **Build:** enrol in the Apple Developer Program (or use free provisioning), flip Alltag's signing settings as described, and run it **on your own iPhone**. Then archive a Release build and push it to TestFlight for yourself.
- **Milestone:** an app you built is running on your phone from TestFlight. You are now, by any practical definition, an iOS developer.

A faster track

If you can give it full days rather than evenings, compress the above into **two weeks**: Part I in days 1-3, Part II in days 4-7, Part III in days 8-10, with the rest of each block spent building. The ordering still holds — do not skip the concurrency or persistence weeks, because they are where a backend engineer's instincts are most often subtly wrong.

A capstone worth doing

When the eight weeks are done, build one small, complete app of your own from a blank project to TestFlight: a single feature, persisted locally, tested, and signed. Nothing teaches the gaps in your knowledge like the unglamorous last 10% — an app icon, a launch screen, a privacy manifest, a rejected build, a resubmission. That last 10% is the whole point of Part III, and the reason this book exists.

Takeaways

- Read a little, build a lot — eight weeks at 5-7 hours each is enough to ship.
- Each week pairs a practice deliverable with a read of the real Alltag code.
- The concurrency and persistence weeks are where backend intuition misleads you most; do not rush them.
- The capstone — blank project to TestFlight — is where the learning compounds.

Next: the resources worth your time when this book runs out — see *Curated Resources*.

Curated Resources

This is a short, opinionated list — the resources worth your time, not an exhaustive link dump. Everything here is either canonical (Apple’s own) or has earned a durable reputation. Each entry says *why* it matters to someone with your background.

Read these first

Resource	What it is	Why for you
<i>The Swift Programming Language</i>	The official language book, free on-line	The authoritative reference; skim, then keep as a lookup
Apple SwiftUI Tutorials	Guided, hands-on projects	The fastest way to internalise the view-and-state loop
Human Interface Guidelines	Apple’s design rules	What reviewers expect; saves rejections
Swift API Design Guidelines	Naming and idiom conventions	Makes your code read like Swift, not ported Java

The Swift book lives at docs.swift.org; the tutorials and guidelines are on developer.apple.com. Bookmark all four.

When you are stuck on SwiftUI

- **Hacking with Swift** (Paul Hudson) — hackingwithswift.com. The “100 Days of SwiftUI” course and the searchable how-to library are the single best free SwiftUI resource. When you hit “how do I do X in SwiftUI,” this is the first search.
- **SwiftUI Field Guide** — swiftui-lab.com and the interactive swiftuifieldguide.com — for genuinely understanding the layout system when the three-step negotiation stops being intuitive.
- **Stack Overflow** — still useful, but check the date: pre-2023 SwiftUI answers often predate [@0b-servable](#), [NavigationStack](#), and [SwiftData](#).

When you want depth

- **WWDC sessions** (in the Developer app or on developer.apple.com/videos). The ones worth your time early: “Discover Observation in SwiftUI,” “Meet SwiftData,” “Meet Swift Testing,” “Migrate to Swift 6,” and any year’s “What’s new in SwiftUI.” Watch at 1.5x; they are dense and accurate.
- **Point-Free** (pointfree.co) — advanced, opinionated Swift on architecture, dependencies, and testing. Paid, and occasionally over-engineered for app work, but it will sharpen how you think about composition and side effects. Especially relevant given your interest in clean dependency injection.

- **Swift Forums** (forums.swift.org) — where the language is actually designed. Read the “Evolution” category to understand *why* Swift is shaped the way it is; as a language-design-literate engineer you will enjoy it.

Tools worth installing

Tool	Purpose	Note
XcodeGen	Project file from YAML	Already used by Alltag
xcbeautify	Readable <code>xcodebuild</code> logs	Pairs with <code>ci.sh</code>
SwiftLint	Style/lint enforcement	Like Checkstyle/PMD
swift-format	Auto-formatter	Like google-java-format
SF Symbols app	Browse system icons	Names like <code>house</code> , <code>folder</code>

`brew install xcodegen xcbeautify swiftlint swift-format`. The **SF Symbols** app is a free download from Apple and is how you find icon names like the ones in Alltag’s `AppTab` (`doc.text.view-finder` , `gearshape`).

Staying current

- **iOS Dev Weekly** (iosdevweekly.com) — a Friday newsletter, the canonical way to keep a finger on the ecosystem without doom-scrolling.
- **Swift by Sundell** (swiftbysundell.com) — articles and a podcast; reliably high quality on practical patterns.
- **Mastodon / the Swift community** — many core team members and prominent developers post there; a better signal-to-noise ratio than most feeds.

For your specific app

Alltag leans on a few areas with their own primary sources worth knowing:

Topic	Where to look
On-device encryption	Apple CryptoKit docs; the Keychain Services guide
Local LLM inference	The llama.cpp repository and its Metal backend
SwiftData modelling	“Meet SwiftData” + “Model your schema” WWDC sessions
Localization	“Discover String Catalogs” WWDC session
Privacy manifests	Apple’s “Privacy manifest files” documentation

A closing note

You have an advantage most people learning iOS do not: you already know how to build software that matters, reason about types and concurrency, and ship under constraints. What remained was a new language, a new UI paradigm, and an unfamiliar release pipeline. That is exactly what this book covered — and now you have both the map and a real app to practise on.

Build the capstone. Ship it. Then come back and re-read the chapter that gave you the most trouble; it will read completely differently from the other side of a shipped app.

Next: nothing — close the book and open Xcode.